# SYLLABUS

**Unit I: Introduction** **06 Hours**

Introduction to information retrieval, Major challenges in IR, Features of an IR system, components of an IR model, IR system block diagram, Boolean retrieval, Information versus Data Retrieval, Text categorization, IR processes and fields, Vector Model, Probabilistic Model and Latent Semantic Indexing Model.

**Case Studies:** Visualization and Information Retrieval Techniques for Network Intrusion Detection.

**Unit II: Dictionaries and Query Processing** **06 Hours**

Components of Index, Index Life Cycle, Static Inverted Index, Dictionaries-Types (Sort Based, Hash Based, Interleaving & Posting Lists), Index Construction (In memory, Sort Based, Merge Based, Disk Based Index Construction), Dynamic Indexing, Query Processing for Ranked Retrieval, Document at a Time Query Processing, Term at a Time Query Processing, Pre-computing Score Contributions, Impact Ordering, Query Optimization.

**Case Studies:** Matching of the searched statement with the database which is already stored.

**Unit III: Probabilistic Retrieval and Language Modelling Related Methods** **06 Hours**

**Probabilistic Retrieval:** Review of Basic Probability Theory; The Probability Ranking Principle: The 1/0 loss case, the PRP with retrieval costs; The Binary Independence Model; Term Frequency; An appraisal and some extensions: An appraisal of probabilistic models, tree-structured dependencies between terms, Okapi BM25: a non-binary model, Bayesian network approaches to IR, Relevance Feedback, Field Weights: BM25F.

**Language Models for Information Retrieval:** generating queries from documents; **Language Models:** finite automata and language models; Types of language models; Multinomial distributions over words; Ranking with Language Models; Divergence from Randomness, Passage Retrieval, and Ranking.

**Case Studies:** A Comparative Study of Probabilistic and Language Models for Information Retrieval.

**Unit IV: Text Classification and Text Clustering** **06 Hours**

**Text Classification:** Introduction to Text Classification, Naiyes Bayes Model, K Nearest neighbor, spam filtering, Support Vector Machine Classifier, Vector Space classification using hyperplanes, kernel function.

**Text Clustering:** Clustering vs Classification, partitioning methods. **Clustering Algorithms:** k-means clustering, Agglomerative hierarchical clustering, Expectation Maximization, Mixture of Gaussians Model.

**Case Studies:** Improving Document Organization and Retrieval in a Digital Library.

**Unit V: Web Retrieval and Web Crawling** **06 Hours**

**Parallel Information Retrieval:** Parallel query processing, Mapreduce.

**Web Retrieval:** Search Engine Architectures, Cluster based Architecture, Distributed Architectures, Search Engine Ranking. Link based Ranking, Page Ranking Algorithm, Simple Ranking Functions and Evaluations.

**Web Crawler:** Web Crawler structure, Web crawler libraries, Python Scrapy, Beautiful Soup, Applications.

**Case Studies:** Study of Google Map/ Geogusser information retrieval.

**Unit VI: IR Applications** **06 Hours**

**Multimedia Information Retrieval:** Spoken Language Audio Retrieval, Non-Speech Audio Retrieval, Graph Retrieval, Imagery Retrieval, Video Retrieval.

**Recommender System:** Collaborative Filtering, Content Based Recommendation, Knowledge Based Recommendation Information Extraction and Integration: Extracting Data from Text. Semantic Web, Collecting and Integrating Specialized Information on the web.

**Case Studies:** Demonstrate Collaborative filtering using any datasets to recommend items to users.

VIKAS

# CONTENTS


**Unit I: Introduction** 1.1-1.30

1.1 Introduction to Information Retrieval 1.1
1.2 Major Challenges in IR 1.1
1.3 Features of an IR System 1.2
1.4 IR System Block Diagram 1.4
1.5 Components of an IR Model 1.5
1.6 Boolean Retrieval 1.6
1.7 Information Versus Data Retrieval 1.7
1.8 Text Categorization 1.12
1.8.1 Naive Bayes (NB) Classifier 1.14
1.8.2 Support Vector Machines (SVM) 1.15
1.8.3 Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) 1.15
1.8.4 Decision Trees (DTs) 1.15
1.9 IR Processes and Fields 1.16
1.10 Vector Model 1.16
1.10.1 Components of the Vector Space Model 1.17
1.10.2 Vector Representation 1.18
1.10.3 Vector Space Model 1.18
1.11 Probabilistic Model 1.19
1.12 Latent Semantic Indexing 1.22
1.12.1 Linguistic and Knowledge-based Approaches 1.25
1.12.2 Latent Semantic Indexing Model 1.25
- **Case Study: Visualization and Information Retrieval Techniques for Network Intrusion Detection** 1.26
- **Exercise** 1.28

**Unit II: Dictionaries and Query Processing** 1.29

2.1 Components of Index 2.1
2.2 Index Life Cycle 2.1
2.3 Static Inverted Index 2.3
2.4 Dictionaries - Types 2.4
2.4.1 Sort Based Dictionary 2.5
2.4.2 Hash Based Dictionary 2.6
2.4.3 Interleaving Dictionary 2.6
2.4.4 Posting Lists Dictionary 2.8
2.5 Index Construction 2.9
2.5.1 Index Construction in Memory 2.9
2.5.2 Indexing-Time Dictionary 2.10
2.5.3 Sort Based Index Construction 2.10
2.5.4 Merge Based Index Construction 2.13
2.5.5 Disk Based Index Construction 2.15
2.6 Dynamic Indexing 2.17
2.7 Query Processing for Ranked Retrieval 2.18
2.8 Document at a Time Query Processing 2.20


VIKAS

2.9 Term at a Time Query Processing 2.22
2.10 Precomputing Score Contributions 2.24
2.11 Impact Ordering 2.26
2.12 Query Optimization 2.28
• **Case Study: Matching of the Searched Statement with the Database Which is Already Stored** 2.29
• **Exercise** 2.29
**Unit III: Probabilistic Retrieval and Language Modelling Related Methods** **3.1-3.20**
3.1 Probabilistic Retrieval 3.1
3.1.1 Review of Basic Probability Theory 3.1
3.2 The Probability Ranking Principle 3.2
3.2.1 The 1/0 Loss Case, the PRP with Retrieval Costs 3.2
3.2.2 The Binary Independence Model 3.3
3.2.3 Term Frequency 3.4
3.3 An Appraisal and Some Extensions 3.5
3.3.1 An Appraisal of Probabilistic Models 3.5
3.3.2 Tree-Structured Dependencies Between Terms 3.6
3.4 Okapi BM25 3.7
3.4.1 A Non-Binary Model 3.7
3.4.2 Bayesian Network Approaches to IR 3.8
3.4.3 Relevance Feedback 3.9
3.5 Field Weights: BM25F 3.9
3.6 Language Models for Information Retrieval 3.11
3.6.1 Generating Queries from Documents 3.11
3.7 Language Models 3.12
3.7.1 Finite Automata and Language Models 3.12
3.8 Types of Language Models 3.14
3.8.1 Multinomial Distributions Over Words 3.14
3.8.2 Ranking with Language Models 3.14
3.8.3 Divergence from Randomness 3.15
3.9 A Model of Randomness 3.17
3.9.1 Passage Retrieval and Ranking 3.17
• **Case Study: A Comparative Study of Probabilistic and Language Models for Information Retrieval** 3.19
• **Exercise** 3.20
**Unit IV: Text Classification and Text Clustering** **4.1-4.26**
4.1 Text Classification 4.1
4.1.1 Introduction to Text Classification 4.1
4.1.2 Naives Bayes Model 4.2
4.1.3 K-Nearest Neighbor 4.6
4.1.4 Spam Filtering 4.10
4.1.5 Support Vector Machine Classifier 4.11
4.1.6 Vector Space classification using Hyperplanes and Kernel Function 4.14
4.2 Text Clustering 4.16
4.2.1 Clustering vs Classification 4.16
4.2.2 Partitioning Methods 4.17


MKAS

4.3 Clustering Algorithms
4.3.1 K-Means Clustering
4.3.2 Agglomerative Hierarchical Clustering
4.3.3 Expectation Maximization
4.3.4 Mixture of Gaussians Model
- **Case Study: Improving Document Organization and Retrieval in a Digital Library**
- **Exercise**

**Unit V: Web Retrieval and Web Crawling** 5.1-5.40

5.1 Parallel Information Retrieval
5.1.1 Parallel Query Processing
5.1.2 Mapreduce
5.2 Web Retrieval
5.2.1 Search Engine Architectures
5.2.2 Cluster Based Architecture
5.2.3 Distributed Architectures
5.2.4 Search Engine Ranking
5.2.5 Link Based Ranking
5.3 Page Ranking Algorithm
5.3.1 Simple Ranking Functions and Evaluations
5.4 Web Crawler
5.4.1 Web Crawler Structure
5.4.2 Web Crawler Libraries
5.4.3 Beautiful Soup
5.4.4 Applications of Web Crawlers
- **Case Study: Study of Google Map / Geogusser Information Retrieval**
- **Exercise**

**Unit VI: IR Applications** 6.1-6.28

6.1 Multimedia Information Retrieval
6.1.1 Spoken Language Audio Retrieval
6.1.2 Non-Speech Audio Retrieval
6.1.3 Graph Retrieval
6.1.4 Imagery Retrieval
6.1.5 Video Retrieval Recommender System
6.2 Recommendation Systems
6.2.1 Collaborative Filtering
6.2.2 Content Based Recommendation
6.3 Knowledge Based Recommendation Information Extraction and Integration
6.3.1 Extracting Data from Text
6.4 Semantic Web
6.5 Collecting and Integrating Specialized Information on the Web
- **Case Study: Demonstrate Collaborative Filtering Using Any Datasets to Recommend Items to Users**
- **Exercise**

- **Model Question Papers**
  - **In-Sem. Exam. (30 Marks)** P.1-P.
  - **End-Sem. Exam. (70 Marks)** P.1-P.

✠ ✠ ✠

VIKAS

UNIT - III

# PROBABILISTIC RETRIEVAL AND LANGUAGE MODELLING RELATED METHODS

## 3.1 PROBABILISTIC RETRIEVAL

- In this chapter and the next we examine a number of IR models that have had substantial influence on both research and practice. In this chapter we present what is known as the probabilistic model. In the context of information retrieval, the word "model" has at least two distinct and important meanings. In one sense it means "an abstraction of the retrieval task itself" (e.g., the vector space model). As we shall see, it may also be used in the sense of "statistical model", in reference to models of relevance or models of document content.
- Although many IR models might be described as probabilistic, the term is usually associated with the methods arising from the work of Stephen Robertson, Karen Sp¨arck Jones, and their colleagues at the City University London, at the University of Cambridge, and at Microsoft Research, Cambridge, beginning in the early 1970s.
- Over that period they developed the probabilistic retrieval model through a series of innovations that have led to its current status as one of the most successful IR models. The model has been adopted by other research groups who have incorporated its retrieval formulae into their experimental IR systems and have often proposed and evaluated extensions of their own. The organization of this chapter closely follows the historical development of the model, highlighting each significant extension and innovation in turn. A substantial portion of the chapter builds toward Equation, the BM25 formula, which currently represents one of the best performing (and best-known) retrieval formulae within the field.
- Although the BM25 formula may be used without understanding the details of its development, we cover this development in considerable detail not only for its own sake but also as a worked example of the reasoning processes underlying much of IR theory. The chapter concludes with an overview of relevance feedback, a query expansion technique closely associated with the probabilistic and vector space models. We also introduce an important extension to the BM25 formula, known as BM25F, which applies differential weights to terms appearing in different document elements, thus recognizing that the presence of a query term in a document's title may be a stronger indicator of relevance than that term's presence in the document's body.

### 3.1.1 Review of Basic Probability Theory

- The probabilistic model takes as its starting point the Probability Ranking Principle: If an IR system's response to each query is a ranking of the documents in the collection in order of decreasing probability of relevance, the overall effectiveness of the system to its user will be maximized.
- Probabilistic retrieval is an information retrieval approach that uses probabilistic models to rank and retrieve documents based on their relevance to a user's query. Before delving into probabilistic retrieval, let us review some basic concepts of probability theory that are relevant to this approach:
  - ➢ **Sample Space (Ω):** The sample space represents the set of all possible outcomes of an experiment or event. For example, when flipping a fair coin, the sample space is {Heads, Tails}.
  - ➢ **Event (E):** An event is a subset of the sample space, representing a specific outcome or combination of outcomes. For instance, getting "Heads" when flipping a coin is an event denoted by E = {Heads}.
  - ➢ **Probability (P):** The probability of an event E, denoted by P(E), is a numerical value between 0 and 1 that indicates the likelihood of that event occurring. A probability of 0 means the event is impossible, while a probability of 1 means the event is certain.



VIKAS

- **Probability Distribution:** A probability distribution assigns probabilities to each possible outcome in the sample space. For example, for a fair six-sided die, the probability distribution would be P(1) = 1/6, P(2) = 1/6, ..., P(6) = 1/6.
- **Joint Probability (P(A ∩ B)):** The joint probability of events A and B, denoted by P(A ∩ B), represents the probability of both events A and B occurring simultaneously.
- **Conditional Probability (P(A | B)):** The conditional probability of event A given event B, denoted by P(A | B), represents the probability of event A occurring, given that event B has already occurred.
- **Independence of Events:** Two events A and B are independent if the occurrence of one event does not affect the probability of the other event. Mathematically, P(A ∩ B) = P(A) * P(B).
- **Bayes' Theorem:** Bayes' theorem allows us to update the probability of an event based on new evidence. It is given by P(A | B) = (P(B | A) * P(A)) / P(B), where P(A | B) is the conditional probability of event A given B.
- **Random Variables:** A random variable is a function that assigns numerical values to each outcome in the sample space. It allows us to calculate probabilities for different events based on their assigned values.
- **Expected Value (Mean):** The expected value of a random variable X, denoted by E(X) or μ, represents the average value or mean of the random variable based on its probability distribution.
- **Variance ($\sigma^2$) and Standard Deviation ($\sigma$):** Variance measures the spread or dispersion of the values of a random variable around its mean. Standard deviation is the square root of variance.

## 3.2 THE PROBABILITY RANKING PRINCIPLE

### 3.2.1 The 1/0 Loss Case, the PRP with Retrieval Costs

- What is the probability that a user will judge this document relevant to this query? We now turn to the problem of estimating this probability. Following the approach taken by Lafferty and Zhai in their discussion of the probabilistic model, we translate the Basic Question into statistical notation by introducing three random variables: D for documents, Q for queries, and the binary random variable R for the user's judgment of relevance. For the random variable D you might view its sample space, or set of outcomes, as a collection of documents indexed by a search engine.
- You might then imagine a group of users typing queries into the same search engine, with the sample space for Q being any query that the search engine allows a user to express. The engine might require these queries to be term vectors, but for now we do not assume this limitation. Finally, the binary random variable R takes on the value either 1 or 0, relevant or not relevant. Using this notation, we translate the Basic Question into the problem of estimating the probability of a positive relevance judgment, which we express as follows:

$$p(R = 1 \mid D = d, Q = q)$$

- This formula may be viewed as a probability mass function over documents and queries. As a notational convenience consistent with the practice in the research literature, we often simply write "D" to stand for "D = d" and "Q" to stand for "Q = q"; this gives

$$p(R = 1 \mid D, Q)$$

- Similarly, we write "r" to mean "R = 1" and "$\bar{r}$" to mean "R = 0". Thus, we have

$$p(r \mid D, Q) = 1 - p(\bar{r} \mid D, Q)$$

- Bayes' Theorem, a fundamental theorem in probability theory, states that

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)}$$

Applying Bayes' Theorem gives

$$p(r \mid D, Q) = \frac{p(D, Q \mid r)\, p(r)}{p(D, Q)}, \text{ and}$$

$$p(\bar{r} \mid D, Q) = \frac{p(D, Q \mid \bar{r})\, p(\bar{r})}{p(D, Q)}.$$

- Now, rather than continue to work directly with probabilities, we switch to a log-odds, or logit, formulation that will simplify the presentation and manipulation of equations. Given a probability p, the logit of p is defined as

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right),$$

where the base of the logarithm may be chosen arbitrarily. Continuing the practice we follow throughout the book when choosing a specific base for an example or experiment, we choose a base of 2.



- Log-odds has a number of useful properties. As p varies from 0 to 1, logit(p) varies from $-\infty$ to $\infty$. If the odds are even (i.e., p = 0.5), then logit(p) = 0. Given two probabilities p and q, logit(p) > logit(q) if and only if p>q. Thus log-odds and probability are rank-equivalent ranking by one produces the same ordering as ranking by the other. Rank-equivalent transformations such as this one (also called rank-preserving or order-preserving transformations) represent a valuable tool for simplifying the development of IR models. Taking the log-odds of equation and applying Bayes' Theorem gives the following:

$$\log\frac{p(r \mid D, Q)}{1 - p(r \mid D, Q)} = \log\frac{p(r \mid D, Q)}{p(\bar{r} \mid D, Q)} \quad \ldots (3.1)$$

$$= \log\frac{p(D, Q \mid r)\, p(r)}{p(D, Q \mid \bar{r})\, p(\bar{r})} \quad \ldots (3.2)$$

- An immediate benefit of switching to log-odds is that the term p(D, Q) cancels out, and does not need to be estimated. We may expand the joint probabilities in Eq. (3.1) into conditional probabilities using the equality p(D, Q | R) = p(D | Q, R) · p(Q | R). Expanding the joint probabilities in this way and applying Bayes' Theorem a second time gives

$$\log\frac{p(D, Q \mid r)\, p(r)}{p(D, Q \mid \bar{r})\, p(\bar{r})} = \log\frac{p(D \mid Q, r)\, p(Q \mid r)\, p(r)}{p(D \mid Q, \bar{r})\, p(Q \mid \bar{r})\, p(\bar{r})} \quad \ldots (3.3)$$

$$= \log\frac{p(D \mid Q, r)\, p(r \mid Q)}{p(D \mid Q, \bar{r})\, p(\bar{r}) \mid Q} \quad \ldots (3.4)$$

$$= \log\frac{p(D \mid Q, r)}{p(D \mid Q, \bar{r})} + \log\frac{p(r \mid Q)}{p(\bar{r} \mid Q)} \quad \ldots (3.5)$$

- The term "log (p(r | Q)/p($\bar{r}$ | Q))" is independent of D. You might consider it an indicator of query difficulty. In any case it may be safely ignored for ranking purposes another order preserving transform leaving us with the ranking formula

$$\log\frac{p(D \mid Q, r)}{p(D \mid Q, \bar{r})} \quad \ldots (3.6)$$

- This formula sits at the heart of the probabilistic retrieval model, and we will return to it several times as we examine various methods for estimating its value.

### 3.2.2 The Binary Independence Model

- For our first attempt to estimate a value for Eq. (3.6), we consider only the presence and absence of terms in documents and queries. We re-define the random variable $D_i$ representing documents, into a vector of binary random variables $D = D_1, D_2, \ldots$ with one dimension for each term in the vocabulary V and with $D_i = 1$ indicating term presence and $D_i = 0$ indicating term absence. Similarly, we refine the random variable Q, representing queries, into a vector of binary random variables $Q = Q_1, Q_2, \ldots$ with $Q_i = 1$ indicating term presence and $Q_i = 0$ indicating term absence.
- We now make two strong assumptions. The first is an independence assumption:

  **Assumption T:** Given relevance, terms are statistically independent.
- In other words, given a positive relevance judgment, the presence or absence of one term does not depend on the presence or absence of any other terms. Similarly, given a negative relevance judgment, the presence or absence of one term does not depend on the presence or absence of any other terms. Naturally, if we do not condition on relevance, terms are not independent because there are usually multiple terms for which the probability of presence depends on relevance. This assumption does not accurately reflect reality. For example, the presence of the term "shakespeare" in a document increases the probability that the terms "william", "hamlet", "stratford" and so on will also occur in the document. Nonetheless, the assumption does greatly simplify the problem of estimating a value for Eq. (3.6). Independence assumptions such as this one are common in information retrieval. Methods developed under these assumptions often provide good performance despite their unrealistic nature. As a result of this assumption we may rewrite the probabilities appearing in Eq. (3.6) as the product of probabilities for the individual random variables associated with the dimensions of D:

$$p(D \mid Q, r) = \prod_{i=1}^{M} p(D_i \mid Q, r) \quad \ldots (3.7)$$

$$p(D \mid Q, \bar{r}) = \prod_{i=1}^{M} p(D_i \mid Q, \bar{r}). \quad \ldots (3.8)$$

VIKAS

Eq. (3.6) then becomes

$$\log \frac{p(D \mid Q, r)}{p(D \mid Q, \bar{r})} = \sum_{i=1}^{M} \log \frac{p(D_i \mid Q, r)}{p(D_i \mid Q, \bar{r})} \quad \ldots (3.9)$$

- Our second strong assumption links the appearance of a term in the query to the probability of its appearance in a relevant document, thus clarifying the role of the query as a bridge between a user's need and a document's relevance. Assumption Q. The presence of a term in a document depends on relevance only when that term is present in the query. To formalize this assumption we need to fix the query, giving Q = q = $q_1, q_2, \ldots$, where each $q_i$ is either 0 or 1. As a result of Assumption Q, if $q_t = 0$ then $p(D_t \mid Q, r) = p(D_t \mid Q, \bar{r})$, and therefore $\log \frac{p(D_t \mid Q, r)}{p(D_t \mid Q, \bar{r})} = 0$
- The effect of this assumption is to change the summation in Eq. (3.9) from a summation over all terms in the vocabulary to a summation over all terms in the query. Because conditioning on the query is now redundant, we can drop it and our ranking formula then becomes

$$\sum_{t \in q} \log \frac{p(D_t \mid r)}{p(D_t \mid \bar{r})}, \quad \ldots (3.10)$$

where $D_t$ is the random variable corresponding to term t in the vector $d_1, d_2, \ldots$.

- Like Assumption T, Assumption Q does not accurately reflect reality. If we are interested in the marriage of William Shakespeare and enter the query "william", "shakespeare", "marriage", a relevant document is much more likely to contain the term "hathaway" than one that is non-relevant, even though this term does not appear in the query. Nonetheless, from a practical standpoint a summation over query terms appears more tractable than a summation over all terms in the vocabulary, and we greet this development with a certain amount of relief. We now fix D = d = $d_1, d_2, \ldots$, where each $d_i$ is either 0 or 1, thus making explicit the values of the random variables. Extending our earlier notation, we then write $D_t = d_t$ to represent the value of the random variable from $d_1, d_2, \ldots$ corresponding to the term t.

$$\sum \log \frac{p(D_t = d_t \mid r)}{p(D_t = d_t \mid \bar{r})} \quad \ldots (3.11)$$

- We then subtract from Eq. (3.11) its own value when no query terms appear in the document and all Dt are 0.

$$\sum_{t \in q} \log \frac{p(D_t = d_t \mid r)}{p(D_t = d_t \mid \bar{r})} - \sum_{t \in q} \log \frac{p(D_t = 0 \mid r)}{p(D_t = 0 \mid \bar{r})} \quad \ldots (3.12)$$

- Because the value of Eq. (3.11) when all query terms are absent is a constant, and because the subtraction of a constant value has no impact on ranking, the subtraction of this value represents another order-preserving transform. A little rearranging then gives:

$$\sum_{t \in (q \cap d)} \log \frac{p(D_t = 1 \mid r)\, p(D_t = 0 \mid \bar{r})}{p(D_t = 1 \mid \bar{r})\, p(D_t = 0 \mid r)} - \sum_{t \in (q - d)} \log \frac{p(D_t = 0 \mid r)\, p(D_t = 0 \mid \bar{r})}{p(D_t = 0 \mid \bar{r})\, p(D_t = 0 \mid r)} \quad \ldots (3.12)$$

where the summation on the left is over all terms appearing in both the query and the document, and the summation on the right is over terms appearing in the query but not in the document. Because the term on the right is 0, the equation becomes

$$\sum_{t \in (q \cap d)} \log \frac{p(D_t = 1 \mid r)\, p(D_t = 0 \mid \bar{r})}{p(D_t = 1 \mid \bar{r})\, p(D_t = 0 \mid r)}. \quad \ldots (3.13)$$

- This refinement of Eq. (3.6) under assumptions T and Q, and considering only the presence and absence of terms, is known as the Binary Independence Model.

### 3.2.3 Term Frequency

- We list a number of simple document features used by basic ranking methods to compare documents and determine an appropriate ranking. These features include term frequency, term proximity, and document length. Thus far, our development of the probabilistic model has considered only term presence, essentially a weakened form of term frequency. In order to extend the model to fully accommodate term frequency, we must return to Eq. (3.6) and reconsider our representation of documents by the random variable D.
- We refined our definition of the random variable D into a vector of binary random variables $D = D_1, D_2, \ldots$, where each random variable $D_i$ represents the presence or absence of a specific term. Returning to this point, we can retrace our steps with a revised definition for D as a vector of random variables $D = F_1, F_2, \ldots$, where each $F_i$ represents the within-document term frequency for the corresponding term. As we did in that section, we also define Ft as notation for the random variable from this vector corresponding to the term t.



- Leaving Assumptions T and Q unchanged, and making minor adjustments to the argument in order to accommodate our switch from term presence to term frequency, brings us to the following analogue of Eq. (3.13):

$$\sum_{t \in q} \log \frac{p(F_t = f_t \mid r)\, p(F_t = 0 \mid \bar{r})}{p(F_t = f_t \mid \bar{r})\, p(F_t = 0 \mid r)}, \quad \ldots (3.14)$$

here $f_t$ represents the number of times t appears in the document. For this equation to be usable we must estimate values for $p(F_t = f_t \mid r)$ and $p(F_t = f_t \mid \bar{r})$, the term frequency probabilities given relevance and non-relevance. In order to estimate these values, we must in turn consider the relationship between term frequency and relevance.

- When writing a document about a particular topic, an author will generally choose terms related to that topic. As a result, when a term is associated with a particular topic, we might expect that term to appear more frequently in a document about the topic than in a document not about the topic. On the other hand that term will still occasionally appear in documents not related to the topic
- Bookstein and Swanson's attempt to capture this relationship between topics and terms with a notion that Robertson et al and Robertson and Walker call eliteness. A document is said to be elite in term t when it is somehow "about" the topic associated with the term. By turning this relationship around it may thus be possible for us to infer a probability of eliteness from term frequency the greater the term frequency, the more likely that the term is elite in the document. Thus, the relationship between relevance and term frequency flows through this notion of eliteness. Roughly, documents about a topic are more likely to be elite in the terms related to that topic and therefore these terms are more likely to appear in these documents.
- For each term we define a binary random variable Et corresponding to each Ft and representing eliteness of the term t. $E_t = 1$ indicates that a document is elite in t, and $E_t = 0$ indicates that the document is not elite in t. Much as we did for relevance, we define e as shorthand for $E_t = 1$ and $\bar{e}$ as shorthand for $E_t = 0$. We omit the subscript t in this shorthand notation because the term being referenced is usually obvious. The relationship between relevance and term frequency may then be formalized as follows:

$p(F_t = f_t \mid r) = p(F_t = f_t \mid e) \cdot p(e \mid r) + p(F_t = f_t \mid \bar{e}) \cdot p(\bar{e} \mid r)$ and

$p(F_t = f_t \mid \bar{r}) = p(F_t = f_t \mid e) \cdot p(e \mid \bar{r}) + p(F_t = f_t \mid \bar{e}) \cdot p(\bar{e} \mid \bar{r}) \ldots (3.15)$

Substituting these expansions into the term of Eq. 3.14 gives

$$\sum_{t \in q} \log \frac{(p(F_t = f_t \mid e)p(e \mid r) + p(F_t = f_t \mid \bar{e})p(\bar{e} \mid r))\,(p(F_t = 0 \mid e)p(e \mid \bar{r}) + p(F_t = 0 \mid \bar{e})p(\bar{e} \mid \bar{r}))}{(p(F_t = f_t \mid e)p(e \mid \bar{r}) + p(F_t = f_t \mid \bar{e})p(\bar{e} \mid \bar{r}))\,(p(F_t = 0 \mid e)p(e \mid r) + p(F_t = 0 \mid \bar{e})p(\bar{e} \mid r))}$$

## 3.3 AN APPRAISAL AND SOME EXTENSIONS

### 3.3.1 An Appraisal of Probabilistic Models

- Probabilistic models play a crucial role in Information Retrieval (IR) systems for ranking and retrieving relevant documents based on their likelihood of being relevant to a user's query. Here is an appraisal of probabilistic models used in IR:

**Advantages**

- **Probabilistic Framework:** Probabilistic models provide a sound theoretical foundation for ranking documents based on probabilities, making it easier to interpret and understand the ranking results.
- **Relevance Ranking:** Probabilistic models directly estimate the probability of relevance for documents, enabling more accurate ranking compared to traditional boolean retrieval models.
- **Term Weights:** These models allow for the incorporation of term weights, such as Inverse Document Frequency (IDF) and Term Frequency (TF), which help capture the importance of terms in documents and queries.
- **Query Expansion:** Probabilistic models can be extended to support query expansion techniques, improving retrieval effectiveness by including synonymous terms.
- **Effectiveness for Long Queries:** In probabilistic models, longer queries tend to perform better than in boolean models, as they allow for richer term weighting and relevance estimation.

VIKAS

**Disadvantages**

- **Complexity:** Probabilistic models can be mathematically complex, requiring extensive calculations to estimate relevance probabilities.
- **Parameter Tuning:** The effectiveness of probabilistic models often depends on the proper tuning of parameters, which can be challenging.
- **Query Ambiguity:** Probabilistic models may struggle to handle ambiguous queries where the user's intent is not clear, leading to less accurate results.
- **Vocabulary Mismatch:** These models might suffer from the vocabulary mismatch problem, where relevant documents may not contain all query terms due to differences in phrasing or language.
- **Data Sparsity:** In cases of sparse data, especially for rare terms, probabilistic models might struggle to accurately estimate term probabilities.

1. **Common Probabilistic Models in IR**

- **Binary Independence Model (BIM):** Assumes that terms are conditionally independent given relevance. It is simple and computationally efficient but may suffer from the independence assumption.
- **Okapi BM25 Model:** A widely used probabilistic model that addresses the shortcomings of BIM by incorporating term frequency saturation and document length normalization. It often outperforms other models in IR tasks.
- **Language Models (LM):** Treat documents and queries as language models and estimate the likelihood of generating the query from the document language model. LM-based models can be effective but require smoothing techniques to handle zero probabilities.
- **Divergence from Randomness (DFR):** Uses the concept of divergence from randomness to estimate relevance. DFR models offer good retrieval performance and flexibility in term weighting.

2. **Applicability**

- Probabilistic models are well-suited for ad hoc retrieval tasks, where users submit one-time queries and expect relevant documents from a large collection. They are effective in web search engines, document databases, and information retrieval systems with large-scale document collections.
- In conclusion, probabilistic models have significantly advanced the field of Information Retrieval by providing robust methods for ranking documents based on their relevance to user queries. Their effectiveness and interpretability make them popular choices in modern IR systems, though they require careful parameter tuning and handling of query ambiguity and data sparsity to achieve optimal retrieval performance.

### 3.3.2 Tree-Structured Dependencies Between Terms

Tree-structured dependencies between terms refer to a hierarchical relationship among terms in an Information Retrieval (IR) system. In this context, terms can represent query keywords, document content, or any textual information that needs to be organized based on their relationships. The tree structure implies that the terms are organized in a parent-child relationship, forming a tree-like hierarchy.

- **Hierarchical Concept Representation:** One common application of tree-structured dependencies is in representing hierarchical concepts. In IR, terms can be grouped into higher-level concepts, forming a tree where parent terms represent broader concepts, and child terms represent more specific sub-concepts. This hierarchical representation allows for more efficient query processing and can improve the accuracy of document ranking.
- **Taxonomy or Ontology Construction:** Tree-structured dependencies are often used to construct taxonomies or ontologies, which are formal representations of the relationships between terms in a specific domain. Taxonomies organize terms hierarchically based on their semantic relationships, while ontologies also capture additional properties and relationships, allowing for more sophisticated knowledge representation.
- **Query Expansion:** In query expansion, tree-structured dependencies can be leveraged to include related terms in the user's query. By traversing the tree and adding child terms to the original query, the system can broaden the search scope and retrieve more relevant documents.
- **Synonym and Hyponym Identification:** By organizing terms in a tree structure, the system can identify synonymous terms (different terms with similar meanings) and hyponyms (more specific terms related to a broader term). This information can be used for query expansion, disambiguation, and better document matching.



- **Faceted Search and Browsing:** In faceted search and browsing, tree-structured dependencies allow for the creation of facets (categories or attributes) that organize search results based on different dimensions. Users can navigate through the facets to filter and refine their search results effectively.
- **Entity Extraction and Relation Discovery:** The hierarchical relationships between terms can aid in entity extraction, where named entities are identified and linked to their corresponding categories or types. Additionally, these relationships can help discover associations or relations between different entities.
- **Semantic Analysis and Knowledge Graphs:** By analyzing the tree-structured dependencies between terms, IR systems can perform semantic analysis to understand the context and meaning of documents better. This understanding can be further integrated into knowledge graphs, which represent knowledge in a structured and interconnected manner.

## 3.4 OKAPI BM25

### 3.4.1 A Non-Binary Model

- Okapi BM25 is a popular probabilistic retrieval model used in Information Retrieval (IR) systems to rank documents based on their relevance to a user's query. It is an improved version of the Binary Independence Model (BIM) and is named after the British Museum in London, where the model was developed in the 1980s. BM25 is widely used in modern search engines and document retrieval systems due to its effectiveness and simplicity. Here is an overview of the Okapi BM25 model:

1. **Term Frequency and Inverse Document Frequency:** BM25 calculates the relevance score of a document for a given query by considering two important factors:

   (i) **Term Frequency (TF):** The number of times a query term appears in the document. Documents with higher term frequencies are considered more relevant to the query.

   (ii) **Inverse Document Frequency (IDF):** A measure of how rare a term is in the entire document collection. Rare terms receive higher IDF scores, indicating their importance in distinguishing relevant documents from non-relevant ones.

2. **BM25 Ranking Formula:** The relevance score (BM25 score) of a document D for a query Q is calculated using the following formula:

$$BM25(D, Q) = \Sigma[IDF(q) \times (TF(q, D) \times (k1 + 1)) / (TF(q, D) + k1 \times (1 - b + b \times (|D| / avg_doc_length)))]$$

   Where:

   IDF(q) is the inverse document frequency of the query term q.

   TF(q, D) is the term frequency of the query term q in document D.

   |D| is the length (number of terms) of the document D.

   avg_doc_length is the average document length in the document collection.

   k1 and b are tuning parameters. k1 controls the saturation point of term frequency, while b adjusts the impact of document length normalization.

3. **Key Characteristics:**

- BM25 introduces document length normalization (using parameter b) to address the issue of longer documents having potentially higher term frequencies.
- The tuning parameter k1 controls the term frequency saturation point. Higher values of k1 increase the sensitivity to term frequency, while lower values make the model less sensitive to term frequency variations.
- BM25 is relatively robust and does not suffer from the "zero-frequency" problem that can occur in language modeling approaches.

**Advantages**

- BM25 has been found to perform well in various IR tasks, including ad hoc retrieval and web search, and is considered one of the state-of-the-art retrieval models.
- It effectively handles term frequency saturation and document length normalization, making it suitable for ranking documents with varying lengths.
- The model is intuitive and straightforward to implement.

**Disadvantages**

- The model still relies on the independence assumption for terms, which may not hold true in certain scenarios.
- BM25 requires tuning of the parameters k1 and b, which can impact the retrieval performance, and the optimal values may vary depending on the dataset.

### 3.4.2 Bayesian Network Approaches to IR

- Turtle and Croft introduced into information retrieval the use of Bayesian networks, a form of probabilistic graphical model. We skip the details because fully introducing the formalism of Bayesian networks would require much too much space, but conceptually, Bayesian networks use directed graphs to show probabilistic dependencies between variables, and have led to the development of sophisticated algorithms for propagating influence so as to allow learning and inference with arbitrary knowledge within arbitrary directed acyclic graphs. Turtle and Croft used a sophisticated network to better model the complex dependencies between a document and a user's information need.
- The model decomposes into two parts: a document collection network and a query network. The document collection network is large, but can be precomputed: it maps from documents to terms to concepts. The concepts are a thesaurus-based expansion of the terms appearing in the document. The query network is relatively small but a new network needs to be built each time a query comes in, and then attached to the document network. The query network maps from query terms, to query sub-expressions (built using probabilistic or "noisy" versions of AND and OR operators), to the user's information need.
- The result is a flexible probabilistic network which can generalize various simpler Boolean and probabilistic models. Indeed, this is the primary case of a statistical ranked retrieval model that naturally supports structured query operators. The system allowed efficient large-scale retrieval, and was the basis of the In Query text retrieval system, built at the University of Massachusetts. This system performed very well in TREC evaluations and for a time was sold commercially. On the other hand, the model still used various approximations and independence assumptions to make parameter estimation and computation possible. There has not been much follow-on work along these lines, but we would note that this model was actually built very early on in the modern era of using Bayesian networks, and there have been many subsequent developments in the theory, and the time is perhaps right for a new generation of Bayesian network-based information retrieval systems.
- Bayesian network approaches have been applied to Information Retrieval (IR) as a way to model and represent the relationships between query terms, documents, and relevance. Bayesian networks are probabilistic graphical models that use Directed Acyclic Graphs (DAGs) to model dependencies between random variables. In the context of IR, Bayesian networks provide a structured and probabilistic framework to estimate the relevance of documents to a given query. Here is an overview of how Bayesian network approaches are applied to IR:
  - **Graphical Representation:** In a Bayesian network for IR, nodes in the graph represent random variables, and directed edges indicate conditional dependencies. The nodes in the graph typically include query terms, document content, relevance scores, and other contextual information.
  - **Probability Estimation:** Bayesian networks use probability distributions to model the relationships between the nodes. Conditional probability tables (CPTs) specify the probabilities for each node given its parent nodes in the graph. These probabilities are often estimated from training data using statistical techniques like maximum likelihood estimation.
  - **Query-Document Relevance Estimation:** Bayesian networks can be used to estimate the relevance of a document to a given query. The query terms are treated as observed evidence, and the relevance score of the document is treated as the target variable to be estimated.
  - **Learning from Data:** Bayesian networks can be learned from data, using techniques like the Expectation-Maximization (EM) algorithm or Bayesian parameter estimation. Learning the network structure and CPTs from data can adapt the model to specific domains and improve its performance.
  - **Query Expansion:** Bayesian networks can aid in query expansion by capturing dependencies between query terms. By traversing the network, related terms can be identified and added to the original query, leading to more comprehensive and effective retrieval.
  - **Relevance Feedback:** Bayesian networks can be used to incorporate relevance feedback from users to improve the retrieval results. User feedback on the relevance of retrieved documents can be incorporated as additional evidence to update the network's probabilities.
  - **Semantic Relations:** Bayesian networks can capture semantic relationships between terms, such as synonymy, hyponymy, and hypernymy, which can enhance retrieval accuracy by considering related terms during retrieval.
  - **Query Disambiguation:** Bayesian networks can aid in query disambiguation by leveraging contextual information and dependencies between query terms. This helps to resolve ambiguity and retrieve documents that match the user's intended meaning.
  - **Uncertainty Handling:** One of the key advantages of Bayesian networks is their ability to handle uncertainty naturally. By explicitly modeling probabilities, Bayesian networks can account for uncertainty in relevance estimation and document ranking.
- Overall, Bayesian network approaches to IR offer a powerful and flexible framework for modeling dependencies between query terms and documents, capturing semantic relationships, handling uncertainty, and incorporating user feedback. They have been successfully applied in various IR tasks, including ad hoc retrieval, query expansion, and relevance feedback, and continue to be an active area of research in the field of Information Retrieval.



### 3.4.3 Relevance Feedback

- The availability of relevance information may improve our ability to estimate a value for $w_t$ and consequently may improve retrieval performance. In one scenario for obtaining this information, the IR system conducts an initial search, presents the results to the user, and allows her to identify a number of relevant documents from these results. Even if these relevance judgments are few in number, they may be used to set values for $n_t$ and $n_{t,r}$ in Eq. (3.15 (a)).

$$w_t = \frac{(n_{t,r} + 0.5)\,(N - N_t - n_t + n_{t,r} + 0.5)}{(n_t - n_{t,r} + 0.5)\,(N_t - n_{t,r} + 0.5)} \quad \ldots (3.15\ (a))$$

- A final search is then conducted using these refined weights. Perhaps more significantly, the system may also use these relevance judgments to obtain additional information on the nature of relevant documents. For example, we might select appropriate terms from the identified documents and add them to the query. This expanded query would then be used for the second and final search. Query expansion techniques such as this one implicitly recognize the limitations of Assumption Q: that the presence of a term in a document can depend on relevance even when that term does not appear in the query. Leaving aside for the moment the question of term selection and assuming only that we have some method for scoring terms, the basic relevance feedback process proceeds as follows:
  - Execute the user's initial query.
  - Present the retrieval results to the user, allowing her to browse the results and indicate relevant documents.
  - Score the terms appearing in the relevant documents and select the top m expansion terms, ignoring terms in the initial query.
  - Add the new terms to the initial query, adjust weights $w_t$, and execute the expanded query.
  - Present the final retrieval results to the user.
- The adjusted weight for term t in the fourth step is determined with $n_t$ equal to the number of relevant documents identified by the user and $n_{t,r}$ equal to the number of these documents containing t. In addition it is not unusual to accord special status to the terms appearing in the initial query, recognizing that these may best reflect the user's information need. This additional adjustment may be effected by multiplying the weight of the expansion terms by a constant factor $\gamma$, where $\gamma = 1/3$ is typical.
- Relevance feedback has been the subject of a substantial body of research, and the basic feedback process may be extended and modified in a number of ways. For example the user might be allowed to identify non-relevant documents, to interactively adjust the list of selected terms, or to repeat the feedback process multiple times, adding and removing terms as she sees fit. In addition, various methods have been proposed for scoring terms.

## 3.5 FIELD WEIGHTS: BM25F

- Along with a list of basic features for ranking such as term frequency and term proximity, we listed a number of additional features usable for ranking, including document structure. For example, terms appearing in certain fields, such as a document's title, may be given more weight than terms appearing in the document's

body. In this section we present an important extension to BM25, known as BM25F that takes this document structure into account Exploiting document structure is particularly important in the context of Web search. Following program illustrates some of the rich structure available to a Web search engine. Following program is a greatly simplified version of the Web page en.wikipedia.org/wiki/Shakespeare as it appeared in the English-language Wikipedia in mid-2007.

- The HTML is split into two parts: (1) the header, delimited by ... tags, which provide a title and other metadata describing the document; and (2) the body, which provides the content of the article for display. The metadata includes important dates related to Shakespeare and annotations related to the organization of Wikipedia. For example, the keyword Persondata indicates that the page contains a standardized table of biographical information. The body of the document includes a section title, which is indicated by tags, as well as text to be displayed in a bold font, which is indicated by tags.
- The body also contains hypertext links to the Wikipedia pages for "poet" and "playwright". The anchor text between the start tag and the end tag of these links can be interpreted as indicating something about the content of the referenced pages, in this case simply restating their titles.
- Although HTML is particularly rich in structure, many documents contain similar fields. For example, the "Subject", "To", and "From" fields in e-mail messages may be exploited for similar purposes. The goal of BM25F is to appropriately reflect the emphasis implied by this structure.

**Program:** A simple HTML page showing typical structure on the Web.

```
<html>
  <head>
    <title>William Shakespeare - Wikipedia, the free encyclopedia</title>
    <meta
      name="keywords"
      content="William Shakespeare, Persondata, Sister projects, Earlybard, 1582, 1583, 1585, 1616"/>
  </head>
  <h1>William Shakespeare</h1>
  <b>William Shakespeare</b> (baptised 26 April 1564- died 23 April 1616) was an english <a href=http://en.wikipedia.org/wiki/Poet">poet</a> and <a href="http://en.wikipedia.org/wiki/Playwright">, playwright</a>.
  He is widely regarded as the greatest writer of the English language and the world's pre-eminent damatist. He wrote approximately 38 plays and 154 sonnets, as well as a variety of other poems...
  </body>
</html>
```

- The intuition underlying BM25F is similar to the intuition underlying the document length normalization. That equation specifies an adjusted term frequency, scaled to account for document length, that replaces the actual term frequency in the retrieval formula. We may extend this idea to field weights by computing an adjusted term frequency that depends on the field in which a term occurs. For example, we might consider the occurrence of a query term in the title of a document to be equivalent to ten occurrences in the document and scale the term frequency accordingly
- To formalize this notion we define $f_{t,d,s}$ to be the number of occurrences of term t in field s of document d. We then apply field-specific document length normalization similar to the BM25 normalization but applied at the field level, reflecting the differences in average length between different field types. For example, the title of a document is usually very short relative to its body

$$f'_{t,d,s} = \frac{f_{t,d,s}}{(1 - b_s) + (l_{d,s}/l_s)} \quad \ldots (3.16)$$

where $l_{d,s}$ is the length of field s in document d, $l_s$ is the average length of field s across all documents, and $b_s$ is a field-specific parameter similar to the b parameter in BM25. Like the b parameter the $b_s$ parameters can range from 0 to 1, producing no normalization at one extreme and complete normalization at the other

- These field-specific adjusted term frequencies, or pseudo-frequencies, may then be combined into an adjusted term frequency for the document as a whole.



$$f'_{t,d} = \sum_s v_s \cdot f'_{t,d,s} \quad \ldots (3.17)$$

where $v_s$ is the weight for field s. For example we might define $v_{title} = 10$ and $v_{body} = 1$. We may then replace the actual term frequency with this adjusted term frequency in Equation

$$\sum_t q_t \cdot \frac{f_{t,d}\,(k_1 + 1)}{k_1 + f_{t,d}} \cdot w_t$$

to give the BM25F formula:

$$\sum_t q_t \cdot \frac{f'_{t,d}\,(k_1 + 1)}{k_1 + f'_{t,d}} \cdot w_t \quad \ldots (3.18)$$

- Because length normalization is applied as part of computing the adjusted term frequency, no explicit normalization is required in this formula. We continue to compute $w_t$, the Robertson/Sparck Jones weight, at the document level. The use of BM25F requires that we tune two parameters for each field, $b_s$ and $v_s$, as well as the overall parameter $k_1$. Implementing BM25F requires documents to contain appropriate tagging and requires the computation of term frequency with respect to each field.

## 3.6 LANGUAGE MODELS FOR INFORMATION RETRIEVAL

### 3.6.1 Generating Queries from Documents

- As we did in previous section we take as our starting point the Probability Ranking Principle, as embodied in Equation

$$\log \frac{p(r \mid D, Q)}{1 - p(r \mid D, Q)} = \log \frac{p(r \mid D, Q)}{p(\bar{r} \mid D, Q)} \quad \ldots (3.17)$$

$$= \log \frac{p(D, Q \mid r)\, p(r)}{p(D, Q \mid \bar{r})\, p(\bar{r})}. \quad \ldots (3.18)$$

- We expanded the joint probabilities appearing in this equation using the equality $p(D, Q \mid R) = p(D \mid Q, R) \cdot p(Q \mid R)$. In this section we develop the equation in the opposite direction, expanding the joint probabilities using the equality $p(D, Q \mid R) = p(Q \mid D, R) \cdot p(D \mid R)$.

$$\log \frac{p(r \mid D, Q)}{p\bar{r} \mid D, Q} = \log \frac{p(D, Q \mid r)\, p(r)}{p(D, Q \mid \bar{r})\, p(\bar{r})} \quad \ldots (3.19)$$

$$= \log \frac{p(Q \mid D, r)\, p(D \mid r)\, p(r)}{p(Q \mid D, \bar{r})\, p(D \mid \bar{r})\, p(\bar{r})} \quad \ldots (3.20)$$

$$= \log \frac{p(Q \mid D, r)\, p(r \mid D)}{p(Q \mid D, \bar{r})\, p(\bar{r} \mid D)} \quad \ldots (3.21)$$

$$= \log p(Q \mid D, r) - \log p(Q \mid D, \bar{r}) + \log \frac{p(r \mid D)}{p(\bar{r} \mid D)} \quad \ldots (3.22)$$

$$= \log p(Q \mid D, r) - \log p(Q \mid D, \bar{r}) + \text{logit}\,(p(r \mid D)) \ldots (3.23)$$

- We examine each of the probabilities in this last formula in detail.
- For a given query q and a relevant document d, the value $p(Q = q \mid D = d, r)$ represents the probability that the user would enter the query q in order to retrieve d. In essence, conditioning on d provides us with an example of a relevant document. From this example we may then estimate the probability of a user entering q if a document such as d is desired. In particular, if a term appears in d with much greater frequency than random chance suggests, we might expect it to have a higher probability of appearing in the query. On the other hand, consider the role of d in the probability

  $p(Q = q \mid D = d, \bar{r})$.
- Here conditioning on d provides us with a weaker picture of the user's requirements. We have an example of what is not relevant but still can only guess about what is relevant. As a result it may be reasonable to assume that this probability is independent of d, that is

  $p(Q = q \mid D = d, \bar{r})$ is a constant. If we accept this assumption, we can drop the constant, leaving us with the rank-equivalent formula

  $\log p(Q \mid D, r) + \text{logit}(p(r \mid D))$. ... (3.24)
- The probability $p(r \mid D)$ in the second term is independent of q and is the same for all queries. This prior probability of relevance might be interpreted as indicating something about the quality or importance of the document. In the context of Web search, for example, the home page of a university might be assigned a higher prior probability than the personal home page of a student attending the university). In the context of file system search, e-mail addressed to you personally might be assigned a higher value than e-mail you receive from a mailing list.
- Nonetheless, in the absence of information suggested by the context of the application, the prior probability of relevance $p(r \mid D)$ is often assumed to be the same for all documents. Again, if we accept this assumption, we may drop this constant as an order-preserving transformation. Moreover, because exponentiation is also order-preserving, we may remove the logarithm, which leaves us with the ranking formula

  $p(Q \mid D, r)$. ... (3.25)

VIKAS

- If we wish, conditioning on relevance can be made implicit, thus reducing the equation to

$$p(Q = q \mid D = d). \quad (3.26)$$

- This very simple formula plays the same role in relation to the language modeling approach Given a document d and a query q, we score d for ranking purposes by estimating the probability p(q | d).
- In order to estimate this probability, the document is often considered to provide a model for generating the query q. Imagine a user formulating a query to retrieve relevant documents. She might attempt to think of some terms that should appear often in these relevant documents but rarely in nonrelevant documents. In making this attempt the user forms an image in her mind of what a relevant document looks like what terms might distinguish it from other documents in the collection. She then selects a few of these terms and enters them into a search engine. These terms become the query q.
- To score d we assume it matches the image in the user's mind and estimate the probability that the user would select q to retrieve it. It is because of this assumed link between the user's information need the image in her mind and the document d that we can view d as providing a generative model for q. Some IR researchers justify Eq. (3.26) more directly through a different reasoning process. They simply state that we are interested in p(Q, D), the joint distribution of queries and documents. The issue of relevance is ignored. Documents are ranked by

$$p(Q, D) = p(Q \mid D) \cdot p(D),$$

or equivalently by log $p(Q \mid D) + \log p(D)$. (3.27)

## 3.7 LANGUAGE MODELS

### 3.7.1 Finite Automata and Language Models

I wish
I wish I wish
I wish I wish I wish
I wish I wish I wish I wish I wish
...
CANNOT GENERATE: wish I wish

**Fig. 3.1 : A simple finite automaton and some of the strings in the language it generates. → shows the start state of the automaton and a double circle indicates a (possible) finishing state**

VIKAS

- What do we mean by a document model generating a query? A traditional generative model of a language, of the kind familiar from formal language theory, can be used either to recognize or to generate strings. For example, the finite automaton shown in Fig. 3.1 can generate strings that include the examples shown. The full set of strings that can be generated is called the language of the automaton

$P(STOP|q_1) = 0.2$

| | |
|---|---|
| the | 0.2 |
| a | 0.1 |
| frog | 0.01 |
| toad | 0.01 |
| said | 0.03 |
| likes | 0.02 |
| that | 0.04 |
| ... | ... |

**Fig. 3.2: A one-state finite automation that acts as a unigram language model. We show a partial specification of the state emission probabilities**

- If instead each node has a probability distribution over generating different terms, we have a language model. The notion of a language model is inherently probabilistic. A language model is a function that puts a probability measure over strings drawn from some vocabulary. That is, for a language model M over an alphabet $\Sigma$:

$$\sum_{s \in \Sigma^*} P(s) = 1 \quad \ldots (3.28)$$

The simple kind of language model is equivalent to a probabilistic finite automaton consisting of just a single node with a single probability distribution over producing different terms, so that $\sum_{t \in V} P(t) = 1$, as shown in Fig. 3.2. After generating each word, we decide whether to stop or to loop around and then produce another word, and so the model also requires a probability of stopping in the finishing state. Such a model places a probability distribution over any sequence of words. By construction, it also provides a model for generating text according to its distribution.



**Worked Example:** To find the probability of a word sequence, we just multiply the probabilities which the model gives to each word in the sequence, together with the probability of continuing or stopping after producing each word. For example,

$P(\text{frog said that toad likes frog}) = (0.01 \times 0.03 \times 0.04 \times 0.01 \times 0.02 \times 0.01)$ ... (3.29)

$\times (0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.2)$ ... (3.30)

$= 0.000000000001573$ ... (3.31)

- As you can see, the probability of a particular string/document, is usually a very small number! Here we stopped after generating frog the second time. The first line of numbers are the term emission probabilities, and the second line gives the probability of continuing or stopping after generating each word. An explicit stop probability is needed for a finite automaton to be a well-formed language model according to Eq. (3.28). Nevertheless, most of the time, we will omit to include STOP and (1 – STOP) probabilities (as do most other authors). To compare two models for a data set, we can calculate their likelihood ratio, which results from simply dividing the probability of the data according to one model by the probability of the data according to the other model. Providing that the stop probability is fixed, its inclusion will not alter the likelihood ratio that results from comparing the likelihood of two language models generating a string. Hence, it will not alter the ranking of documents.
- Finite automata and language models are two fundamental concepts in computer science, with applications in various fields, including Natural Language Processing (NLP) and Information Retrieval (IR). Let us explore each concept and their relationship:

1. **Finite Automata:** Finite automata are computational models used to recognize and process strings in formal languages. They are abstract machines with a finite number of states that can transition from one state to another based on input symbols. Finite automata can be deterministic (DFA) or non-deterministic (NFA).
   - ➢ **Deterministic Finite Automaton (DFA):** In a DFA, for each state and input symbol, there is a unique transition to another state. DFAs are used to recognize regular languages, which are a subset of formal languages.
   - ➢ **Non-Deterministic Finite Automaton (NFA):** In an NFA, multiple transitions may exist for a given state and input symbol. NFAs are more expressive than DFAs and can recognize a broader class of languages, including regular languages.
2. **Language Models:** Language models are probabilistic models used to estimate the likelihood of sequences of words or tokens in a language. They are essential in NLP tasks, such as speech recognition, machine translation, and information retrieval.
   - ➢ **N-gram Language Models:** N-gram language models estimate the probability of a word based on the previous (N-1) words. For example, a trigram model estimates $P(w_i \mid w_{i-1}, w_{i-2})$ - the probability of word $w_i$ given the previous two words.
   - ➢ **Statistical Language Models:** Statistical language models use statistics from a corpus of text to estimate the likelihood of word sequences. Common statistical models include n-gram models and smoothed models like add-k or Kneser-Ney smoothing.
3. **Relationship between Finite Automata and Language Models:** The relationship between finite automata and language models lies in their application to language recognition and processing:
   - ➢ **Language Recognition:** Finite automata are used to recognize regular languages, which can be represented by regular expressions. Language models, on the other hand, are used to estimate the likelihood of word sequences in a language. Both concepts are employed in language recognition tasks.
   - ➢ **Information Retrieval:** Language models play a crucial role in IR systems. They are used to model the statistical properties of documents and queries. In contrast, finite automata are used for efficient string matching and indexing, which is an essential component of IR systems for fast retrieval of relevant documents.
   - ➢ **Natural Language Processing:** Language models are widely used in NLP tasks, such as speech recognition, machine translation, and text generation. Finite automata are less commonly used in these tasks but can be applied in specific cases that involve regular language recognition.

## 3.8 TYPES OF LANGUAGE MODELS

### 3.8.1 Multinomial Distributions Over Words

Multinomial distributions over words are a type of probability distribution used to model the occurrence of words in a given context, typically in the context of Natural Language Processing (NLP) and Information Retrieval (IR). The multinomial distribution is a generalization of the binomial distribution, which models the number of successes in a fixed number of independent Bernoulli trials. In the context of words, the multinomial distribution represents the probabilities of observing different words from a predefined vocabulary in a document or a sequence of words.

- **Multinomial Distribution:** The multinomial distribution models the probabilities of different outcomes in a categorical experiment. In the context of words, the outcomes are the occurrences of individual words from a fixed vocabulary in a given context (e.g., a document, a sentence, or a bag of words).
- **Word Frequencies:** In NLP and IR, a common way to represent text is through the bag-of-words model, where a document is represented as a vector of word frequencies. Each element of the vector corresponds to the frequency of a specific word from the vocabulary in the document. The multinomial distribution can be used to model the probabilities of observing each word frequency in the document.
- **Probability Estimation:** To estimate the multinomial probabilities, one typically uses Maximum Likelihood Estimation (MLE) based on a training corpus. Given a corpus of documents, the probability of observing each word's frequency in a document can be estimated as the relative frequency of that word in the corpus.
- **Language Models:** Multinomial distributions over words are commonly used in language modeling. Language models aim to estimate the probability of a sequence of words in a language. A simple form of a language model is the n-gram model, where the probability of a word is estimated based on the occurrence of the previous (n-1) words. The n-gram probabilities can be modeled using multinomial distributions over words.

**Applications:**

- **Information Retrieval:** In IR, multinomial distributions can be used to model the occurrence of query terms in documents and estimate the relevance of documents to
- **Document Classification:** Multinomial distributions can be used in text classification tasks, where the goal is to classify documents into predefined categories based on word frequencies.
- **Language Generation:** Multinomial distributions are used in text generation tasks, such as language generation, machine translation, and text summarization, to generate coherent and fluent sentences.
- **Smoothing Techniques:** In practice, some words in a corpus may have zero frequencies for certain contexts, leading to zero probabilities in the multinomial distribution. To address this issue, smoothing techniques like Laplace (add-one) smoothing or Dirichlet smoothing are used to avoid zero probabilities and improve the robustness of language models.

### 3.8.2 Ranking with Language Models

- Eq. (3.26) suggests that we are ranking documents according to the probability that q will be entered as a query, given that d is an example of a relevant document. We are now ready to apply the smoothed language models the problem of estimating this probability. We start by applying independence assumptions similar to Assumption Q, thereby reducing the problem to that of estimating probabilities for individual terms in the query. Given a query vector $q = t_1, t_2, ..., t_n$ and a document d, we may estimate $p(q \mid d)$ as

$$p(q \mid d) = p(|q| = n) \cdot \prod_{i=1}^{n} p(t_i \mid d) \quad \ldots (3.32)$$

where $p(|q| = n)$ represents the probability that the user would enter a query of length n. In order to estimate the probability that the user will enter q, we must estimate this probability for the query length assuming that this length is independent of the document and query terms as well as a probability for each individual term assuming that the selection of one term is independent of the selection of the others. Fortunately, it is safe to ignore the query length. Because the query is fixed, it is sufficient to consider the probability that the user would enter q out of all possible queries of length n rather than out of all possible queries of any length. we estimate $p(q \mid d)$ as

$$p(q \mid d) = \prod_{i=1}^{n} p(t_i \mid d) \quad \ldots (3.33)$$



- Because the order of terms in the query vector is of no significance in this equation, we may treat q as a set and make the query-term frequency explicit in the equation:

$$p(q \mid d) = \prod_{t \in q} p(t \mid d)^{q_t} \quad \ldots (3.34)$$

- A document language model may then be used to estimate a value for each $p(t \mid d)$, and thus we would rank documents according to

$$p(q \mid d) = \prod_{t \in n} M_d(t)^{q_t} \quad \ldots (3.35)$$

- However, it is valuable to expend a little extra effort over the next few pages in order to achieve some additional insight into the language modeling approach and its relationship to other methods. Unlike the previous formulae, these equations do not take a TF-IDF form. Moreover, the number of documents in the collection (N) and the number of documents containing the term ($N_t$) do not appear in the equations at all. Nonetheless, with a little work we can find something close to TF-IDF hidden in these equations while simplifying them at the same time.
- For convenience as well as consistency with the other approaches, we work with logarithms from this point forward rather than directly with the probabilities. First, we consider the terms contained in the document separately from the terms not contained in the document.

$$\log p(q \mid d) = \sum_{t \in q} q_t \cdot \log p(t \mid d) \quad \ldots (3.36)$$

$$= \sum_{t \in q \cap d} q_t \cdot \log p(t \mid d) + \sum_{t \in q \setminus d} q_t \cdot \log p(t \mid d) \quad \ldots (3.37)$$

### 3.8.3 Divergence from Randomness

**Kullback-Leibler Divergence**

- An alternative theoretical framework for understanding and working with the language modeling approach is provided by a concept known as Kullback-Leibler divergence. KL divergence, also known as relative entropy, is a method for comparing two probability distributions.
- Given continuous probability distributions f(x) and g(x) the KL divergence between them is defined as

$$\int_{-\infty}^{\infty} f(x) \cdot \log \frac{f(x)}{g(x)} \, dx \quad \ldots (3.38)$$

- In information retrieval, we normally work with discrete distributions, for which KL divergence takes the form

$$\sum_{x} f(x) \cdot \log \frac{f(x)}{g(x)} \quad \ldots (3.39)$$

- Larger values indicate greater divergence. When f and g represent the same distribution, their KL divergence is zero, since $\log (f(x)/g(x)) = \log 1 = 0$.
- For example, the flip of a "fair" coin will produce heads or tails with equal probability. If an "unfair" coin produces heads with a 40% probability and tails with a 60% probability, the KL divergence between the fair and unfair coin may be computed as

$$0.4 \cdot \log \frac{0.4}{0.5} + 0.6 \cdot \log \frac{0.6}{0.5} = 0.02905$$

- Due to this asymmetry KL divergence is sometimes viewed as comparing a "true" distribution with another distribution in which f(x) in Eq. (3.39) represents this true distribution. From an information-theoretic standpoint, KL divergence indicates the average number of extra bits per symbol needed to transmit or compress a message if we assume its symbols are distributed according to g instead of the true distribution f.
- In order to apply KL divergence to ranking, we construct a language model from the query $M_q(t)$ in much the same way that we constructed a language model from the document. The simplest language model is the maximum likelihood model: the ratio of the number of times t appears in the query to the length of the query

$$M_q^{ml}(T) = \frac{q_t}{n} \quad \ldots (3.40)$$

- It is also possible to create more complex query language models through smoothing or other processes, just as we did for the document language models. We then apply KL divergence to rank documents by computing the divergence between the query language model and the document language model:

$$\sum_{t \in v} M_q(t) \cdot \log \frac{M_q(t)}{M_d(t)} = \sum_{t \in v} M_q(t) \cdot \log M_q(t) - \sum_{t \in v} M_q(t) \cdot \log M_d(t) \quad \ldots (3.41)$$

- The summation on the left is the same for all documents and may be dropped as an order preserving transform. The summation on the right, without the negative sign, increases with decreasing divergence and is therefore suitable as a ranking formula:

$$\sum_{t \in v} M_a(t) \cdot \log M_a(t) \quad \ldots (3.42)$$

- Now, if we replace Mq(t) with the maximum likelihood language model $M_q^{ml}(t)$, the formula becomes

$$\sum_{t \in v} M_q^{ml}(t) \cdot \log M_a(t) = \frac{1}{n} \cdot \sum_{t \in q} qt \cdot \log M_d(t) \quad \ldots (3.43)$$

- The constant 1/n may be dropped to give the rank-equivalent formula

$$\sum_{t \in q} q_t \cdot \log M_d(t), \quad \ldots (3.44)$$

- Instead of using a maximum likelihood model for $M_a(t)$ in Eq. (3.43), we may instead view the presence of a query language model as an opportunity for query expansion, extending the query language model to estimate nonzero probabilities for terms not appearing in the original query. In this way we can assign positive scores to documents that may not contain any of the query terms.

**Divergence from Randomness**

- The Divergence From Randomness (DFR) approach to information retrieval explicitly assumes a random process for the distribution of terms in documents, and then ranks documents by considering the probability that the actual term distribution found in a document would occur by chance. Similar considerations appear implicitly in the language modeling approach through its incorporation of a collection language model into the smoothing process.

$$\sum_{t \in q} q_t \cdot \log\left(1 + \frac{1-\lambda}{\lambda} \cdot \frac{f_{t,d}}{l_d} \cdot \frac{l_c}{l_t}\right)$$

includes the ratio of the actual number of occurrences in a document to the expected number of occurrences based on the collection language model. An important property of DFR is the absence of the seemingly arbitrary parameters that appear in other methods and require tuning over a training set. Parameters such as μ in LMD, λ in LMJM, and k1 in BM25 often appear unintuitive, defying an easy explanation for their presence. DFR offers retrieval effectiveness comparable with these methods in a non-parametric form. In this section we also revisit the notion of eliteness that we introduced. Having determined the probability that a random document d would contain $f_{t,d}$ occurrences of term t where $f_{t,d} > 0$, we exploit the notion of eliteness to estimate the probability that the document is actually "about" the concept embodied in the term. this notion was modeled by a two-Poisson distribution. In this section we present an approach based on Laplace's law of succession.

- At its most generic the core of the DFR approach may be summarized by the formula

$$(1 - P2) \cdot (-\log P1) \quad (3.45)$$

- In this formula P1 represents the probability that a random document d contains exactly $f_{t,d}$ occurrences of t. The value $-\log P1$ may be viewed as the number of bits of information, called the self-information, associated with d containing exactly $f_{t,d}$ occurrences of t. Our random process for distributing occurrences of the term t across the document collection is unlikely to assign a large proportion of these terms to the specific document d. Thus P1 may decrease rapidly as $f_{t,d}$ increases
- P2 provides an adjustment that reflects eliteness, thus correcting for this rapid decrease. If d is elite in the t, we might assume that the occurrences of t that appear within it are not accidental. Suppose we begin reading d, which is elite in t, in order to count the number of occurrences of t it contains. Well before we reach its end, we discover $f_{t,d} - 1$ occurrences of t. This discovery suggests that we should expect to find more occurrences and P2 is the probability of finding at least one. P2 increases as $f_{t,d}$ increases. Thus, $(1 - P2)$ in Eq. (3.45) decreases as $f_{t,d}$ increases.
- Thus far we have considered only a single term. To rank documents with respect to a multiterm query, we make the usual independence assumption, giving the ranking formula

$$\sum_{t \in q} q_t \cdot (1 - P_2) \cdot (-\log P_1) \quad \ldots (3.46)$$

- In the remainder of the presentation we focus on estimating P1 and P2 for a given term t with the understanding that these estimates will be used in this formula for ranking.



## 3.9 A MODEL OF RANDOMNESS

- Suppose we randomly distribute terms into documents. If we have $l_t$ occurrences of term t distributed across N documents, then

$$f_{t,1} + f_{t,2} + \ldots + f_{t,N} = l_t \quad \ldots (3.47)$$

where $f_{t,i}$ is the number of occurrences of term t in the $i^{th}$ document.

- How many different ways can $l_t$ occurrences be distributed across N documents, assuming that the documents are indistinguishable? For example, four occurrences can be distributed across three documents in four different ways:
  1. All occurrences into one document,
  2. Three occurrences into one document and one into another,
  3. Two occurrences in one document and one in each of the other documents,
  4. Two occurrences in two documents. In other words, how many different arrangements of terms will satisfy Eq. (3.47)?
- To answer this question, Amati and van Rijsbergen recognize that this problem is identical to the problem addressed by Bose-Einstein statistics, which computes the number of ways indistinguishable particles may be assigned to energy states in thermal equilibrium. The solution may be expressed as a binomial coefficient:

$$\binom{N + l_t - 1}{l_t} = \frac{(N + l_t - 1)!}{(N-1)!\, l_t!} \quad \ldots (3.48)$$

- Alternatively, the problem may be viewed as equivalent to the problem of determining the number of ways in which m balls can be distributed across n indistinguishable bins, a version of the occupancy problem from combinatorics. In order to compute an estimate of P1 for term t, we assume that a given document d is found to contain $f_{t,d}$ occurrences of t. A random distribution of the remaining $l_t - f_{t,d}$ occurrences into the remaining documents must satisfy the equation

$$f_{t,1} + \ldots + f_{t,d} - 1 + f_{t,d} + 1 + \ldots + f_{t,N} = l_t - f_{t,d} \quad \ldots (3.49)$$

- The number of ways to satisfy this equation follows from Eq. (3.48):

$$\binom{(N-1) + (l_t - f_{t,d}) - 1}{l_t - f_{t,d}} = \frac{((N-1) + (l_t - f_{t,d}) - 1)!}{(N-2)!\,(l_t - f_{t,d})!} \quad \ldots (3.50)$$

- This equation assumes that a selected document contains $f_{t,d}$ occurrences and represents the number of ways the remaining terms can be distributed. Represents the number of ways $l_t$ occurrences may be distributed across N documents. Thus, the ratio of these equations represents P1, the probability that a selected document contains $f_{t,d}$ occurrences:

$$P_1 = \frac{\binom{(N-1) + (l_t - f_{t,d}) - 1}{l_t - f_{t,d}}}{\binom{N + l_t - 1}{l_t}}$$

$$= \frac{((N-1) + (l_t - f_{t,d}) - 1)!\,(N-1)!\,l_t!}{(N-2)!(l_t - f_{t,d})!\,(N + l_t - 1)!} \quad \ldots (3.51)$$

- Unfortunately, the presence of factorials in Eq. (3.51) makes it difficult to work with it directly. Instead, Amati and van Rijsbergen provide two methods for estimating its value and demonstrate that both methods provide similar performance in terms of effectiveness. The simpler of these methods provides an estimate for P1 of

$$P_1 = \left(\frac{1}{1 + l_t/N}\right)\left(\frac{l_t/N}{1 + l_t/N}\right)^{f_{t,d}}, \quad \ldots (3.52)$$

and therefore

$$-\log P_1 = \log(1 + l_t/N) + f_{t,d} \cdot \log(1 + N/l_t). \quad \ldots (3.53)$$

- The term on the right has a form reminiscent of TF-IDF, but with $l_t$ taking the role of $N_t$.

### 3.9.1 Passage Retrieval and Ranking

- Most of the ranking methods we examine in this book rank documents. Depending on the application environment, these documents may correspond to Web pages, news articles, e-mail messages, or a combination of these and other document types. In some environments it may be appropriate to rank elements within documents, such as the pages in a book or the sections in a news article. In other circumstances arbitrary passage retrieval may be desirable so that any fragment of text within a document may be returned as a ranked result.

William **Shakespeare** – Wikipedia, the free encyclopedia

At the age of 18, **Shakespeare** married the 26-year-old Anne Hathway. The consistory court of the Diocese of Worcester issued a **marriage** licence on 27 November 1582. Two of Hathway's neighbours posted bonds the next day…

en.wikipedia.org/wiki/William_**Shakespeare**

A typical search engine result for the query ("shakespeare", "marriage"), evaluated against Wikipedia. The result provides a brief snippet from the document that shows the query terms in context.

- One such circumstance is generation of snippets to provide a simple summary of the contents of a retrieved document for presentation to the user. An example of the result a search engine might return from Wikipedia given the query "shakespeare" ,"marriage". Along with the title of the Web page and its URL, the search engine provides a snippet extracted from the body of the document to illustrate the context in which the search terms appear.
- Question answering is another application in which arbitrary passage retrieval may be valuable Given a question such as "What is the population of India?", a question answering system attempts to provide an exact answer ("1.12 billion") rather than a document that may contain the answer. Passage retrieval is often an early step in question answering. Keywords are extracted from the question and formed into a query for processing by an IR system ( "population","india"). Passages containing these terms are then retrieved from the corpus and analyzed to extract and validate possible answers. An interval containing the query terms in close proximity ("...population of India, ...") is likely to be part of a slightly longer but still short passage containing the answer ("...The population of India is..."). The interval represents a "hot spot" within the collection near which the answer may be found.
- We now extend the notion of a cover to subsets of the query terms. We define an m-cover for $m \leq n$ terms as an interval [u, v] in the collection that contains an occurrence of at least m distinct terms in the query and does not contain a smaller interval that also contains m distinct terms. For simplicity we assume that terms are not repeated in q because the passage ranking method in this section does not take repeated query terms into account. For example, the set of 2-covers for the query vector "you", "quarrel", "sir" over the collection in Table 2.1 (page 50) consists of the intervals [2, 3], [3, 4], [4, 5], [5, 6], [8, 10], [10, 12], [12, 16], [24, 28]. Note that the interval [12, 24] is not included in the set because it contains [12, 16]. Note also that [24, 28] is included even though it cuts across the last three documents. In most applications m-covers such as this one would be filtered from the set before use. However, for simplicity we define m-covers without consideration for document boundaries and other structures and apply a post-processing pass as appropriate.

The notion of an m-cover may be used to support snippet generation and arbitrary passage retrieval for question answering. Suppose we are looking for a snippet to display within an indexed Web page. Ideally we would display a snippet containing all the query terms in close proximity, but this goal is not always possible. Some of the query terms may appear only far apart, at the start and the end of the document. Some of the query terms may appear only in the title of the document and not in the body. Some of the query terms may not appear in the document at all. Instead, the snippet might be composed of text fragments corresponding to one or more m-covers that together contain as many of the terms as possible. Once an appropriate set of m-covers is determined, each could be extended to include the nearest sentence boundaries and then trimmed to fit into the required space.

- In the context of question answering, a passage containing a strict subset of the query terms in close proximity may be more likely to yield an answer than a much longer passage containing all the query terms. For example, suppose we are answering the question "Who starred in the film Shakespeare in Love?" We might execute the query vector "starred","film", "shakespeare", "love" and analyze the result for possible answers. A short 3-cover containing the terms "shakespeare" and "love" along with just one of the terms "starred" or "film" may be a better indicator of an answer than a longer passage containing all the terms, in which the answer may be mixed with other names and details. More generally, for applications such as snippet generation and question answering, we must select intervals by trading off length against the combination of query terms they contain

**Passage Scoring**

- Assume we have an interval [u, v] that we wish to score with respect to a query $q = t_1, t_2, ..., t_n$. Further assume that the interval contains a subset of the query terms $q' \subseteq q$, where $q' = t_1, t_2, ..., t_m$ and $m \leq n$. We assign a score to [u, v], with length $l = v - u + 1$, by estimating the probability that a randomly selected interval of this length would contain at least one occurrence of these query terms
- For this purpose we model the collection as a sequence of independently generated terms and assume that there is a fixed probability pt of a term $t \in q$ matching at any given document location. Note that this assumption allows multiple terms to match at a particular location. Although unrealistic, this



assumption is tolerable when pt is small, as it is for most terms, and helps to simplify the derivation of a passage scoring formula. Given the interval [u, v] with length $l = v - u + 1$, the probability $p(t, l)$ that the interval contains one or more occurrences of t is

$$p(t, l) = 1 - (1 - p_t)^l \quad \ldots (3.54)$$

$$= 1 - (1 - l \cdot p_t + O(p_t^2)) \quad \ldots (3.55)$$

$$\approx l \cdot p_t \quad \ldots (3.56)$$

- The probability that [u, v] contains all the terms from q is then

$$p(q', l) = \prod_{t \in q'} p(t, l) = \prod_{t \in q'} l \cdot p_t = l^m \cdot \prod_{t \in q'} p_t \quad \ldots (3.57)$$

- Finally, we estimate pt as the collection frequency of the term t:

$$p_t = \frac{l_t}{l_c} \quad \ldots (3.58)$$

where $l_t$ is the total number of times t appears in the collection and lC is the total length of the collection. Substituting and taking a negative logarithm (i.e., the self-information) gives

$$\sum_{t \in q'} (\log(l_c/l_t)) - m \cdot \log(l) \quad \ldots (9.59)$$

## CASE STUDY: A COMPARATIVE STUDY OF PROBABILISTIC AND LANGUAGE MODELS FOR INFORMATION RETRIEVAL

**Example:** Search Engines and Information Retrieval

Imagine you are working on a comparative study of probabilistic and language models for information retrieval within the context of search engines. In this scenario, your goal is to understand how different models impact the quality of search results provided by a search engine.

1. **Probabilistic Model (Binary Independence Model):**

- Let us say you are using a classic probabilistic model like the Binary Independence Model (BIM).
- When a user enters a search query like "best smartphone camera," the model breaks down the query into individual terms: ["best," "smartphone," "camera"].
- The model calculates the probability that each document contains these terms, considering their presence or absence.
- It ranks documents based on the calculated probabilities. For instance, a document that contains all three terms is ranked higher than one that contains only two terms.

2. **Language Model (Okapi BM25):**

- Now, you are using a language model such as Okapi BM25.
- The model not only considers term presence but also takes into account the frequency of terms in documents and the length of the documents.
- When the same query "best smartphone camera" is entered, the model analyzes the query and the documents in its database to estimate relevance.
- It ranks documents by calculating a score that considers term frequency and document length. Documents with higher scores are considered more relevant.

**Comparative Study:** For your study, you gather a dataset of queries and corresponding documents. You implement both the Binary Independence Model (probabilistic) and the Okapi BM25 model (language-based) to retrieve documents for each query. Here is what you might find:

- **Strengths of Probabilistic Model:** The probabilistic model might excel in cases where the query consists of distinct, specific terms. It could be effective in scenarios where you are looking for precise matches and the presence or absence of certain keywords matters significantly.
- **Strengths of Language Model:** The language model, on the other hand, might perform better with longer, natural language queries that require understanding the context. It can consider the frequency of terms and document length, which provides a more nuanced understanding of relevance.
- **Insights:** Through your study, you might observe that probabilistic models are suitable for cases where the query terms are straightforward and independent, while language models shine when queries are more complex and involve nuances.
- **Practical Application:** Your findings could guide search engine developers in selecting appropriate models for different user scenarios. For instance, for queries that are short and focused, a probabilistic model might be preferred. For queries that are more conversational or descriptive, a language model could yield better results.

## EXERCISE

1. What is the Probability Ranking Principle (PRP) in Information Retrieval, and how does it relate to basic probability theory?
2. Discuss the 1/0 loss case in the context of the Probability Ranking Principle and its implications for retrieval performance.
3. How does the Binary Independence Model (BIM) handle term independence assumptions, and what are its limitations?
4. Explain the concept of term frequency in IR and its significance in ranking documents based on relevance.
5. Provide an appraisal of probabilistic models used in IR, highlighting their advantages and limitations compared to other retrieval models.
6. How do tree-structured dependencies between terms enhance Information Retrieval and document ranking?
7. Describe the Okapi BM25 model and its key characteristics as a non-binary probabilistic retrieval model.
8. Discuss the application of Bayesian network approaches to IR and how they can model relationships between query terms and documents.
9. What is Relevance Feedback in IR, and how does it improve the accuracy of retrieval results for user queries?
10. Explain the concept of Field Weights in the BM25F model and how they contribute to relevance ranking in multi-field documents.
11. How are language models used to generate queries from documents in Information Retrieval systems?
12. Compare finite automata and language models, highlighting their roles in language recognition and processing.
13. Enumerate the different types of language models used in NLP and IR tasks and explain their respective applications.
14. How are multinomial distributions over words applied in language modeling and text representation?
15. Describe the concept of Ranking with Language Models and how it differs from traditional retrieval models like BM25.
16. What is Divergence from Randomness, and how is it utilized in Information Retrieval and passage retrieval tasks?
17. Discuss the challenges and techniques involved in passage retrieval and ranking using language models.
18. Explain the concept of Passage Retrieval in IR and its significance in providing more granular and relevant document snippets to users.
19. How do language models differ from boolean retrieval models in terms of ranking and relevance estimation?
20. Provide examples of IR systems or applications where probabilistic retrieval and language models are used to improve search results and user experience.



VIKAS

UNIT - IV

# TEXT CLASSIFICATION AND TEXT CLUSTERING

## 4.1 TEXT CLASSIFICATION

### 4.1.1 Introduction to Text Classification

Text classification is a Natural Language Processing (NLP) task that involves categorizing text documents into predefined classes or categories. The goal of text classification is to automatically analyze and assign a label or class to a given text based on its content and context. This task is commonly used in a wide range of applications, such as spam detection, sentiment analysis, topic categorization, language identification, intent detection in chatbots, and more.

The process of text classification typically involves the following steps:

**Step 1: Data Collection:** The first step is to gather a large dataset of labeled text documents. These documents should be representative of the categories you want to classify. For instance, if you want to perform sentiment analysis, you would need a collection of texts labeled as positive or negative sentiments.

**Step 2: Data Preprocessing:** Text data often requires preprocessing to make it suitable for analysis. Common preprocessing steps include tokenization (breaking the text into individual words or tokens), converting text to lowercase, removing punctuation, and filtering out stop words (commonly used words like "the," "and," "in" that do not carry significant meaning).

**Step 3: Feature Extraction:** To feed the text data into a machine learning algorithm, you need to convert the text into numerical features. Various techniques can be used for this purpose, such as Bag-of-Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), or word embeddings like Word2Vec or GloVe.

**Step 4: Model Selection:** Choose an appropriate machine learning or deep learning model for your text classification task. Common algorithms include Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Random Forest, and more recently, deep learning models like Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs).

**Step 5: Model Training:** Split your dataset into a training set and a testing set. Train the selected model on the training set by providing the labeled texts and their corresponding numerical features.

**Step 6: Model Evaluation:** Evaluate the performance of the trained model on the testing set using appropriate evaluation metrics such as accuracy, precision, recall, F1-score, etc. This step helps assess how well the model generalizes to new, unseen data.

**Step 7: Model Deployment:** Once you are satisfied with the model's performance, you can deploy it to make predictions on new, unseen text data.

- Document classification/categorization is a topic in information science, a science dealing with the collection, analysis, classification, categorization, manipulation, retrieval, storage and propagation of information.
- This might sound very abstract, but there are lots of situations nowadays, where companies are in need of automatic classification or categorization of documents.

**Table 4.1: Variants of Inverse Document Frequency (IDF) Weight**

| Weighting Scheme | IDF weight ($n_t = \|\{d \in D \mid t \in d\}\|$) |
|---|---|
| Unary | 1 |
| Inverse document frequency | $\log \frac{N}{n_t} = -\log \frac{n_t}{N}$ |
| Inverse document frequency smooth | $\log\left(1 + \frac{N}{n_t}\right)$ |
| Inverse document frequency max | $\log\left(\frac{\max\{n' \in d\}\, n_{t'}}{1 + n_t}\right)$ |
| Probabilistic inverse document frequency | $\log \frac{N - n_t}{n_t}$ |

- Just think about a large company with thousands of incoming mail pieces per day, both electronic or paper based. Lot's of these mail pieces are without specific addressee names or departments. Somebody has to read these texts and has to decide what kind of a letter it is ("change of address", "complaints letter", "inquiry about products", and so on) and to whom the

document should be proceeded. This "somebody" can be an automated text classification system.

**Table 4.2: Variants of Term Frequency (TF) Weight**

| Weighting Scheme | TF Weight |
|---|---|
| Binary | 0, 1 |
| Raw count | $f_{t,d}$ |
| Term frequency | $f_{t,d} \sum_{t \in d} f_{t,d}$ |
| Double normalization 0.5 | $0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$ |
| Double normalization K | $K + (1 - K) \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$ |

- Automated text classification, also called categorization of texts, has a history, which dates back to the beginning of the 1960s. But the incredible increase in available online documents in the last two decades, due to the expanding internet, has intensified and renewed the interest in automated document classification and data mining. In the beginning text classification focussed on heuristic methods, i.e. solving the task by applying a set of rules based on expert knowledge. This approach proved to be highly inefficient, so nowadays the focus has turned to fully automatic learning and clustering methods.
- The task of text classification consists in assigning a document to one or more categories, based on the semantic content of the document. Document (or text) classification runs in two modes:
  (i) The training phase
  (ii) Prediction (or classification) phase
- The training phase can be divided into three kinds:
  (i) supervised document classification is performed by an external mechanism, usually human feedback, which provides the necessary information for the correct classification of documents,
  (ii) Semi-supervised document classification, a mixture between supervised and unsupervised classification: some documents or parts of documents are labelled by external assistance,
  (iii) Unsupervised document classification is entirely executed without reference to external information.
- We will implement a text classifier in Python using Naive Bayes. Naive Bayes is the most commonly used text classifier and it is the focus of research in text classification. A Naive Bayes classifier is based on the application of Bayes' theorem with strong independence assumptions. "Strong independence" means: the presence or absence of a particular feature of a class is unrelated to the presence or absence of any other feature. Naive Bayes is well suited for multiclass text classification

### 4.1.2 Naives Bayes Model

The Naive Bayes model is a simple and effective probabilistic algorithm commonly used for text classification tasks. Despite its simplicity, it often performs surprisingly well in various NLP applications, including text classification. The model is based on Bayes' theorem, which allows us to calculate the probability of a hypothesis (class label) given the evidence (text features).

In the context of text classification, the Naive Bayes model assumes that the features (words or tokens) in the text are conditionally independent, given the class label. This is known as the "naive" assumption because, in reality, the features of a text are likely to be dependent on each other to some extent. However, despite this simplification, the Naive Bayes model can still provide good results in many cases.

Here's a step-by-step overview of how the Naive Bayes model works for text classification:

**Step 1: Data Preprocessing:** As with any text classification task, the first step is to preprocess the text data. This includes tokenization, converting text to lowercase, removing punctuation, and filtering out stop words.

**Step 2: Feature Extraction:** To represent the text data numerically, the model typically uses the Bag-of-Words (BoW) representation. BoW is a simple and effective way of representing text data, where each document is represented as a vector of word frequencies. The vocabulary is constructed from the entire corpus of training data.

**Step 3: Calculating Class Priors:** The Naive Bayes model calculates the prior probabilities of each class label (the probability that a document belongs to a specific class). This is done by counting the occurrences of each class label in the training data and dividing it by the total number of training documents.

**Step 4: Calculating Likelihoods:** For each class, the model calculates the likelihood of each word occurring in documents of that class. This is done by counting the occurrences of each word in documents of that class and dividing it by the total number of words in all documents of that class.



**Step 5: Classifying New Texts:** Given a new text document, the Naive Bayes model calculates the probability of the document belonging to each class label using Bayes' theorem. It multiplies the class prior probability with the product of the likelihoods of the words in the document. The class with the highest probability is then assigned as the predicted label for the new document.

**Step 6: Smoothing:** In practice, to avoid zero probabilities for words that might not occur in the training data for a particular class, a technique called Laplace smoothing or add-one smoothing is often used. This involves adding a small constant to all word counts.

- The Naive Bayes classifier can be used in the following applications:
  - Emails are automatically classified into folders, including: "Family", "Friends", "Updating", and "Promotions".
  - Job listings are automatically tagged. Job listings in raw text format can get classified according to keywords: "software development", "design", and "marketing".
  - Products automatically get categorized. We can classify the products according to their description, such as books, electronics, clothing, etc.

1. **Naive Bayes Classifier**
   1. Sorts emails into folders
   2. Tags job listing
   3. Categorizes products

- Often, even very sophisticated classification methods, especially those utilizing very large datasets, do not perform as Naive Bayes. This is mainly because Naive Bayes is very simple.

**Advantages**

- This algorithm is fast and easy to use and helps in predicting the class of a dataset very quickly.
- You can easily solve multiclass prediction problems as it's quite useful.
- As compared to other models with independent features, the Naive Bayes classifier performs better with less training data.
- The Naive Bayes algorithm performs exceptionally well with categorical input variables.
- Using this method, you can predict the class of test data easily and quickly. It also performs well when predicting multiple classes at once.
- When the assumption of independence is true, Naive Bayes classifiers outperform logistic regression.
- For categorical variables, it performs well compared to numerical input variables. When dealing with numerical input variables, the bell curve is assumed.

**Disadvantages**

- It is impossible for the Naive Bayes model to make any predictions if your test data set contains a categorical variable that was not present in your training data set. A smoothing technique known as Zero Frequency can solve this problem.
- In addition to being a lousy estimation algorithm, 'predict_proba' also computes probability outputs.
- While in theory, it sounds great, you'll not find many independent features in real life.
- Consequently, the model fails to predict if it assigns a zero (zero) probability to the categorical variable (in the test data set) that it did not observe in the training data set. In this case, we are dealing with "Zero Frequency". This is accomplished by using Laplace estimation, one of the simplest smoothing techniques.
- Alternatively, Naive Bayes is a poor estimator, so we shouldn't take too much advantage of the results from predict_proba.
- Naive Bayes suffers from another limitation in that it assumes independent predictors. In reality, independent predictors are almost impossible to obtain in practice.

2. **Naive Bayes Assumption**

- Assuming that each word is independent of all the others will help us with the equation and ultimately, with creating codes.
- To simplify the math, we can make this assumption, which, in practice, works quite well. Knowing which words come before/after has a direct impact on the next/previous word.
- Naive Bayes is based on this assumption. Based on that assumption, we can decompose the numerator as follows.

$$p(\text{Spam}|w_1,\ldots,w_n) = \frac{p(w_1|\text{Spam})p(w_2|\text{Spam})\ldots p(w_n|\text{Spam})p(\text{Spam})}{p(w_1,\ldots,w_n)}$$

**3. When to Use Naive Bayes?**

- A Naive Bayesian classifier performs worse than a complex classifier due to the strict assumptions it makes about the data. The classifier, however, has some advantages:
  - ➢ Training and predicting the model is done at a high speed.
  - ➢ Probabilistic predictions can be created purely based on the data.
  - ➢ They are usually pretty easy to interpret.
  - ➢ Their parameters are usually not tunable.
    1. High-speed training and prediction
    2. Probability prediction based on data
    3. Easy to interpret
    4. Non-tunable parameters
- An initial baseline classifier based on a Naive Bayesian classifier offers these advantages. In case it performs well, you will have a classifier for your problem that is intuitive and very fast to interpret.
- With some basic knowledge of how well they should perform, you can explore more sophisticated models if it does not perform well initially.

**How to Execute Naive Bayes in Python**

Let's get started and upload the libraries first:

```
import numpy as np, pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.metrics import confusion_matrix, accuracy_score
sns.set() # use seaborn plotting style
```

We will now load the data (training and test data):

```
# Now Load the dataset
data_ = fetch_20newsgroups()
# Get the text categories in action
text_categories_ = data_.target_names
# Now we define the training set
train_data_ = fetch_20newsgroups(subset="train", categories=text_categories_)
# define the test set
test_data_ = fetch_20newsgroups(subset="test", categories=text_categories_)
```

Let's count the classes and samples:

```
print("Number of unique classes {}".format(len(text_categories_)))
print("Number of training samples {} ".format(len(train_data_.data_)))
print("Number of test samples {}".format(len(test_data_.data_)))
```

You will get output as:

```
Number of unique classes 20
Number of training samples 11314
Number of test samples 7532
```

As a result, we have a 20-class (which is the by default no. of classes in which the algorithm divides the data) text classification problem with a training sample size of 11314 and a test sample size of 7532 (text sentences).

Let's take a look at the third training sample:

```
print(test_data_.data_[3])
```

You should see something like this printed out since our data is texts (specifical emails):

**Outputs:**

It looks like Ben Baz's mind and heart are also blind, not only his eyes.

>I used to respect him, today I lost the minimal amount of respect that

>I struggled to keep for him.

>To All Muslim netters: This is the same guy who gave a "Fatwah" that

>Saudi Arabia can be used by the United States to attack Iraq .

They were attacking the Iraqis to drive them out of Kuwait,

a country whose citizens have close blood and business ties

to Saudi citizens. And me thinks if the US had not helped out

the Iraqis would have swallowed Saudi Arabia, too (or at

least the eastern oilfields). And no Muslim country was doing



much of anything to help liberate Kuwait and protect Saudi

Arabia; indeed, in some masses of citizens were demonstrating

in favor of that butcher Saddam (who killed lotsa Muslims),

just because he was killing, raping, and looting relatively

rich Muslims and also thumbing his nose at the West.

So how would have *you* defended Saudi Arabia and rolled

back the Iraqi invasion, were you in charge of Saudi Arabia???

>Fatwah is as legitimate as this one. With that kind of "Clergy", it might

>be an Islamic duty to separate religion and politics, if religion

>means "official Clergy".

I think that it is a very good idea to not have governments have an

official religion (de facto or de jure), because with human nature

like it is, the ambitious and not the pious will always be the

ones who rise to power. There are just too many people in this

world (or any country) for the citizens to really know if a

leader is really devout or if he is just a slick operator.

>

> CAIRO, Egypt (UPI) -- The Cairo-based Arab Organization for Human

> Rights (AOHR) Thursday welcomed the establishement last week of the

> Committee for Defense of Legal Rights in Saudi Arabia and said it was

> necessary to have such groups operating in all Arab countries.

You make it sound like these guys are angels, Ilyess. (In your

clarinet posting you edited out some stuff; was it the following???)

Friday's New York Times reported that this group definitely is

more conservative than even Sheikh Baz and his followers (who

think that the House of Saud does not rule the country conservatively

enough). The NYT reported that, besides complaining that the

government was not conservative enough, they have:

- - asserted that the (approx. 500,000) Shiites in the Kingdom

are apostates, a charge that under Saudi (and Islamic) law

brings the death penalty.

Diplomatic guy (Sheikh bin Jibrin), isn't he Ilyess?

- called for severe punishment of the 40 or so women who

drove in public a while back to protest the ban on

women driving. The guy from the group who said this,

Abdelhamoud al-Toweijri, said that these women should

be fired from their jobs, jailed, and branded as

prostitutes.

Is this what you want to see happen, Ilyess? I've

heard many Muslims say that the ban on women driving

has no basis in the Qur'an, the ahadith, etc.

Yet these folks not only like the ban, they want

these women falsely called prostitutes?

If I were you, I'd choose my heroes wisely,

Ilyess, not just reflexively rally behind

anyone who hates anyone you hate.

- say that women should not be allowed to work.

- say that TV and radio are too immoral in the Kingdom.

Now, the House of Saud is neither my least nor my most favorite government on earth; I think they restrict religious and political reedom a lot, among other things. I just think that the most likely replacements for them are going to be a lot worse for the citizens of the country. But I think the House of Saud is feeling the heat lately. In the last six months or so I've read there have been stepped up harassing by the muttawain (religious police---*not*

government) of Western women not fully veiled (something stupid for women to do, IMO, because it sends the wrong signals about your morality). And I've read that they've cracked down on the few, home-based expartiate religious gatherings, and even posted rewards in (government-owned) newspapers offering money for anyone who turns in a group of expartiates who dare worship in their homes or any other secret place. So the government has grown even more intolerant to try to take some of the wind out of the sails of the more-conservative opposition. As unislamic as some of these things are, they're just a small taste of what would happen if these guys overthrow the House of Saud, like they're trying to in the long run.

Is this really what you (and Rached and others in the general west-is-evil-zionists-rule-hate-west-or-you-are-a-puppet crowd) want, Ilyess?

--

Dave Bakken

==>"the President is doing a fine job, but the problem is we don't know what to do with her husband." James Carville (Clinton campaign strategist),2/93

==>"Oh, please call Daddy. Mom's far too busy." Chelsea to nurse, CSPAN, 2/93

Next, we will build a Naive Bayes classifier and train it. Our example will generate a matrix of token counts based on a collection of text documents. To do so, we will use the make_pipeline function.

```
# Model building
model = make_pipeline(TfidfVectorizer(),
MultinomialNB())
# Training the model with the training data
model.fit(train_data.data, train_data.target)
# Predicting the test data categories
predicted_categories = model.predict(test_data.data)
```

We can predict the labels of the test set in the las line of the code.

Here are the predicted category names:

```
print(np.array(test_data.target_names)[predicted_categories])
array(['rec.autos', 'sci.crypt', 'alt.atheism', ...,
'rec.sport.baseball',
'comp.sys.ibm.pc.hardware',
'soc.religion.christian'],dtype='<U24')
```

Let's construct a multi-class confusion matrix to check if the model is suitable or if it only predicts certain text types correctly.

```
# plotting the confusion matrix
mat = confusion_matrix(test_data.target,
predicted_categories)
sns.heatmap(mat.T, square = True, annot=True, fmt =
"d",
xticklabels=train_data.target_names,yticklabels=train_
data.target_names)
plt.xlabel("true labels")
plt.ylabel("predicted label")
plt.show()
print("Accuracy:
{}".format(accuracy_score(test_data.target,
predicted_categories)))
Accuracy: 0.7738980350504514
```

- Naive Bayes is a powerful machine learning algorithm that you can use in Python to create your own spam filters and text classifiers. Naive Bayes classifiers are simple and robust probabilistic classifiers that are particularly useful for text classification tasks. The Naive Bayes algorithm relies on an assumption of conditional independence of features given a class, which is often a good first approximation to real-world phenomena.
- Naive Bayes is becoming a popular text classification technique that can quickly provide a somewhat accurate "guess" as to the category of a document. It is a probabilistic classifier and can give very impressive results. It also scales nicely, allowing you to process thousands of documents. Its ability to keep up with new words makes it more accurate in predicting categories than other popular methods.

### 4.1.3 K-Nearest Neighbor

- K-Nearest Neighbors (KNN) is a simple and intuitive algorithm used for text classification and other machine learning tasks. It is a non-parametric and lazy learning algorithm, meaning it doesn't make explicit assumptions about the underlying data distribution and doesn't build an explicit model during the training phase. Instead, it memorizes the training data and class labels to make predictions on new, unseen data.
- The basic idea behind KNN is to classify a new data point based on the majority class of its "k" nearest neighbors in the feature space. For text classification, the feature space typically consists of numerical representations of the text, such as the Bag-of-Words (BoW) or Term Frequency-Inverse Document Frequency (TF-IDF) vectors.



- Here's an overview of how KNN works for text classification:
  - **Data Preprocessing:** As with other text classification approaches, the first step is to preprocess the text data, including tokenization, lowercase conversion, punctuation removal, and stop word removal.
  - **Feature Extraction:** Convert the preprocessed text data into numerical feature vectors, such as BoW or TF-IDF.
  - **Distance Metric:** Choose an appropriate distance metric to measure the similarity or distance between feature vectors. Common distance metrics for text data include Euclidean distance, Manhattan distance, or Cosine similarity.
  - **Choose "k":** Select the value of "k," which represents the number of nearest neighbors to consider when making predictions. The value of "k" is a hyperparameter that can be chosen through cross-validation or other tuning methods.
  - **Classification:** For each new document to classify, find the "k" nearest neighbors in the training set based on the chosen distance metric. These neighbors are the training documents with the closest feature vectors to the new document.
  - **Majority Voting:** Determine the majority class among the "k" nearest neighbors. The new document is assigned the class label that appears most frequently among its neighbors. In case of a tie, the class label of the nearest neighbor with the smallest distance can be used.
- KNN is an instance-based algorithm, which means it doesn't have a traditional training phase. The entire training dataset is kept in memory, making the prediction phase potentially slower for large datasets. However, it can be computationally efficient for smaller datasets.
- Let's first understand the term neighbors here. The **closeness / proximity** amongst samples of data determines their neighborhood. There are several ways to calculate the proximity/distance between data points depending on the problem to solved. Most familiar and popular is straight-line distance (Euclidean Distance).
- Generally, neighbors share similar characteristics and behavior that's why they can be treated as they belong to the same group.
- This is the main idea of this simple supervised learning classification algorithm.
- Now, for the K in KNN algorithm that is we consider the K-Nearest Neighbors of the unknown data we want to classify and assign it the group appearing majorly in those K neighbors. For K=1, the unknown/unlabeled data will be assigned the class of its closest neighbor.
- We want to select a value of K that is reasonable and not something too big (it will predict the class having majority among all data samples) or something too small.

Let's see how this works on this example dataset of music fans,

| NAME | AGE | GENDER | MUSIC BAND |
|---|---|---|---|
| Amantha | 19 | F | Coldplay |
| Brendon | 23 | M | Coldplay |
| Nate | 24 | M | LinkinPark |
| Sam | 30 | M | LininPark |
| Betty | 16 | F | Coldplay |
| Christine | 18 | F | LinkinPark |
| Gin | 22 | F | LinkinPark |
| Ken | 18 | M | Coldplay |
| Susy | 15 | F | Coldplay |

- Now, we have a person named Gary who is a 23 year male and we want to predict which band will he like more. Here's how we can use the KNN algorithm,
- Firstly we'll have to translate gender to some numbers for the distance/ proximity relation needed for finding neighbors.

We do this by translating male->0 and female->1.

For each data entry distance is calculated from Gary and distance for ith data is given as,

```
\begin{equation} \sqrt{(age_i - age_{Gary})^2 +
(gender_i - gender_{Gary})^2} \end{equation}
```

| NAME | AGE | GENDER | MUSIC BAND | DISTANCE^2 |
|---|---|---|---|---|
| Amantha | 19 | 1 | Coldplay | 17 |
| Brendon | 23 | 0 | Coldplay | 0 |
| Nate | 24 | 0 | LinkinPark | 1 |
| Sam | 30 | 0 | LininPark | 49 |
| Betty | 16 | 1 | Coldplay | 50 |
| Christine | 18 | 1 | LinkinPark | 26 |
| Gin | 22 | 1 | LinkinPark | 2 |
| Ken | 18 | 0 | Coldplay | 25 |
| Susy | 15 | 1 | Coldplay | 65 |

Let's say, K=3 then the K-Nearest Neighbors are

| NAME | AGE | GENDER | MUSIC BAND | DISTANCE^2 |
|---|---|---|---|---|
| Brendon | 23 | 0 | Coldplay | 0 |
| Nate | 24 | 0 | LinkinPark | 1 |
| Gin | 22 | 1 | LinkinPark | 2 |

LinkinPark is followed more by Gary's Neighbors so we predict that Gary will also like LinkinPark more than Coldplay.

**How K-NN Works in Text?**

- The major problem in classifying texts is that they are mixture of characters and words. We need numerical representation of those words to feed them into our K-NN algorithm to compute distances and make predictions.
- One way of doing that numerical representation is bag of words with tf-idf(Term Frequency - Inverse document frequency).
- Let's say we have our text data represented in feature vectors as,

| ID | CLASS | WORD1 | WORD2 | WORD3 |
|---|---|---|---|---|
| 1 | 0 | 2 | 0 | 3 |
| 2 | 1 | 3 | 1 | 1 |
| 3 | 1 | 0 | 3 | 2 |
| 4 | 0 | 1 | 2 | 1 |

- We will have a feature vector of unlabeled text data and it's distance will be calculated from all these feature vectors of our data-set. Out of them, K-Nearest vectors will be selected and the class having maximum frequency will be labeled to the unlabeled data.

**Making It Work**

**Setup**

For this task, we'll need:

- **Python:** To run our script
- **Pip:** Necessary to install Python packages

Now we can install some packages using pip, open your terminal and type these out.

```
pip install numpy
pip install sklearn
Processing
```

Copy

**Numpy:** Useful mathematical functions Sklearn: Machine learning tools for python

**Code**

Let's import the libraries for the task,

```
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
```

Python

Copy

Now, we'll get the dataset ready,

```
from sklearn.datasets import fetch_20newsgroups
```

Python

Copy

Now, we define the categories we want to classify our text into and define the training data set using sklearn.

```
# We defined the categories which we want to classify
categories = ['rec.motorcycles', 'sci.electronics',
              'comp.graphics', 'sci.med']

# sklearn provides us with subset data for training and testing
train_data = fetch_20newsgroups(subset='train',
                                categories=categories,
shuffle=True, random_state=42)
```



```
print(train_data.target_names)

print("\n".join(train_data.data[0].split("\n")[:3]))
print(train_data.target_names[train_data.target[0]])

# Let's look at categories of our first ten training data
for t in train_data.target[:10]:
    print(train_data.target_names[t])
```

Python

Copy

```
['comp.graphics', 'rec.motorcycles', 'sci.electronics',
'sci.med']
From: kreyling@lds.loral.com (Ed Kreyling 6966)
Subject: Sun-os and 8bit ASCII graphics
Organization: Loral Data Systems
comp.graphics
comp.graphics
comp.graphics
rec.motorcycles
comp.graphics
sci.med
sci.electronics
sci.electronics
comp.graphics
rec.motorcycles
sci.electronics
```

Processing

Copy

Here we are pre-processing on text and generating feature vectors of token counts and then transform into tf-idf representation.

Consider a document containing 100 words wherein the word 'car' appears 7 times.

The term frequency (tf) for phone is then (7 / 100) = 0.07. Now, assume we have 1 million documents and the word car appears in one thousand of these. Then, the inverse document frequency (i.e., IDF) is calculated as log(10,00,000 / 100) = 4. Thus, the Tf-IDF weight is the product of these quantities: 0.07 * 4 = 0.28.

i.e. Apply Vectorizer=> Transformer.

```
# Builds a dictionary of features and transforms documents to feature vectors and convert our text documents to a
# matrix of token counts (CountVectorizer)
count_vect = CountVectorizer()
X_train_counts =
count_vect.fit_transform(train_data.data)

# transform a count matrix to a normalized tf-idf representation (tf-idf transformer)
tfidf_transformer = TfidfTransformer()
X_train_tfidf =
tfidf_transformer.fit_transform(X_train_counts)
```

Python

Copy

We fit our Multinomial Naive Bayes classifier on train data to train it.

```
knn = KNeighborsClassifier(n_neighbors=7)

# training our classifier ; train_data.target will be having numbers assigned for each category in train data
clf = knn.fit(X_train_tfidf, train_data.target)

# Input Data to predict their classes of the given categories
docs_new = ['I have a Harley Davidson and Yamaha.', 'I have a GTX 1050 GPU']
# building up feature vector of our input
X_new_counts = count_vect.transform(docs_new)
# We call transform instead of fit_transform because it's already been fit
X_new_tfidf =
tfidf_transformer.transform(X_new_counts)
```

Python

Copy

Predict the output of our input text by using the classifier we just trained.

```
# predicting the category of our input text: Will give out number for category
predicted = clf.predict(X_new_tfidf)
```

```
for doc, category in zip(docs_new, predicted):
    print('%r => %s' % (doc,
train_data.target_names[category]))
```

'I have a Harley Davidson and Yamaha.' => rec.motorcycles

'I have a GTX 1050 GPU' => sci.med

We now finally evaluate our model by predicting the test data. Also, you'll see how to do all of the tasks of vectorizing, transforming and classifier into a single compund classifier using Pipeline.

```
# We can use Pipeline to add vectorizer -> transformer
-> classifier all in a one compound classifier
text_clf = Pipeline([
    ('vect', CountVectorizer()),
    ('tfidf', TfidfTransformer()),
    ('clf', knn),
])
# Fitting our train data to the pipeline
text_clf.fit(train_data.data, train_data.target)

# Test data
test_data = fetch_20newsgroups(subset='test',
                    categories=categories,
shuffle=True, random_state=42)
docs_test = test_data.data
# Predicting our test data
predicted = text_clf.predict(docs_test)
print('We got an accuracy of',np.mean(predicted ==
test_data.target)*100, '% over the test data.')
```

Python

Copy

We got an accuracy of 82.36040609137056 % over the test data.

### .1.4 Spam Filtering

Spam filtering is a text classification task in the field of Natural Language Processing (NLP) that aims to automatically identify and filter out unsolicited or unwanted messages, commonly known as spam, from legitimate messages (ham) in various communication channels such as emails, text messages, and comments.

- The goal of spam filtering is to create a model that can accurately distinguish between spam and legitimate messages, ensuring that only relevant and desired messages reach the user's inbox or notification system. This task is crucial in reducing information overload, protecting users from malicious content, and improving the overall user experience.
- Here's a general approach to building a spam filter using text classification:
  - **Data Collection:** Gather a large dataset of labeled messages that are classified as spam or ham. This dataset will serve as the basis for training and evaluating the spam filter.
  - **Data Preprocessing:** Preprocess the text data by tokenizing, converting to lowercase, removing punctuation, and filtering out stop words. You might also apply techniques like stemming or lemmatization to reduce words to their root forms.
  - **Feature Extraction:** Represent the preprocessed text messages as numerical features. Common approaches for spam filtering include using the Bag-of-Words (BoW) representation or Term Frequency-Inverse Document Frequency (TF-IDF) vectors.
  - **Model Selection:** Choose an appropriate text classification algorithm, such as Naive Bayes, Support Vector Machines (SVM), Logistic Regression, or more advanced techniques like deep learning models (e.g., Convolutional Neural Networks - CNNs, Recurrent Neural Networks - RNNs).
  - **Model Training:** Split the labeled dataset into training and testing sets. Train the selected model on the training data, providing the messages' numerical features and their corresponding class labels (spam or ham).
  - **Model Evaluation:** Evaluate the performance of the trained spam filter on the testing set using metrics like accuracy, precision, recall, F1-score, and area under the Receiver Operating Characteristic (ROC) curve.
  - **Hyperparameter Tuning:** Depending on the chosen algorithm, you may need to tune hyperparameters to optimize the model's performance. Techniques like cross-validation can help find the best hyperparameter values.



  - **Deployment and Monitoring:** Once you are satisfied with the model's performance, deploy the spam filter in the target application (e.g., email client, messaging platform). Monitor the filter's performance regularly and consider incorporating user feedback to continuously improve its accuracy.
- Spam filtering is an ongoing challenge, as spammers continuously adapt their techniques to evade detection. Therefore, it is essential to update and maintain the spam filter regularly to keep up with new spamming tactics and ensure effective protection against unwanted messages.

### 4.1.5 Support Vector Machine Classifier

- Support Vector Machine (SVM) is a powerful and widely used algorithm for text classification tasks. It is a supervised machine learning algorithm that can effectively handle high-dimensional data, making it suitable for text data, where the number of features (words) can be very large.

Here's how SVM works for text classification:

- **Data Preprocessing:** As with any text classification task, the first step is to preprocess the text data, including tokenization, lowercase conversion, punctuation removal, and stop word removal.
- **Feature Extraction:** Convert the preprocessed text data into numerical feature vectors. Common approaches for text data include the Bag-of-Words (BoW) representation or Term Frequency-Inverse Document Frequency (TF-IDF) vectors. These representations create numerical vectors that capture the frequency of each word in the text, relative to the entire dataset.
- **Label Encoding:** Convert the class labels (e.g., spam and ham) into numerical values, so the SVM algorithm can process them.
- **Model Selection:** Choose the SVM kernel and set hyperparameters. The choice of kernel determines how the SVM algorithm separates the data in the feature space. Common kernel functions for text data include the linear kernel, radial basis function (RBF) kernel, and polynomial kernel.
- **Training the SVM Model:** Feed the labeled training data (text feature vectors and their corresponding class labels) into the SVM classifier. The SVM algorithm aims to find an optimal hyperplane that maximizes the margin between different classes in the feature space. This hyperplane effectively separates the text data into their respective classes.
- **Model Evaluation:** Evaluate the performance of the trained SVM model on a separate test dataset using appropriate evaluation metrics such as accuracy, precision, recall, F1-score, etc.
- **Hyperparameter Tuning:** Depending on the performance of the SVM model, you may need to tune hyperparameters like the regularization parameter (C) for soft-margin SVM, the kernel-specific parameters, and the kernel choice itself.
- **Deployment:** Once the SVM model's performance is satisfactory, deploy it in the target application for text classification tasks.

- SVM is known for its ability to handle high-dimensional data and find optimal decision boundaries, making it suitable for text classification tasks with large vocabularies. However, SVM can be computationally expensive, especially for large datasets. In such cases, techniques like Stochastic Gradient Descent (SGD) and linear SVM can be used to speed up training while still achieving reasonable performance.
- In practice, when dealing with text classification tasks, SVM is often compared to other algorithms such as Naive Bayes, Random Forest, and deep learning models like Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) to find the most suitable approach for the specific problem at hand.

Now lets realize this with a supervised ML model to classify text:

I will be using the Amazon Review Data set which has 10,000 rows of Text data which is classified into "Label 1" and "Label 2". The Data set has two columns "Text" and "Label".

**Step 1 :** Add the Required Libraries

The following libraries will be used ahead in the article. If not available, these can be easily downloaded through their respective websites.

```
import pandas as pd
import numpy as np
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.corpus import stopwords
```

```
from nltk.stem import WordNetLemmatizer
from sklearn.preprocessing import LabelEncoder
from collections import defaultdict
from nltk.corpus import wordnet as wn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn import model_selection, naive_bayes, svm
from sklearn.metrics import accuracy_score
```

**Step 2:** Set random seed

This is used to reproduce the same result every time if the script is kept consistent otherwise each run will produce different results. The seed can be set to any number.

```
np.random.seed(500)
```

**Step 3:** Add the Corpus

The data set can be easily added as a pandas Data Frame with the help of 'read_csv' function. I have set the encoding to 'latin-1' as the text had many special characters.

```
Corpus =
pd.read_csv(r"C:\Users\gunjit.bedi\Desktop\NLP Project\corpus.csv",encoding='latin-1')
```

**Step 4:** Data pre-processing

This is an important step in any data mining process. This basically involves transforming raw data into an understandable format for NLP models. Real-world data is often incomplete, inconsistent, and/or lacking in certain behaviors or trends, and is likely to contain many errors. Data pre-processing is a proven method of resolving such issues. This will help in getting better results through the classification algorithms.

- Below, I have explained the two techniques that are also performed besides other easy to understand steps in data pre-processing:
  - ➢ **Tokenization:** This is a process of breaking a stream of text up into words, phrases, symbols, or other meaningful elements called tokens. The list of tokens becomes input for further processing. NLTK Library has word_tokenize and sent_tokenize to easily break a stream of text into a list of words or sentences, respectively.
  - ➢ **Word Stemming/Lemmatization:** The aim of both processes is the same, reducing the inflectional forms of each word into a common base or root. Lemmatization is closely related to stemming. The difference is that a stemmer operates on a single word without knowledge of the context, and therefore cannot discriminate between words which have different meanings depending on part of speech. However, stemmers are typically easier to implement and run faster, and the reduced accuracy may not matter for some applications.

| Form | Stem | Lemma |
|---|---|---|
| Studies | Studi | Study |
| Studying | Study | Study |
| beautiful | beauti | beautiful |
| beautifully | beauti | beautifully |

**Lemma Performs Better**

- Here's the complete script which performs the aforementioned data pre-processing steps, you can always add or remove steps which best suits the data set you are dealing with:
  - (a) Remove Blank rows in Data, if any
  - (b) Change all the text to lower case
  - (c) Word Tokenization
  - (d) Remove Stop words
  - (e) Remove Non-alpha text
  - (f) Word Lemmatization

```
# Step - a : Remove blank rows if any.
Corpus['text'].dropna(inplace=True)# Step - b : Change all the text to lower case. This is required as python interprets 'dog' and 'DOG' differently
Corpus['text'] = [entry.lower() for entry in Corpus['text']]# Step - c : Tokenization : In this each entry in the corpus will be broken into set of words
Corpus['text']= [word_tokenize(entry) for entry in Corpus['text']]# Step - d : Remove Stop words, Non-Numeric and perfom Word Stemming/Lemmenting.# WordNetLemmatizer requires Pos tags to understand if the word is noun or verb or adjective etc. By default it is set to Noun
tag_map = defaultdict(lambda : wn.NOUN)
tag_map['J'] = wn.ADJ
tag_map['V'] = wn.VERB
tag_map['R'] = wn.ADV#for index,entry in enumerate(Corpus['text']):
    # Declaring Empty List to store the words that follow the rules for this step
    Final_words = []
    # Initializing WordNetLemmatizer()
    word_Lemmatized = WordNetLemmatizer()
    # pos_tag function below will provide the 'tag' i.e if the word is Noun(N) or Verb(V) or something else.
    for word, tag in pos_tag(entry):
        # Below condition is to check for Stop words and consider only alphabets
        if word not in stopwords.words('english') and word.isalpha():
            word_Final = word_Lemmatized.lemmatize(word,tag_map[tag[0]])
            Final_words.append(word_Final)
    # The final processed set of words for each iteration will be stored in 'text_final'
    Corpus.loc[index,'text_final'] = str(Final_words)
```



| Raw Text | Pre-processed Text |
|---|---|
| Stunning even for the non-gamer: This sound track was beautiful! It paints the senery in your mind so well I would recomend it even to people who hate video game music! I have played the game Chrono Cross but out of all of the games I have ever played it has the best music! It backs away from crude keyboarding and takes a fresher step with grate guitars and soulful orchestras. It would impress anyone who cares to listen! ^-^ | ['stun', 'even', 'sound', 'track', 'beautiful', 'paint', 'senery', 'mind', 'well', 'would', 'recomend', 'even', 'people', 'hate', 'video', 'game', 'music', 'play', 'game', 'chrono', 'cross', 'game', 'ever', 'play', 'best', 'music', 'back', 'away', 'crude', 'keyboarding', 'take', 'fresh', 'step', 'grate', 'guiter', 'soulful', 'orchestra', 'would', 'impress', 'anyone', 'care', 'listen'] |

Text after all the pre-processing steps are performed

**Step 5:** Prepare Train and Test Data sets

- The Corpus will be split into two data sets, Training and Test. The training data set will be used to fit the model and the predictions will be performed on the test data set.This can be done through the *train_test_split* from the *sklearn* library. The Training Data will have 70% of the corpus and Test data will have the remaining 30% as we have set the parameter *test_size=0.3* .

```
Train_X, Test_X, Train_Y, Test_Y =
model_selection.train_test_split(Corpus['text_final'],Corpus['label'],test_size=0.3)
```

**Train_X** → Training Data Predictors
**Train_Y** → Training Data Target
**Test_X** → Test Data Predictors
**Test_Y** → Test Data Target
Content of each Data Set

**Step 6:** Encoding

Label encode the target variable — This is done to transform Categorical data of string type in the data set into numerical values which the model can understand.

```
Encoder = LabelEncoder()
Train_Y = Encoder.fit_transform(Train_Y)
Test_Y = Encoder.fit_transform(Test_Y)
```

| Raw Labels | Encoded Labels |
|---|---|
| __label__1 | 0 |
| __label__2 | 1 |

Text Encoding

**Step 7:** Word Vectorization

It is a general process of turning a collection of text documents into numerical feature vectors. Their are many methods to convert text data to vectors which the model can understand but by far the most popular method is called TF-IDF. This is an acronym than stands for "Term Frequency — Inverse Document" Frequency which are the components of the resulting scores assigned to each word.

- **Term Frequency:** This summarizes how often a given word appears within a document.
- **Inverse Document Frequency:** This down scales words that appear a lot across documents.

Without going into the math, TF-IDF are word frequency scores that try to highlight words that are more interesting, e.g. frequent in a document but not across documents.

The following syntax can be used to first fit the TG-IDF model on the whole corpus. This will help TF-IDF build a vocabulary of words which it has learned from the corpus data and it will assign a unique integer number to each of these words. Their will be maximum of 5000 unique words / features as we have set parameter max_features = 5000.

Finally we will transform Train_X and Test_X to vectorized Train_X_Tfidf and Test_X_Tfidf. These will now contain for each row a list of unique integer number and its associated importance as calculated by TF-IDF.

```
Tfidf_vect = TfidfVectorizer(max_features=5000)
Tfidf_vect.fit(Corpus['text_final'])Train_X_Tfidf = Tfidf_vect.transform(Train_X)
Test_X_Tfidf = Tfidf_vect.transform(Test_X)
```

You can use the below syntax to see the vocabulary that it has learned from the corpus

```
print(Tfidf_vect.vocabulary_)
```

This will give an output as

{'even': 1459, 'sound': 4067, 'track': 4494, 'beautiful': 346, 'paint': 3045, 'mind': 2740, 'well': 4864, 'would': 4952, 'recomend': 3493, 'people': 3115, 'hate': 1961, 'video': 4761 .....}

And you can directly print the vectorized data to see how it looks like

```
print(Train_X_Tfidf)
```

```
(0,4983) 0.0832939943173435
(0,4975) 0.12608576963017
(0,4952) 0.048118300438349
(0,4937) 0.056890321750098
(0,4864) 0.107107227363495
(0,4844) 0.0642585627590296
(0,4549) 0.132670533563672
(0,4521) 0.136687478016585
(0,4443) 0.049634679106099
(0,4366) 0.0961447380114 76
(0,4302) 0.136687478016585
  :
  :
```

**Output:** 1: Row number of 'Train_X_Tfidf', 2: Unique Integer number of each word in the first row, 3: Score calculated by TF-IDF Vectorizer

Now our data sets are ready to be fed into different classification Algorithms.

**Step 7:** Use the ML Algorithms to Predict the outcome

```
# fit the training dataset on the NB classifier
Naive = naive_bayes.MultinomialNB()
Naive.fit(Train_X_Tfidf,Train_Y)# predict the labels on validation dataset
predictions_NB = Naive.predict(Test_X_Tfidf)# Use accuracy_score function to get the accuracy
print("Naive Bayes Accuracy Score -> ",accuracy_score(predictions_NB, Test_Y)*100)
```

**Output:**

Naive Bayes Accuracy Score -> 83.1%

```
# Classifier - Algorithm – SVM
# fit the training dataset on the classifier
SVM = svm.SVC(C=1.0, kernel='linear', degree=3, gamma='auto')
SVM.fit(Train_X_Tfidf,Train_Y)# predict the labels on validation dataset
predictions_SVM = SVM.predict(Test_X_Tfidf)# Use accuracy_score function to get the accuracy
print("SVM Accuracy Score -> ",accuracy_score(predictions_SVM, Test_Y)*100)
```

**Output:**

SVM Accuracy Score -> 84.6%

### 4.1.6 Vector Space classification using Hyperplanes and Kernel Function

**1. Hyperplanes**

- Vector Space Classification using hyperplanes is a fundamental concept in machine learning and is widely used in various classification tasks, including text classification. The idea is to represent data points as vectors in a multi-dimensional space, where each dimension corresponds to a feature of the data. A hyperplane is a subspace of one dimension less than the original space that acts as a decision boundary to separate data points of different classes.
- Let's consider a binary classification problem, where we have two classes, Class 1 and Class 2, and each data point can be represented as a vector $x$ in an $n$-dimensional feature space:

$$x = [x_1, x_2, \ldots, x_n]$$

- The hyperplane that separates these two classes is represented by a linear equation:

$$w \cdot x + b = 0$$

Where:

$w$ is the weight vector of the hyperplane, perpendicular to the hyperplane's orientation.

$\cdot$ denotes the dot product between $w$ and $x$.

$b$ is the bias term, which represents the offset of the hyperplane from the origin along the $w$ vector.



- For a data point $x$ to be classified, we evaluate the following equation:

$$f(x) = w \cdot x + b$$

where,

If $f(x) > 0$, the data point lies on one side of the hyperplane (positive side) and is classified as Class 1.

If $f(x) < 0$, the data point lies on the other side of the hyperplane (negative side) and is classified as Class 2.

- The magnitude of $f(x)$ indicates how far the data point is from the hyperplane. A large positive value means the point is far on the positive side, and a large negative value means it's far on the negative side.
- To train the classifier, we use a labeled training dataset with n-dimensional feature vectors and corresponding class labels (1 for Class 1 and -1 for Class 2). The goal is to find the optimal values of $w$ and $b$ that maximize the margin between the two classes, ensuring that data points from different classes are well-separated.
- SVM (Support Vector Machine) is a popular algorithm that uses the concept of hyperplanes to find the optimal decision boundary. It finds the hyperplane with the maximum margin while minimizing classification errors. The data points that lie closest to the hyperplane and are called support vectors, and they play a crucial role in determining the hyperplane's position.
- The training process involves solving an optimization problem to find the best values of $w$ and $b$, which is often done using techniques like gradient descent or the Sequential Minimal Optimization (SMO) algorithm. Once the hyperplane parameters are learned from the training data, new data points can be classified using the $f(x)$ equation as described earlier.
- In the context of Information Retrieval (IR) and Natural Language Processing (NLP), vector space classification using hyperplanes is often applied to solve various text-related tasks. One common application is text classification, where the goal is to assign predefined categories or labels to documents based on their content.
- In text classification for IR/NLP, the data points are typically represented as high-dimensional feature vectors, with each dimension corresponding to a term (word or n-gram) present in the document. The value in each dimension can be a count (in the Bag-of-Words representation) or a weighted value (in the TF-IDF representation) reflecting the importance of the term in the document. This representation converts the text data into a numerical format suitable for machine learning algorithms.
- The hyperplane in this context is a decision boundary that separates documents belonging to different classes. The linear equation representing the hyperplane is similar to the previous formulation:

$$w \cdot x + b = 0$$

where,

$w$ is the weight vector that represents the orientation of the hyperplane in the feature space.

$x$ is the feature vector of the document to be classified.

$\cdot$ denotes the dot product between $w$ and $x$.

$b$ is the bias term that represents the offset of the hyperplane from the origin along the $w$ vector.

- In the context of text classification, the SVM algorithm is commonly used to find the optimal hyperplane. SVM aims to maximize the margin between documents of different classes while minimizing classification errors. The hyperplane is determined by finding the support vectors, which are the data points that lie closest to the decision boundary.
- Training the SVM involves optimizing the hyperplane's parameters ($w$ and $b$) using the labeled training dataset. The feature vectors of the documents in the training set are used to learn the optimal weights, and the bias term is determined during the training process.
- Once the SVM model is trained, it can be used to classify new documents into their respective classes by evaluating the following equation:

$$f(x) = w \cdot x + b$$

where,

If $f(x) > 0$, the document is classified into one class.

If $f(x) < 0$, the document is classified into the other class.

- Text classification using SVM with hyperplanes is a powerful technique in IR/NLP, and it has been successfully applied to tasks such as sentiment analysis, topic categorization, spam filtering, and intent detection in chatbots, among others. The ability to handle high-dimensional text data and find optimal decision boundaries makes SVM an essential tool for various text-related applications.

**2. Kernel Function**

- In the context of Support Vector Machines (SVM) and other machine learning algorithms, a kernel function is a mathematical function that allows data points to be transformed into a higher-dimensional feature space, where they might become more easily separable by a linear decision boundary. The kernel function is essential in the kernel trick, which enables SVM to efficiently find nonlinear decision boundaries without explicitly computing the transformations in the higher-dimensional space.
- The basic idea of the kernel trick is to compute the dot product between two data points in the higher-dimensional space without explicitly calculating the transformation of each data point. This saves computational resources and memory, as it avoids the need to deal with the potentially large feature vectors in the higher-dimensional space.
- In the context of SVM, the original linear equation for the hyperplane separating two classes is:

  $w \cdot x + b = 0$

  where,

  w is the weight vector.

  x is the feature vector of the data point.

  · denotes the dot product.
- When a kernel function is applied, the equation for the hyperplane becomes:

  $\Sigma (\alpha_i y_i K(x_i, x)) + b = 0$

  where:

  $\alpha_i$ is a Lagrange multiplier corresponding to the $i^{th}$ support vector.

  $y_i$ is the class label of the ith support vector.

  $K(x_i, x)$ is the kernel function, which computes the dot product between the feature vectors of $x_i$ and x in the higher-dimensional space.
- The kernel function is chosen based on the problem at hand, and different kernel functions have different properties that can influence the SVM's performance. Some common kernel functions include:
  - **Linear Kernel ($K(x_i, x) = x_i \cdot x$):** The linear kernel corresponds to the standard dot product in the original feature space.
  - **Polynomial Kernel ($K(x_i, x) = (\gamma x_i \cdot x + r)^d$):** The polynomial kernel allows SVM to learn polynomial decision boundaries of degree d.
  - **Radial Basis Function (RBF) Kernel ($K(x_i, x) = \exp(-\gamma||x_i - x||^2)$):** The RBF kernel is effective in capturing complex and nonlinear decision boundaries. It uses a similarity measure based on the Euclidean distance between data points.
  - **Sigmoid Kernel ($K(x_i, x) = \tanh(\gamma x_i \cdot x + r)$):** The sigmoid kernel can be used to map data into a space with hyperbolic tangent activation.
- The choice of the kernel function and its hyperparameters (such as γ, d, and r) significantly impact the SVM's performance, so it is important to experiment with different kernels and perform hyperparameter tuning to achieve the best results for a given problem.

## 4.2 TEXT CLUSTERING

### 4.2.1 Clustering vs Classification

Text clustering and text classification are both techniques used in Natural Language Processing (NLP) to analyze and organize text data. However, they serve different purposes and have distinct approaches:

**1. Text Clustering:**

- Clustering is an unsupervised learning technique that groups similar data points together based on their features without any predefined labels or categories.
- In the context of text data, text clustering involves organizing a collection of documents into clusters, where documents within the same cluster are more similar to each other than to documents in other clusters.
- The similarity between documents is typically measured using distance metrics such as cosine similarity, Euclidean distance, or Jaccard similarity, depending on the representation of the text data (e.g., Bag-of-Words, TF-IDF, word embeddings).
- Common clustering algorithms for text data include k-means, hierarchical clustering, and density-based clustering methods like DBSCAN.
- Text clustering is useful for tasks such as document organization, topic modeling, and identifying hidden structures in large text corpora.



**2. Text Classification:**

- Classification is a supervised learning technique used to categorize text documents into predefined classes or labels based on their content and features.
- In text classification, each document in the dataset is labeled with a known class, and the goal is to train a model that can accurately predict the class of unseen documents.
- The text data is represented by numerical features (e.g., BoW, TF-IDF, word embeddings), and a classifier (e.g., SVM, Naive Bayes, deep learning models) is trained on the labeled data to learn the decision boundaries between different classes.
- The performance of the classifier is evaluated using metrics like accuracy, precision, recall, and F1-score.
- Text classification is commonly used for tasks such as sentiment analysis, spam detection, topic categorization, and intent detection in chatbots.

| Aspect | Text Clustering | Text Classification |
|---|---|---|
| Task Type | Unsupervised learning | Supervised learning |
| Objective | Group similar documents together | Categorize documents into predefined classes/labels |
| Labeled Data | Not required | Required |
| Input Data | Collection of text documents | Collection of text documents with labeled classes |
| Algorithm | k-means, hierarchical clustering, DBSCAN, etc. | SVM, Naive Bayes, deep learning models, etc. |
| Evaluation | Internal measures (e.g., silhouette score) | External metrics (e.g., accuracy, precision, recall, F1-score) |
| Use Cases | Topic modeling, document organization, finding hidden structures | Sentiment analysis, spam detection, topic categorization, intent detection in chatbots |

### 4.2.2 Partitioning Methods

Text clustering is the process of grouping similar text documents together based on their content. One category of clustering algorithms commonly used for text data is partitioning methods. Partitioning methods aim to divide the data into a fixed number of clusters, where each data point belongs to exactly one cluster. These methods are efficient and relatively easy to implement.

This clustering method classifies the information into multiple groups based on the characteristics and similarity of the data. Its the data analysts to specify the number of clusters that has to be generated for the clustering methods. In the partitioning method when database(D) that contains multiple (N) objects then the partitioning method constructs user-specified (K) partitions of the data in which each partition represents a cluster and a particular region. There are many algorithms that come under partitioning method some of the popular ones are K-Mean, PAM (K-Medoids), CLARA algorithm (Clustering Large Applications) etc.

**Here are Some Well-Known Partitioning Methods for Text Clustering:**

**1. K-Means Clustering:**

- K-Means is one of the most popular partitioning algorithms for text clustering.
- It requires specifying the number of clusters (K) in advance.
- The algorithm initializes K centroids randomly, and then iteratively assigns each data point to the nearest centroid and updates the centroids' positions based on the assigned points.
- The process continues until the centroids stabilize or a maximum number of iterations is reached.
- K-Means aims to minimize the sum of squared distances between data points and their assigned centroids.

**K-Mean (A Centroid Based Technique):** The K means algorithm takes the input parameter K from the user and partitions the dataset containing N objects into K clusters so that resulting similarity among the data objects inside the group (intracluster) is high but the similarity of data objects with the data objects from outside the cluster is low (intercluster). The similarity of the cluster is determined with respect to the mean value of the cluster. It is a type of square error algorithm. At the start randomly k objects from the dataset are chosen in which each of the objects represents a cluster mean (centre). For the rest of the data objects, they are assigned to the nearest cluster based on their distance from the cluster mean. The new mean of each of the cluster is then calculated with the added data objects.

**Algorithm: K Mean:**

**Input:**

K: The number of clusters in which the dataset has to be divided

D: A dataset containing N number of objects

**Output:**

A dataset of K clusters

**Method:**

1. Randomly assign K objects from the dataset(D) as cluster centres(C)
2. (Re) Assign each object to which object is most similar based upon mean values.
3. Update Cluster means, i.e., Recalculate the mean of each cluster with the updated values.
4. Repeat Step 2 until no change occurs.

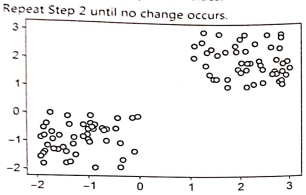



**Fig. 4.1: K-mean clustering**

**Flowchart:**

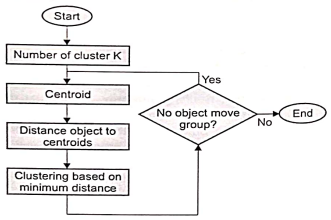



**Fig. 4.2: K-mean clustering**

**Example:** Suppose we want to group the visitors to a website using just their age as follows:

16, 16, 17, 20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66

**Initial Cluster:**

K=2

Centroid(C1) = 16 [16]

Centroid(C2) = 22 [22]

**Note:** These two points are chosen randomly from the dataset.

**Iteration-1:**

C1 = 16.33 [16, 16, 17]

C2 = 37.25 [20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66]

**Iteration-2:**

C1 = 19.55 [16, 16, 17, 20, 20, 21, 21, 22, 23]

C2 = 46.90 [29, 36, 41, 42, 43, 44, 45, 61, 62, 66]

**Iteration-3:**

C1 = 20.50 [16, 16, 17, 20, 20, 21, 21, 22, 23, 29]

C2 = 48.89 [36, 41, 42, 43, 44, 45, 61, 62, 66]

**Iteration-4:**

C1 = 20.50 [16, 16, 17, 20, 20, 21, 21, 22, 23, 29]

C2 = 48.89 [36, 41, 42, 43, 44, 45, 61, 62, 66]

No change Between Iteration 3 and 4, so we stop. Therefore we get the clusters **(16-29)** and **(36-66)** as 2 clusters we get using K Mean Algorithm.

2. **K-Medoids Clustering (PAM - Partitioning Around Medoids):**

- K-Medoids is similar to K-Means, but instead of using centroids, it uses actual data points as representatives of clusters, known as medoids.
- The algorithm randomly selects K data points as initial medoids, and then iteratively replaces medoids with other data points to improve the clustering's quality.
- K-Medoids is more robust than K-Means in the presence of outliers because it directly uses data points as cluster representatives.



3. **CLARA (Clustering Large Applications):**

- CLARA is a sampling-based variant of K-Medoids designed to handle large datasets efficiently.
- It repeatedly samples subsets of the data and applies K-Medoids to each subset to find candidate medoids.
- The best set of medoids is then selected from these candidates.

4. **CLARANS (Clustering Large Applications based upon Randomized Search):**

- CLARANS is another sampling-based variant of K-Medoids, which performs a randomized search for medoids.
- Instead of examining all possible data points for candidate medoids, CLARANS randomly selects data points and explores the neighborhood of current medoids.
- It repeats this process until convergence or a specified number of iterations.

## 4.3 CLUSTERING ALGORITHMS

### 4.3.1 K-Means Clustering

K-Means clustering is a popular and widely used unsupervised machine learning algorithm for partitioning data into K clusters. It is a simple and efficient algorithm that aims to minimize the sum of squared distances between data points and their assigned cluster centroids. K-Means is commonly used for various clustering tasks, including text clustering, image segmentation, and customer segmentation. Let's explore the key steps of the K-Means algorithm:

**Step 1: Initialization:** Choose the number of clusters K that you want to create.

Initialize K cluster centroids randomly. These centroids will act as representatives for the clusters.

**Step 2: Assign Data Points to Clusters:** For each data point in the dataset, calculate its distance to each of the K centroids.

Assign the data point to the cluster whose centroid is the closest (usually using Euclidean distance or other distance metrics).

**Step 3: Update Cluster Centroids:** After all data points have been assigned to clusters, update the centroids of each cluster.

Calculate the mean (average) of all data points belonging to each cluster. This new mean becomes the updated centroid for that cluster.

**Step 4: Reassign Data Points:** Repeat the assignment step, but this time using the updated centroids as references.

Reassign each data point to the cluster whose updated centroid is closest.

**Step 5: Convergence:** Repeat steps 3 and 4 iteratively until the assignments no longer change significantly or a maximum number of iterations is reached.

The algorithm converges when the centroids stabilize, and the data points' assignments remain consistent between iterations.

**Step 6: Final Result:** After convergence, the algorithm outputs K clusters, and each data point belongs to one of the K clusters.

K-Means is an iterative algorithm and may converge to local optima, depending on the initial centroid positions. To mitigate this, K-Means is often run multiple times with different initializations, and the best result (with the lowest total within-cluster variance) is selected.

In text clustering, K-Means can be applied after converting the text documents into numerical feature vectors (e.g., Bag-of-Words or TF-IDF representations). The algorithm groups similar documents together based on their content, and the resulting clusters can provide insights into the underlying structure and patterns in the text data.

```
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import TruncatedSVD
import matplotlib.pyplot as plt

# Load the 20 Newsgroups dataset
newsgroups = fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'))
documents = newsgroups.data
# Convert text data into numerical vectors using TF-IDF
vectorizer = TfidfVectorizer(max_df=0.5, min_df=2, stop_words='english')
```

```
X = vectorizer.fit_transform(documents)
# Reduce dimensionality using TruncatedSVD to speed up K-Means
svd = TruncatedSVD(n_components=100, random_state=42)
X_reduced = svd.fit_transform(X)

# Perform K-Means clustering
num_clusters = 20  # Number of clusters (topics)
kmeans = KMeans(n_clusters=num_clusters, random_state=42)
kmeans.fit(X_reduced)

# Get the cluster assignments and centroids
cluster_labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Print the top terms in each cluster (feature names after SVD)
num_top_terms = 10
original_feature_names = vectorizer.get_feature_names_out()
top_terms_per_cluster = []
for i in range(num_clusters):
    cluster_indices = np.where(cluster_labels == i)[0]
    cluster_centroid = centroids[i]
    top_term_indices = cluster_centroid.argsort()[::-1][:num_top_terms]
    top_terms = [original_feature_names[idx] for idx in top_term_indices]
    top_terms_per_cluster.append(top_terms)
    print(f"Cluster {i + 1}: {', '.join(top_terms)}")

# Visualize the clusters (only if reduced dimensionality to 2D)
if X_reduced.shape[1] == 2:
    plt.figure(figsize=(10, 6))
    for i in range(num_clusters):
        cluster_indices = np.where(cluster_labels == i)[0]
        plt.scatter(X_reduced[cluster_indices, 0], X_reduced[cluster_indices, 1], label=f"Cluster {i + 1}")
    plt.scatter(centroids[:, 0], centroids[:, 1], marker='*', s=200, c='black', label='Centroids')
    plt.xlabel('Dimension 1')
    plt.ylabel('Dimension 2')
    plt.title('K-Means Clustering of 20 Newsgroups Dataset')
    plt.legend()
    plt.show()
```

### 4.3.2 Agglomerative Hierarchical Clustering

Agglomerative Hierarchical Clustering is a bottom-up hierarchical clustering technique that starts with each data point as its own cluster and iteratively merges the closest clusters until all data points belong to a single cluster. The process creates a dendrogram, which is a tree-like structure representing the hierarchical relationships between data points.

Theoretical Steps of Agglomerative Hierarchical Clustering:

**Step 1: Data Representation:** Convert the raw text data into a numerical representation using techniques like TF-IDF or Bag-of-Words.

**Step 2: Distance Matrix:** Compute the pairwise distance between all data points using a distance metric such as Euclidean distance or cosine similarity. This creates a distance matrix.

**Step 3: Initial Clusters:** Assign each data point to its own individual cluster.

**Step 4: Merge Closest Clusters:** Find the two closest clusters based on the distance matrix. The distance between clusters can be measured using different linkage methods like single linkage, complete linkage, or average linkage. Merge these two closest clusters into a new cluster.

**Step 5: Update Distance Matrix:** Recalculate the distances between the newly formed cluster and all other clusters in the distance matrix.

**Step 6: Repeat Merging and Updating:** Repeat steps 4 and 5 iteratively until all data points belong to a single cluster or until a specified number of clusters is reached.

**Step 7: Dendrogram Visualization:** Plot the dendrogram, which visualizes the hierarchical clustering process and shows how clusters are merged at each step.



**Step 8: Select Number of Clusters:** Based on the dendrogram, choose the appropriate number of clusters by cutting the dendrogram at a desired height.

Python Code for Text Clustering using Agglomerative Hierarchical Clustering:

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

# Sample text documents
documents = [
    "I love machine learning",
    "Text clustering is fun",
    "Machine learning is interesting",
    "Natural Language Processing is fascinating",
    "Text data can be analyzed",
    "Machine learning algorithms are powerful",
    "Text clustering and classification are NLP tasks"
]

# Convert text data into numerical vectors using TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)

# Perform Agglomerative Hierarchical Clustering
num_clusters = 2  # Number of clusters
agg_clustering = AgglomerativeClustering(n_clusters=num_clusters, linkage='ward')
cluster_labels = agg_clustering.fit_predict(X.toarray())

# Print the clusters and their documents
for i in range(num_clusters):
    cluster_indices = np.where(cluster_labels == i)[0]
    print(f"Cluster {i + 1}:")
    for idx in cluster_indices:
        print(f" - {documents[idx]}")
    print()

# Create dendrogram for visualization
linked = linkage(X.toarray(), method='ward')
plt.figure(figsize=(10, 6))
dendrogram(linked, labels=documents, orientation='top', distance_sort='descending', show_leaf_counts=True)
plt.title('Agglomerative Hierarchical Clustering Dendrogram')
plt.xlabel('Documents')
plt.ylabel('Distance')
plt.show()
```

### 4.3.3 Expectation Maximization

Text clustering using Expectation-Maximization (EM) typically involves the use of Gaussian Mixture Models (GMMs). GMMs are a probabilistic model that assumes the data points in each cluster are generated from a mixture of several Gaussian distributions. EM is used to estimate the parameters of the Gaussian components and to assign data points to their most likely clusters.

Theoretical Steps for Text Clustering using EM with GMM:

**Step 1: Data Representation:** Convert the raw text data into a numerical representation using techniques like TF-IDF or Bag-of-Words.

**Step 2: Initialize GMM Parameters:** Choose the number of clusters (K) that you want to create.

Initialize the parameters of K Gaussian components. These parameters include the mean, covariance matrix, and mixing coefficient (prior probability) for each component.

**Step 3: Expectation Step (E-Step):** Calculate the likelihood of each data point belonging to each Gaussian component using the current GMM parameters.

Use Bayes' theorem to calculate the posterior probabilities (responsibilities) of each data point for each Gaussian component.

**Step 4: Maximization Step (M-step):** Update the GMM parameters (mean, covariance, mixing coefficient) based on the responsibilities calculated in the E-step.

Re-estimate the mean and covariance of each Gaussian component using the weighted data points assigned to it.

Update the mixing coefficient based on the average responsibility of each component.

**Step 5: Iterative Update:** Repeat the E-step and M-step iteratively until the model parameters converge or until a maximum number of iterations is reached.

**Step 6: Convergence Criterion:** The EM algorithm typically stops when the change in the log-likelihood or the model parameters falls below a specified tolerance threshold, indicating that the algorithm has reached a stable solution.

**Python Code for Text Clustering using EM with GMM:**

For this example, we'll use the scikit-learn library to perform text clustering using Gaussian Mixture Models with Expectation-Maximization.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.mixture import GaussianMixture

# Sample text documents
documents = [
    "I love machine learning",
    "Text clustering is fun",
    "Machine learning is interesting",
    "Natural Language Processing is fascinating",
    "Text data can be analyzed",
    "Machine learning algorithms are powerful",
    "Text clustering and classification are NLP tasks"
]

# Convert text data into numerical vectors using TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)

# Perform Gaussian Mixture Model with Expectation-Maximization
num_clusters = 2  # Number of clusters
gmm = GaussianMixture(n_components=num_clusters, random_state=42)
gmm.fit(X.toarray())

# Get the cluster assignments for each data point
cluster_labels = gmm.predict(X.toarray())

# Print the clusters and their documents
for i in range(num_clusters):
    cluster_indices = np.where(cluster_labels == i)[0]
    print(f"Cluster {i + 1}:")
    for idx in cluster_indices:
        print(f" - {documents[idx]}")
    print()
```

In this example, we first convert the text data into numerical vectors using TF-IDF vectorization. Then, we use scikit-learn's GaussianMixture class to perform Gaussian Mixture Model with Expectation-Maximization for text clustering with num_clusters = 2. The code prints the clusters and documents within each cluster.

Keep in mind that the choice of the number of clusters (K) can significantly impact the clustering results. You may need to experiment with different values of K to obtain the most appropriate number of clusters for your specific task and dataset.

### 4.3.4 Mixture of Gaussians Model

Text clustering using the Mixture of Gaussians (MoG) Model is a probabilistic model that assumes the data points in each cluster are generated from a mixture of several Gaussian distributions. The MoG model allows for soft assignment of data points to clusters, meaning that each data point can have a probability of belonging to each cluster, instead of a hard assignment.

Theoretical Steps for Text Clustering using Mixture of Gaussians Model:

**Step 1: Data Representation:** Convert the raw text data into a numerical representation using techniques like TF-IDF or Bag-of-Words.

**Step 2: Initialize MoG Parameters:** Choose the number of clusters (K) that you want to create.

Initialize the parameters of K Gaussian components. These parameters include the mean, covariance matrix, and mixing coefficient (prior probability) for each component.

**Step 3: Expectation Step (E-step):** Calculate the likelihood of each data point belonging to each Gaussian component using the current MoG parameters.

Use Bayes' theorem to calculate the posterior probabilities (responsibilities) of each data point for each Gaussian component.



**Step 4: Maximization Step (M-step):** Update the MoG parameters (mean, covariance, mixing coefficient) based on the responsibilities calculated in the E-step.

Re-estimate the mean and covariance of each Gaussian component using the weighted data points assigned to it.

Update the mixing coefficient based on the average responsibility of each component.

**Step 5: Iterative Update:** Repeat the E-step and M-step iteratively until the model parameters converge or until a maximum number of iterations is reached.

**Step 6: Convergence Criterion:** The MoG algorithm typically stops when the change in the log-likelihood or the model parameters falls below a specified tolerance threshold, indicating that the algorithm has reached a stable solution.

**Python Code for Text Clustering using Mixture of Gaussians Model:**

For this example, we'll use the scikit-learn library to perform text clustering using the Mixture of Gaussians model.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.mixture import GaussianMixture

# Sample text documents
documents = [
    "I love machine learning",
    "Text clustering is fun",
    "Machine learning is interesting",
    "Natural Language Processing is fascinating",
    "Text data can be analyzed",
    "Machine learning algorithms are powerful",
    "Text clustering and classification are NLP tasks"
]

# Convert text data into numerical vectors using TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)
# Perform Mixture of Gaussians (MoG) Model
num_clusters = 2  # Number of clusters
mog = GaussianMixture(n_components=num_clusters, random_state=42)
mog.fit(X.toarray())

# Get the cluster assignments for each data point
cluster_labels = mog.predict(X.toarray())

# Print the clusters and their documents
for i in range(num_clusters):
    cluster_indices = np.where(cluster_labels == i)[0]
    print(f"Cluster {i + 1}:")
    for idx in cluster_indices:
        print(f" - {documents[idx]}")
    print()
```

In this example, we first convert the text data into numerical vectors using TF-IDF vectorization. Then, we use scikit-learn's GaussianMixture class to perform the Mixture of Gaussians model for text clustering with num_clusters = 2. The code prints the clusters and documents within each cluster.

As with any clustering algorithm, the choice of the number of clusters (K) can significantly impact the clustering results. You may need to experiment with different values of K to obtain the most appropriate number of clusters for your specific task and dataset. Additionally, consider multiple runs with different initializations to mitigate sensitivity to the initial parameter values.

## CASE STUDY: IMPROVING DOCUMENT ORGANIZATION AND RETRIEVAL IN A DIGITAL LIBRARY

- **Abstract:** This case study delves into the challenges faced by digital libraries in efficiently organizing and retrieving documents. The exponential growth of digital content necessitates innovative solutions. The study explores strategies such as metadata enhancement, Natural Language Processing (NLP), and user-centric design to enhance document categorization and retrieval. By amalgamating these approaches, digital libraries can significantly enhance user experience and knowledge discovery.

- **Introduction:** Digital libraries serve as repositories for a vast array of documents. However, as the volume of digital content proliferates, ensuring effective document organization and retrieval becomes paramount. Traditional methods fall short in dealing with the scale and complexity of modern digital libraries. This case study investigates techniques that can revolutionize how documents are organized and accessed.
- **Challenges in Document Organization:** The challenges include metadata inconsistency, lack of contextual understanding, and user-friendliness. Documents often lack comprehensive metadata, impeding accurate categorization. Additionally, keyword-based systems fail to capture nuanced contextual information present in the documents.
- **Metadata Enhancement:** Augmenting metadata through automated means can significantly improve document categorization. Techniques like entity recognition from NLP can identify and tag key entities in documents, enabling more accurate content classification. Additionally, sentiment analysis can determine the tone of documents, aiding emotional categorization.
- **Natural Language Processing (NLP):** NLP techniques such as topic modeling, keyword extraction, and summarization contribute to richer document representations. Topic modeling identifies latent themes within documents, while keyword extraction helps in creating precise document indices. Summarization produces condensed versions of documents, aiding users in gauging relevance quickly.
- **User-Centric Design:** User experience plays a pivotal role in the success of digital libraries. Implementing intuitive search interfaces, personalized recommendations, and interactive visualizations can transform how users interact with the library. Collaborative filtering techniques can offer content suggestions based on user behavior, enhancing serendipitous discovery.
- **Integration and Benefits:** The integration of metadata enhancement, NLP, and user-centric design offers multiple benefits:
  - **Enhanced Discoverability:** Rich metadata and improved content categorization lead to more accurate search results, allowing users to discover relevant documents efficiently.
  - **Contextual Understanding:** NLP techniques provide a deeper understanding of document content, enabling more nuanced categorization and retrieval.
  - **Time Efficiency:** Summarization and advanced indexing enable users to grasp document relevance quickly, saving time during information seeking.
  - **Personalization:** User-centric design leads to personalized experiences, increasing user engagement and satisfaction.
- **Conclusion:** Modern digital libraries must evolve to meet the demands of information seekers. By leveraging metadata enhancement, NLP, and user-centric design, digital libraries can transform into powerful knowledge repositories. The holistic approach presented in this case study fosters accurate organization, meaningful content retrieval, and seamless user interactions, ultimately revolutionizing the way users access and interact with digital content.

**Step 1: Metadata Enhancement with NLP**

Imagine a digital library containing research papers on various topics. Initially, these papers might have basic metadata like title, author, and publication date. To enhance metadata using NLP, you can perform entity recognition and sentiment analysis:

```
import spacy
from textblob import TextBlob

# Load NLP model
nlp = spacy.load("en_core_web_sm")

# Sample document text
document_text = "The study presents a novel approach for sentiment analysis in social media texts."

# Entity recognition
doc = nlp(document_text)
entities = [ent.text for ent in doc.ents]

# Sentiment analysis
blob = TextBlob(document_text)
sentiment_score = blob.sentiment.polarity
```



```
# Update metadata with entities and sentiment score
metadata = {
  "title": "Sentiment Analysis in Social Media",
  "author": "John Doe",
  "publication_date": "2023-08-01",
  "entities": entities,
  "sentiment_score": sentiment_score
}
```

**Step 2: Document Indexing with NLP Techniques**

Next, let's use topic modeling and keyword extraction to create more informative document indices:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

# Sample document texts
documents = [
  "Sentiment analysis in social media",
  "Natural language processing techniques",
  "Topic modeling for document categorization"
]

# Topic modeling
vectorizer = CountVectorizer()
doc_term_matrix = vectorizer.fit_transform(documents)
lda = LatentDirichletAllocation(n_components=2, random_state=0)
topic_matrix = lda.fit_transform(doc_term_matrix)

# Keyword extraction
keywords = [word for word, count in zip(vectorizer.get_feature_names(), doc_term_matrix.sum(axis=0).A1)]

# Update document indices with topics and keywords
document_indices = [
  {"title": "Sentiment Analysis in Social Media", "topics": topic_matrix[0], "keywords": keywords},
  {"title": "Natural Language Processing Techniques", "topics": topic_matrix[1], "keywords": keywords},
  {"title": "Topic Modeling for Document Categorization", "topics": topic_matrix[2], "keywords": keywords}
]
```

**Step 3: User-Centric Design with Personalized Recommendations**

Finally, let's implement a basic user-centric search interface with personalized recommendations:

```
def search_documents(query):
  results = []
  for document in document_indices:
    if query.lower() in document["title"].lower():
      results.append(document)
  return results

def recommend_documents(user_preferences):
  recommended = []
  for document in document_indices:
    if any(topic > 0.5 for topic in document["topics"]):
      recommended.append(document)
  return recommended

# User query and preferences
user_query = "Sentiment Analysis"
user_preferences = {"topics": [0.8, 0.2], "keywords": ["analysis", "social media"]}

# Search and recommend documents
search_results = search_documents(user_query)
recommended_results = recommend_documents(user_preferences)

print("Search Results:")
for result in search_results:
  print(result["title"])
print("\nRecommended Results:")
for result in recommended_results:
  print(result["title"])
```

This example showcases how metadata enhancement, NLP techniques, and user-centric design can work together to improve document organization and retrieval in a digital library. In a real-world scenario, more advanced NLP models, personalized recommendation algorithms, and interactive interfaces would be used for a more comprehensive solution.

## EXERCISE

1. What is text classification, and how is it different from text clustering?
2. Explain the Naive Bayes model and its application in text classification.
3. How does the k-nearest neighbor algorithm work for text classification?
4. Describe the process of spam filtering and the techniques used for it.
5. What is the Support Vector Machine (SVM) classifier, and how is it used in text classification?
6. Explain the concept of vector space classification using hyperplanes in the context of text data.
7. Compare and contrast supervised and unsupervised text clustering methods.
8. Describe the K-Means clustering algorithm and its steps for text clustering.
9. How does Agglomerative Hierarchical Clustering work for text clustering?
10. Discuss the Expectation-Maximization (EM) algorithm and its role in text clustering using Gaussian Mixture Models.
11. Describe the process of converting text data into numerical vectors using TF-IDF.
12. What are some common distance metrics used in text clustering, and when would you use each?
13. How do you choose the appropriate number of clusters for a text clustering task?
14. Discuss the advantages and limitations of the K-Means algorithm for text clustering.
15. How can text clustering be useful in tasks like document organization and topic modeling?
16. Explain the role of kernel functions in Support Vector Machine (SVM) classifiers for text classification. Provide examples of popular kernel functions used in NLP.
17. Describe the process of hierarchical clustering and how it differs from partitioning methods like K-Means for text clustering.
18. Discuss the challenges and potential solutions for handling high-dimensional text data in clustering algorithms.
19. What are the main evaluation metrics used to assess the performance of text classification models? Explain the meaning and significance of precision, recall, F1-score, and accuracy.
20. Compare and contrast Gaussian Mixture Models (GMMs) and K-Means clustering for text clustering. What are the advantages and disadvantages of each approach?

✠ ✠ ✠

UNIT - V

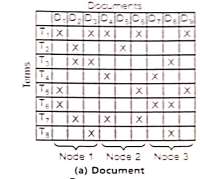
# WEB RETRIEVAL AND WEB CRAWLING

## 5.1 PARALLEL INFORMATION RETRIEVAL

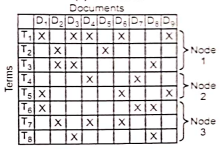
- Information retrieval systems often have to deal with very large amounts of data. They must be able to process many gigabytes or even terabytes of text, and to build and maintain an index for millions of documents. It is clear that, at some point, sophisticated data structures and clever optimizations alone are not sufficient anymore. A single computer simply does not have the computational power or the storage capabilities required for indexing even a small fraction of the World Wide Web.
- In this chapter we examine various ways of making information retrieval systems scale to very large text collections such as the Web. The first part is concerned with parallel query processing, where the search engine's service rate is increased by having multiple index servers process incoming queries in parallel. It also discusses redundancy and fault tolerance issues in distributed search engines. In the second second part, we shift our attention to the parallel execution of off-line tasks, such as index construction and statistical analysis of a corpus of text.
- We explain the basics of MapReduce, a framework designed for massively parallel computations carried out on large amounts of data.

### 5.1.1 Parallel Query Processing

- While nobody knows the exact size of the indexable part of the Web, it is estimated to be at least 100 billion pages. In August 2005, when Yahoo! last disclosed the size of its index, it had reached a total size of 19.2 billion documents (http://www.ysearchblog.com/archives/000172.html)
- There are many ways in which parallelism can help a search engine process queries faster. The two most popular approaches are index partitioning and replication. Suppose we have a total of n index servers. Following the standard terminology, we refer to these servers as nodes. By creating n replicas of the index and assigning each replica to a separate node, we can realize an n-fold increase of the search engine's service rate (its theoretical throughput) without affecting the time required to process a single query. This type of parallelism is referred to as inter-query parallelism, because multiple queries can be processed in parallel but each individual query is processed sequentially. Alternatively, we could split the index into n parts and have each node work only on its own small part of the index. This approach is referred to as intra-query parallelism, because each query is processed by multiple servers in parallel.
- It improves the engine's service rate as well as the average time per query. In this section we focus primarily on methods for intra-query parallelism. We study index partitioning schemes that divide the index into independent parts so that each node is responsible for a small piece of the overall index. The two predominant index partitioning schemes are document partitioning and term partitioning (visualized in Fig. 5.1). In a document-partitioned index, each node holds an index for a subset of the documents in the collection. For instance, the index maintained by node 2 in Fig. 5.1 (a) contains the following docid lists:




**(a) Document**




**(b) Term partitioning**

**Fig. 5.1: The two prevalent index partitioning schemes: document partitioning and term partitioning (shown for a hypothetical index containing 8 terms and 9 documents)**

L1 = 4, 6, L2 = 5, L4 = 4, L5 = 6, L7 = 4, 6

- In a term-partitioned index, each node is responsible for a subset of the terms in the collection. The index stored in node 1 in Fig. 5.1 (b) contains the following lists:

L1 = 1, 3, 4, 6, 9, L2 = 2, 5, L3 = 2, 3, 5

- The two partitioning strategies differ greatly in the way queries are processed by the system. In a document-partitioned search engine, each of the n nodes is involved in processing all queries received by the engine. In a term-partitioned configuration, a query is seen by a given node only if the node's index contains at least one of the query terms.

**Document Partitioning**

- In a document-partitioned index, each index server is responsible for a subset of the documents in the collection. Each incoming user query is received by a frontend server, the receptionist, which forwards it to all n index nodes, waits for them to process the query, merges the search results received from the index nodes, and sends the final list of results to the user. A schematic of a document-partitioned search engine is shown in Fig. 5.2. The main advantage of the document-partitioned approach is its simplicity. Because all index servers operate independently of each other, no additional complexity needs to be introduced into the low-level query processing routines. All that needs to be provided is the receptionist server that forwards the query to the backends and, after receiving the top $k$ search results from each of the n nodes, selects the top $m$, which are then returned to the user (where the value of m is typically chosen by the user and k is chosen by the operator of the search engine). In addition to forwarding queries and search results, the receptionist may also maintain a cache that contains the results for recently/frequently issued queries. If the search engine maintains a dynamic index that allows updates (e.g., document insertions / deletions), then it may even be possible to carry out the updates in a distributed fashion, in which each node takes care of the updates that pertain to its part of the overall index.
- This approach eliminates the need for a complicated centralized index construction/maintenance process that involves the whole index. However, it is applicable only if documents may be assumed to be independent of each other, not if inter-document information, such as hyperlinks and anchor text, is part of the index. When deciding how to divide the collection across the n index nodes, one might be tempted to bias the document-node assignment in some way, for instance, by storing similar documents in the same node. We have seen that the index can be compressed better if the documents in the collection are reordered according to their URL so that documents with similar URLs receive nearby docids. Obviously, this method is most effective if all pages from the same domain are assigned to the same node.
- The problem with this approach is that it may create an imbalance in the load distribution of the index servers. If a given node primarily contains documents associated with a certain topic, and this topic suddenly becomes very popular among users, then the query processing load for that node may become much higher than the load of the other nodes. In the end this will lead to a suboptimal utilization of the available resources. To avoid this problem, it is usually best to not try anything fancy but to simply split the collection into n completely random subsets.
- Once the index has been partitioned and each subindex has been loaded into one of the nodes, we have to make a decision regarding the per-node result set size k. How should k be chosen with respect to m, the number of search results requested by the user? Suppose the user has asked for m = 100 results. If we want to be on the safe side, we can have each index node return k = 100 results, thus making sure that all of the top 100 results overall are received by the receptionist. However, this would be a poor decision for two reasons:



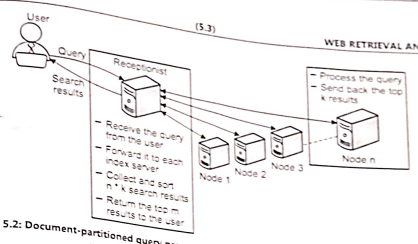



**Fig. 5.2: Document-partitioned query processing with one receptionist and several index servers. Each incoming query is forwarded to all index servers.**

1. It is quite unlikely that all of the top 100 results come from the same node. By having each index server return 100 results to the receptionist, we put more load on the network than necessary.
2. The choice of k has a non-negligible effect on query processing performance. Because of that decreasing the result set size from k = 100 to k = 10 can reduce the average CPU time per query by around 15%, when running queries against a frequency index for GOV2.

**Term Partitioning**

- Although document partitioning is often the right choice and scales almost linearly with the number of nodes, it can unfold its true potential only if the index data found in the individual nodes are stored in main memory or any other low-latency random-access storage medium, such as flash memory, but not if it is stored on disk.
- Consider a document-partitioned search engine in which all postings lists are stored on disk. Suppose each query contains 3 words on average, and we want the search engine to handle a peak query load of 100 queries per second. Due to queueing effects, we usually cannot sustain a utilization level above 50%, unless we are willing to accept occasional latency jumps. Thus, a query load of 100 qps translates into a required service rate of at least 200 qps or equivalently 600 random access operations per second (one for each query term). Assuming an average disk seek latency of 10 ms, a single hard disk drive cannot perform more than 100 random access operations per second, a factor of 6 less than what is required to achieve the desired throughput. We could try to circumvent this limitation by adding more disks to each index node, but equipping each server with six hard disks may not always be practical. Moreover, it is obvious that we will never be able to handle loads of more than a few hundred queries per second, regardless of how many nodes we add to the system.
- Term partitioning addresses the disk seek problem by splitting the collection into sets of terms instead of sets of documents. Each index node $v_i$ is responsible for a certain term set $T_i$ and is involved in processing a given query only if one or more query terms are members of $T_i$. Our discussion of term-partitioned query processing is based upon the pipelined architecture proposed by Moffat. In this architecture, queries are processed in a term-at-a-time fashion.
- Suppose a query contains $q$ terms $t_1, t_2, \ldots, t_q$. Then the receptionist will forward the query to the node $v(t_1)$ responsible for the term $t_1$. After creating a set of document score accumulators from $t_1$'s postings list, $v(t_1)$ forwards the query, along with the accumulator set, to the node $v(t_2)$ responsible for $t_2$, which updates the accumulator set, sends it to $v(t_3)$, and so forth. When the last node in this pipeline, $v(t_q)$, is finished, it sends the final accumulator set to the receptionist. The receptionist then selects the top m search results and returns them to the user. A schematic of this approach is shown in Fig. 5.3.
- Many of the optimizations used for sequential term-at-a-time query processing also apply to term-partitioned query processing: infrequent query terms should be processed first; accumulator pruning strategies should

be applied to keep the size of the accumulator set, and thus the overall network traffic, under control, impact ordering can be used to efficiently identify the most important postings for a given term.

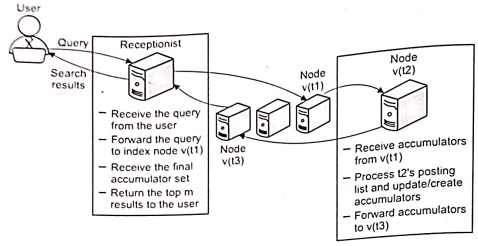



**Fig. 5.3: Therm-partitioned query processing with one receptionist and four index servers. Each incoming query is passed from index server to index server, depending on the terms found in the query (shown for a query containing three terms)**

Note that the pipelined query processing architecture outlined above does not use intra-query parallelism, as each query at a given point in time is processed by only a single node. Therefore, although it can help increase the system's theoretical throughput, term partitioning, in the simple form described here, does not necessarily decrease the search engine's response time. To some extent this limitation can be addressed by having the receptionist send prefetch instructions to the nodes $v(t_2)$, …, $v(t_q)$ at the same time it forwards the query to $v(t_1)$. This way, when a node $v(t_i)$ receives the accumulator set, some of $t_i$'s postings may already have been loaded into memory and the query can be processed faster. Despite its potential performance advantage over the document-partitioned approach, at least for on-disk indices, term partitioning has several shortcomings that make it difficult to use the method in practice:

- **Scalability:** As the collection becomes bigger, so do the individual postings lists. For a corpus composed of a billion documents, the postings list for a frequent term, such as "computer" or "internet", can easily occupy several hundred megabytes. Processing a query that contains one or more of these frequent terms will require at least a few seconds, far more than what most users are accustomed to. In order to solve this problem, large postings lists need to be cut into smaller chunks and divided among several nodes, with each node taking care of a small part of the postings list and all of them working in parallel. Unfortunately, this complicates the query processing logic quite a bit.
- **Load Imbalance:** Term partitioning suffers from an uneven load across the index nodes. The load corresponding to a single term is a function of the term's frequency in the collection as well as its frequency in users' queries. If a term has a long postings list and is also very popular in search queries, then the corresponding index node may experience a load that is much higher than the average load in the system. To address this problem, postings lists that are responsible for a high fraction of the overall load should be replicated and distributed across multiple nodes. This way, the computational load associated with a given term can be shared among several machines, albeit at the cost of increased storage requirements.
- **Term-at-a-Time:** Perhaps the most severe limitation of the term-partitioned approach is its inability to support efficient document-at-a-time query processing. In order to realize document-at-a-time scoring on top of a term-partitioned index, entire postings lists, as opposed to pruned accumulator sets, would need to be sent across the network. This is impractical, due to the size of the postings lists. Therefore, ranking methods that necessitate a document-at-a-time approach, such as the proximity ranking function from are incompatible with a term-partitioned index.



### 5.1.2 Mapreduce

- Apart from processing search queries, there are many other data-intensive tasks that need to be carried out by a large-scale search engine. Such tasks include building and updating the index; identifying duplicate documents in the corpus; and analyzing the link structure of the document collection
- MapReduce is a framework developed at Google that is designed for massively parallel computations (thousands of machines) on very large amounts of data (many terabytes), and that can accomplish all of the tasks listed above. MapReduce was first presented by Dean and Ghemawat. In addition to a high-level overview of the framework, their paper includes information about many interesting implementation details and performance optimizations.

**The Basic Framework**

- MapReduce was inspired by the map and reduce functions found in functional programming languages, such as Lisp. The map function takes as its arguments a function f and a list of elements $l = l1, l2, \ldots, ln$. It returns a new list

  $$map(f,l) = f(l1), f(l2), \ldots, f(ln).$$

  The reduce function (also known as fold or accumulate) takes a function g and a list of elements $l = l1, l2, \ldots, ln$. It returns a new element $l$, such that

  $$l = reduce(g,l) = g(l1, g(l2, g(l3, \ldots))).$$

**Program:** A MapReduce that counts the number of occurrences of each term in a given corpus of text. The input values processed by the map function are documents or other pieces of text. The input keys are ignored. The outcome of the MapReduce is a sequence of (t, $f_t$) tuples, where t is a term, and $f_t$ is the number of times t appeared in the input.

```
map (k, v) ≡
    split v int tokens
    for each token t d
        output(t, 1)
    return
reduce (k, <v1, v2, …, vn>) ≡
    count ← 0
    for i ← 1 to n do
        count ← count + v,
    output(count)
    return
```

- When people refer to the map function in the context of MapReduce, they usually mean the function f that gets passed to map (where map itself is provided by the framework). Similarly, when they refer to the reduce function, they mean the function g that gets passed to reduce. We will follow this convention.
- From a high-level point of view, a MapReduce program (often simply called "a MapReduce") reads a sequence of key/value pairs, performs some computations on them, and outputs another sequence of key/value pairs. Keys and values are often strings, but may in fact be any data type. A MapReduce consists of three distinct phases:
  - In the map phase, key/value pairs are read from the input and the map function is applied to each of them individually. The function is of the general form

    map : (k, v) % → (k1, v1), (k2, v2), ….

    That is, for each key/value pair, map outputs a sequence of key/value pairs. This sequence may or may not be empty, and the output keys may or may not be identical to the input key (they usually aren't).
  - In the shuffle phase, the pairs produced during the map phase are sorted by their key, and all values for the same key are grouped together.
  - In the reduce phase, the reduce function is applied to each key and its values. The function is of the form

    reduce : (k, v1, v2, …) % → (k, v1, v2, …)
- That is, for each key the reduce function processes the list of associated values and outputs another list of values. The output values may or may not be the same as the input values. The output key usually has to be the same as the input key, although this depends on the implementation.

- MapReduces are highly parallelizable, because both map and reduce can be executed in parallel on many different machines. Suppose we have a total of $n = m + r$ machines, where $m$ is the number of map workers and $r$ is the number of reduce workers. The input of the MapReduce is broken into small pieces called map shards. Each shard typically holds between 16 and 64 MB of data. The shards are treated independently, and each shard is assigned to one of the $m$ map workers. In a large MapReduce, it is common to have dozens or hundreds of map shards assigned to each map worker.
- A worker usually works on only 1 shard at a time, so all its shards have to be processed sequentially. However, if a worker has more than 1 CPU, it may improve performance to have it work on multiple shards in parallel. In a similar fashion, the output is broken into separate reduce shards, where the number of reduce shards is often the same as $r$, the number of reduce workers. Each key/value pair generated by the map function is sent to one of the $r$ reduce shards. Typically, the shard that a given key/value pair is sent to depends only on the key. For instance, if we have $r$ reduce shards, the target shard for each pair could be chosen according to

  shard(key, value) = hash(key) mod r,

  where hash is an arbitrary hash function. Assigning the map output to different reduce shards in this manner guarantees that all values for the same key end up in the same reduce shard. Within each reduce shard, incoming key/value pairs are sorted by their key (this is the shuffle phase), and are eventually fed into the reduce function to produce the final output of the MapReduce. Each fragment represents a separate map shard. The key/value pairs emitted by the map workers are partitioned onto the three reduce shards based on the hash of the respective key. For the purpose of the example, we assume hash("heart") mod 3 = 0, hash("soul") mod 3 = 1, and so forth.
- The map phase may overlap with the shuffle phase, and the shuffle phase may overlap with the reduce phase. However, the map phase may never overlap with the reduce phase. The reason for this is that the reduce function can only be called after all values for a given key are available. Since, in general, it is impossible to predict what keys the map workers will emit, the reduce phase cannot commence before the map phase is finished.

## 5.2 WEB RETRIEVAL

### 5.2.1 Search Engine Architectures

- **Effectiveness (Quality of Results):** Effectiveness in the context of a search engine refers to the ability to provide high-quality and relevant search results to users. The primary objective of a search engine is to deliver the most relevant documents based on the user's query. Achieving high effectiveness involves sophisticated ranking algorithms, advanced information retrieval techniques, and powerful indexing mechanisms.
  - **Ranking Algorithms:** Search engines use various ranking algorithms to determine the relevance of documents to a user's query. These algorithms consider factors like keyword frequency, document popularity, user behavior, and context to assign relevance scores to documents.
  - **Information Retrieval Techniques:** Advanced information retrieval techniques, like vector space models, probabilistic models, or language models, are employed to represent and match the query and document content effectively.
  - **Indexing:** The search engine's indexing mechanism should efficiently store and retrieve vast amounts of data to enable quick access to relevant documents.
- **Efficiency (Response Time and Throughput):** Efficiency in a search engine refers to its ability to handle a large number of queries and deliver search results with minimal delay. Users expect fast response times and high throughput, especially for popular search engines that handle enormous volumes of queries.
  - **Indexing and Caching:** Optimized indexing and caching mechanisms can significantly improve response times by reducing the time required to fetch relevant documents from the index.
  - **Distributed Architecture:** Large-scale search engines often adopt a distributed architecture to handle high traffic loads. This involves distributing the workload across multiple servers and data centers, allowing for better load balancing and improved scalability.
  - **Query Optimization:** Efficient query processing techniques, such as query rewriting and optimization, can speed up the search process and reduce the computational overhead.



  - **Hardware and Infrastructure:** The search engine's performance can be enhanced by using powerful hardware, networking, and storage solutions to support the heavy processing and storage requirements.

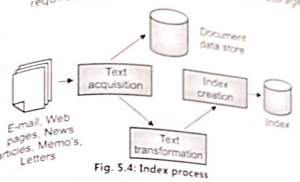



**Fig. 5.4: Index process**

The indexing process is a crucial step in search engine architectures, where documents are collected, transformed, and organized into data structures to enable fast and efficient searching. Let's break down each step of the indexing process:

- **Text Acquisition:** In this step, the search engine acquires documents from various sources, such as websites, databases, or document repositories. Web crawlers, also known as spiders or bots, are typically used to traverse the internet and collect web pages. Other data acquisition mechanisms might involve reading documents from local storage or external APIs.
- **Text Transformation:** Once the documents are acquired, the next step is to transform the raw text into index terms or features. This transformation involves several preprocessing steps to prepare the text for indexing and querying. Common text transformations include:
  - **Tokenization:** Breaking the text into individual words or tokens, removing punctuation, and separating words based on spaces or other delimiters.
  - **Stop Word Removal:** Filtering out common words like "the," "and," "is," etc., which do not add significant meaning to the content.
  - **Stemming/Lemmatization:** Reducing words to their base or root form to group variations of the same word together (e.g., "running" and "runs" become "run").
  - **Lowercasing:** Converting all words to lowercase to ensure case-insensitive searching.
  - **Removing Special Characters:** Eliminating symbols, emojis, and other non-alphanumeric characters.
- **Index Creation:** After the text transformation, the search engine takes the processed documents and creates data structures, commonly known as indexes, to facilitate fast searching. The indexing process involves mapping index terms (words or features) to the corresponding documents. The primary data structure used in indexing is an inverted index.
- **Inverted Index:** An inverted index is a data structure that maps each unique index term to a list of documents or occurrences where that term appears. Each entry in the inverted index typically contains a list of document IDs, term frequencies, and possibly other metadata.

For example, consider the following documents:

**Document 1:** "The quick brown fox jumps over the lazy dog."

**Document 2:** "A fast fox ran through the forest."

After text transformation, the documents may be represented as follows (assuming tokenization, stop word removal, and lowercasing):

**Document 1:** ["quick", "brown", "fox", "jumps", "lazy", "dog"]

**Document 2:** ["fast", "fox", "ran", "forest"]

And the inverted index would look like:

"quick": [Document 1]

"brown": [Document 1]

"fox": [Document 1, Document 2]

"jumps": [Document 1]

"lazy": [Document 1]

"dog": [Document 1]

"fast": [Document 2]

"ran": [Document 2]

"forest": [Document 2]

This inverted index allows the search engine to quickly retrieve documents containing specific index terms, enabling efficient search operations.

**Query Process**

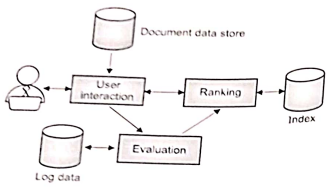



**Fig. 5.5: Query process**

- Query Process, Ranking, and Evaluation. These steps work together to ensure that the search engine effectively delivers relevant search results to users. Let's examine each step in more detail:
- **Query Process:** The Query Process is the interaction between the user and the search engine. It starts when a user enters a search query in the search bar. The search engine then processes the query to understand the user's intent and retrieve relevant results.
  - **Query Creation and Refinement:** The user enters a query consisting of keywords or phrases that describe their information needs. The search engine may offer query suggestions or auto-corrections to help the user refine their query for better results.
  - **Query Parsing:** The search engine parses the user's query, breaking it down into individual terms and applying various text preprocessing techniques (e.g., tokenization, stop word removal, stemming) to prepare the query for retrieval.
  - **Query Expansion:** In some cases, the search engine may perform query expansion to broaden the search scope. This involves adding synonyms or related terms to the original query to capture more relevant documents.
  - **Display of Results:** Once the search engine processes the query, it retrieves relevant documents from its index and presents them to the user as a list of search results. The user can interact with the results, view snippets, click on links, or refine the query further to find the desired information.
- **Ranking:** Ranking is a crucial step in the search engine process. It determines the order in which search results are presented to the user. The goal of ranking is to display the most relevant and authoritative documents at the top of the search results list.
  - **Query-Document Matching:** The search engine matches the query terms with the terms in the indexed documents to identify potential matches.
  - **Ranking Algorithm:** The search engine uses a ranking algorithm to assign a relevance score to each document based on factors like keyword frequency, document popularity, relevance to the query, and other signals. Common ranking algorithms include tf-idf (Term Frequency-Inverse Document Frequency), BM25, and more advanced machine learning-based models like learning-to-rank algorithms.
  - **Result Ordering:** The documents are sorted based on their relevance scores, with the most relevant ones appearing at the top of the search results.
- **Evaluation:** Evaluation is an essential component to ensure the effectiveness and efficiency of the search engine. It involves monitoring and measuring the performance of the search engine to determine how well it meets user needs.
  - **Effectiveness Evaluation:** Effectiveness evaluation focuses on measuring the quality of search results. It may involve human evaluators manually assessing the relevance of search results for specific queries. Metrics like precision, recall, and F1 score are commonly used to evaluate effectiveness.
  - **Efficiency Evaluation:** Efficiency evaluation assesses the search engine's performance in terms of response time and throughput. It measures how quickly the search engine can process queries and deliver search results to users. This evaluation ensures that the search engine can handle a large number of queries efficiently.
  - **Offline Evaluation:** Evaluation is primarily done offline using historical data and benchmark datasets. However, online evaluation, which involves real-time monitoring of user interactions and feedback, is also common for continuous improvement.





**Index Process**

1. **Text Acquisition**

- Text Acquisition is the first step in the search engine indexing process, and it involves acquiring documents from various sources to build the search engine's index. A critical component of the text acquisition process is the crawler (also known as spider or bot), which is responsible for identifying and retrieving documents from different locations.
- Here are the key details about Text Acquisition and the role of the crawler. A crawler is an automated program that systematically navigates the internet or other repositories to discover and collect documents. It acts as a web bot that follows hyperlinks to find and download web pages. There are various types of crawlers, each designed for specific purposes:
  - **Web Crawlers:** These are the most common crawlers and are used to index web pages from the internet. They traverse the web by following links from one page to another, essentially creating a vast map of interconnected documents.
  - **Enterprise Crawlers:** Enterprise crawlers are used for indexing documents within a private network, such as an organization's intranet. They are responsible for collecting documents from internal servers and databases.
  - **Desktop Crawlers:** Desktop crawlers are used in desktop search applications to index local files and documents stored on a user's computer.
- **Coverage and Freshness:** Web crawlers face the challenge of efficiently finding and collecting a vast number of web pages (coverage) and ensuring that the indexed content is up-to-date (freshness). They must continuously explore the web to keep the search engine's index current with the latest information available online.
- **Specialized Crawlers:** Depending on the search engine's requirements, specialized crawlers can be designed for specific purposes:
  - **Single Site Crawlers:** These crawlers focus on indexing documents from a single website or domain. They are commonly used for site-specific search engines.
  - **Topical or Focused Crawlers:** Topical or focused crawlers target specific topics or subjects, concentrating their efforts on relevant websites and documents within a particular vertical or domain.
  - **Document Crawlers:** Document crawlers are utilized in enterprise and desktop search scenarios to index files and documents from various formats, such as Word documents, PDFs, spreadsheets, etc.
- **Link Following and Directory Scanning:** Web crawlers follow hyperlinks from one web page to another, essentially traversing the interconnected web. By following links, they discover new pages and add them to the index. Additionally, they may scan directories to locate and index documents stored in specific web server directories.

**1. Feeds and Conversion**

Two essential aspects of Text Acquisition are Feeds and Conversion, which facilitate the process of collecting and preparing documents for indexing.

- **Feeds:** Feeds refer to real-time streams of documents that are regularly updated and made available by websites or content providers. These feeds are often used for delivering dynamic content, such as news articles, blog posts, videos, radio broadcasts, and TV shows, to users and applications in a standardized format. One common standard for such feeds is RSS (Really Simple Syndication).
  - **Web Feeds:** Websites that frequently publish new content, like news websites or blogs, often provide web feeds in RSS or other formats. These feeds contain summaries or full texts of the latest articles or blog posts.
  - **RSS Reader:** An RSS reader or aggregator is a software or web application that fetches and monitors RSS feeds from different sources. It collects new XML documents from these feeds and presents them to users in a unified interface for easy consumption.
  - **Search Engine Integration:** A search engine can take advantage of RSS readers by subscribing to relevant RSS feeds. The RSS reader then provides new XML documents to the search engine, enabling real-time updates of search results based on the latest content from the subscribed sources.
- **Conversion:** Conversion is the process of transforming documents from various formats, such as HTML, XML, Word, PDF, etc., into a consistent text plus metadata format, which is typically in XML. The conversion process aims to make the content easily accessible and searchable by the search engine.

- **Document Format Conversion:** Since documents can exist in diverse formats, the search engine needs to convert them into a unified format to ensure uniform indexing. For example, web pages may be in HTML format, while some documents could be in proprietary formats like Word or PDF. Conversion to XML standardizes the content for indexing.
- **Text Encoding Conversion:** Different languages use various text encodings, such as UTF-8, UTF-16, ISO-8859, etc. Converting the text encoding to a Unicode standard like UTF-8 ensures that the search engine can process and display content in different languages accurately.

By employing feeds and conversion processes, search engines can efficiently acquire documents from a variety of sources and formats, ensuring that the content is made accessible for indexing and search. Real-time feeds enable the search engine to stay up-to-date with the latest content, and conversion allows the system to present a unified view of the content in a consistent manner.

**2. Document Storage**

Document Storage is a critical component of search engine architectures, responsible for storing text, metadata, and other related content for documents in a format that enables fast access and retrieval. Efficient document storage is essential to support various search engine components, such as result list generation, and handle the massive volume of documents encountered in large-scale search engines.

**Key Aspects of Document Storage**

- **Storing Text and Metadata:** Document storage involves the storage of both the main text content of documents and associated metadata. The main text content typically consists of the textual information that users search for, such as the content of web pages, articles, or documents. Metadata provides additional information about the document, such as its type (e.g., web page, PDF, video), creation date, author, language, and other relevant attributes. Storing both text and metadata in a structured manner is crucial for efficiently managing and retrieving documents during the search process.
- **Storage of Other Related Content:** In addition to text and metadata, document storage may also include related content, such as links and anchor text. For web documents, links refer to hyperlinks present in a document that point to other web pages. Anchor text is the clickable text of a hyperlink that provides context about the linked page. Storing these related content elements allows the search engine to index and process hyperlink information effectively.
- **Fast Access for Search Engine Components:** The document storage system must provide fast and efficient access to document contents and associated information. When a user enters a search query, the search engine needs to retrieve relevant documents quickly to generate search results. Fast access to document contents is crucial for various components of the search engine, including result list generation and ranking algorithms.
- **Choice of Storage System:** For large-scale search engines that deal with a vast number of documents, traditional relational database systems may not be the most efficient choice for document storage. Relational databases can become bottlenecks due to their complex structure and overhead associated with managing relationships between tables. Instead, simpler and more efficient storage systems are often used.
  - **NoSQL Databases:** NoSQL databases, such as document-oriented databases or key-value stores, are popular choices for document storage in large-scale search engines. They offer horizontal scalability, high performance, and flexible schema, making them well-suited for handling massive amounts of unstructured data.
  - **Distributed File Systems:** Distributed file systems, like Hadoop Distributed File System (HDFS), are used to store and manage large volumes of data across multiple nodes, providing fault tolerance and parallel processing capabilities.

**II. Text Transformation**

**(A) Parser**

Text transformation involves processing the sequence of text tokens in a document to identify and recognize structural elements. Structural elements refer to specific components of the document's layout and organization, such as titles, links, headings, and other elements that contribute to the document's overall structure and presentation.

- **Tokenizer:** The first step in text transformation is tokenization, which is performed by a tokenizer. The tokenizer breaks down the document's text into individual tokens or words, which form the basis for further processing. However, tokenization can be complex, as the tokenizer needs to consider various issues, such as:
  - **Capitalization:** Treating words with different capitalization (e.g., "apple" and "Apple") as the same token to ensure case-insensitive indexing and search.
  - **Hyphens and Apostrophes:** Handling hyphenated words and contractions with apostrophes (e.g., "mother-in-law," "don't") correctly to preserve their meanings.
  - **Non-Alphanumeric Characters:** Addressing non-alphanumeric characters (e.g., punctuation marks, special symbols) and determining how they affect tokenization.
  - **Separators:** Identifying and handling word separators, such as spaces or specific delimiters in the text.

**Markup Languages and Document Structure:** Markup languages, such as HTML (HyperText Markup Language) or XML (eXtensible Markup Language), are commonly used to specify the structure of documents on the web. In these languages, tags are used to define document elements and their relationships.

For example, consider the following HTML snippet:

```
<h2> Overview </h2>
```

The <h2> tag indicates that the enclosed content ("Overview") is a second-level heading. Similarly, other tags in HTML or XML can represent different structural elements like paragraphs, lists, links, etc.

**Document Parser:** The document parser is responsible for using the syntax of the markup language (or other formatting) to identify and extract the structural elements from the document. It processes the tokens provided by the tokenizer, recognizes the relevant tags, and interprets their meaning within the context of the document's structure.

By understanding the document's structure, the parser enables the search engine to index and utilize different elements efficiently. For instance, headings might be given higher weight in ranking algorithms to emphasize their significance in representing the document's main topics.



**(B) Stop Words**

Text Transformation in the context of a search engine includes various techniques to preprocess the text and prepare it for indexing. Two essential techniques are Stop Words removal and Stemming, which are used to enhance the effectiveness and efficiency of the search engine's retrieval process.

- **Stop Words Removal:** Stop Words are common words that occur frequently in most documents and often carry little meaningful information about the document's content. Examples of Stop Words include "and," "or," "the," "in," "a," "is," etc. Removing these words during text transformation can lead to a more compact and efficient index.
  - **Impact on Efficiency and Effectiveness:** Removing Stop Words reduces the size of the index and speeds up the search process as there are fewer terms to process. It can significantly improve the efficiency of the search engine, particularly for large-scale systems. However, there might be a trade-off between efficiency and effectiveness, as some Stop Words can carry essential meaning in certain contexts.
  - **Problem for Some Queries:** For specific queries that heavily rely on Stop Words, their removal can negatively impact search results' relevance. For example, queries like "the best restaurants in New York" might suffer from relevant documents being excluded if "the" and "in" are treated as Stop Words.
- **Stemming:** Stemming is a technique used to group words derived from the same root or stem together. By reducing words to their base form, stemming can enhance recall by ensuring that variations of a word are matched to the same root term during search.

  **Example:** For the word "computers," "computing," and "compute," stemming would transform them all to the common stem "comput." This way, a search for "computing" would also return documents containing "computers" and "compute."
  - **Effectiveness and Limitations:** Stemming is usually effective in improving recall and capturing relevant documents containing variations of the same term. However, it is not always accurate, and there might be cases where stemming results in irrelevant matches. For some languages with complex word inflections, stemming might not be as effective.

- **Benefits Vary for Different Languages:** The effectiveness of stemming varies across different languages, as some languages have more regular word inflections, making stemming more reliable. In contrast, others have highly irregular and complex inflections, making accurate stemming more challenging.

**(C) Link Analysis**

Link Analysis is a technique used in search engine algorithms to leverage the information present in hyperlinks (links) and their associated anchor text from web pages. Link analysis plays a significant role in understanding the relationships between web pages and determining the importance or authority of specific pages on the web.

- **Links in Web Pages:** In HTML, hyperlinks are represented using anchor tags <a>. A typical hyperlink looks like this:

```
<a href="https://www.example.com">Information
Sciences Department</a>
```

In this example, "Information Sciences Department" is the anchor text, and the URL "https://www.example.com" is the destination of the hyperlink.

- **Utilizing Link Information:** Search engines use link analysis to gather valuable insights about web pages and their content. Some key aspects of link analysis include:
  - **Page Importance:** The number and quality of inbound links (backlinks) to a web page are often used as a measure of the page's importance or authority. A page with many high-quality backlinks is considered more important or trustworthy.
  - **Link Context:** The anchor text of a hyperlink provides context about the linked page's content. Search engines analyze anchor text to understand the topic or subject matter of the linked page. Pages with relevant and descriptive anchor text tend to rank higher for specific queries related to that anchor text.
  - **Page Ranking:** Link analysis can be used to calculate page rankings using algorithms like PageRank, which was developed by Google. PageRank assigns a numeric value to each web page based on the importance of pages linking to it. This value is used as a factor in determining search result rankings.
  - **Link-Based Connectivity:** Analyzing the structure of hyperlinks across the web helps identify how pages are interconnected. This connectivity aids in discovering new pages and estimating the relevance of a page based on its relationship with other pages.
- **Link Analysis Challenges:** While link analysis can provide valuable insights, it also poses challenges, such as:
  - **Link Spam:** Some website owners engage in link-building strategies to artificially increase their page rankings. This practice, known as link spamming, can negatively impact the quality and accuracy of link-based analysis.
  - **Link Aging:** Over time, the relevance and importance of a page's backlinks might change. Old or outdated links might not accurately reflect a page's current importance.
  - **Link Diversity:** A diverse set of high-quality backlinks is more valuable than a large number of backlinks from a single source. Search engines consider the diversity and quality of backlinks in their analysis.
- Link Analysis is a powerful technique that helps search engines identify popularity and community information on the web. It plays a significant role in web search and ranking algorithms, particularly with methods like PageRank. Anchor text, in particular, enhances the representation of pages pointed to by hyperlinks, thereby influencing the effectiveness of link analysis in web search.
- **Identifying Popularity and Community Information:** Link analysis, particularly through algorithms like PageRank, helps search engines assess the popularity and importance of web pages based on their incoming links. Pages with many high-quality backlinks from other authoritative pages are considered more popular and valuable in the eyes of the search engine.

  Additionally, link analysis can reveal patterns of web page interconnections, which aids in identifying community structures on the web. Pages that are linked to by other pages within the same community often share related topics or content.
- **PageRank Algorithm:** PageRank, developed by Google co-founders Larry Page and Sergey Brin, is a widely known link analysis algorithm. It assigns a numerical value (PageRank score) to each web page based on the importance of pages linking to it. Pages with higher PageRank scores are considered more authoritative and are more likely to rank higher in search results.



- **Impact of Anchor Text:** Anchor text plays a crucial role in enhancing the representation of linked pages. The anchor text provides context about the linked page's content, helping search engines understand the relevance of the link. Pages with descriptive and relevant anchor text are more likely to be ranked higher for queries related to that anchor text.

  For example, if a page is frequently linked to using anchor text like "best smartphones," the search engine will associate that page with the topic of "best smartphones" and rank it higher for queries related to that topic.
- **Significant Impact on Web Search:** Link analysis, particularly with algorithms like PageRank, is a cornerstone of web search engines. It helps search engines determine the importance and relevance of web pages, resulting in more accurate and valuable search results for users.
- **Less Importance in Other Applications:** While link analysis is crucial for web search, it may have less significance in other applications or contexts. In domains outside the web, such as document search within an intranet or specific databases, link analysis might not be as relevant or effective. In such cases, other text transformation and indexing techniques may play a more prominent role.

**(D) Information Extraction**

- Information Extraction is a crucial process that involves identifying specific classes of index terms or metadata from the text to enhance the understanding and representation of documents. Information extraction techniques aim to extract structured and valuable information from unstructured text, making it more useful for various applications. Two key components of information extraction are Named Entity Recognition and Classification.
- **Named Entity Recognition (NER):** Named Entity Recognition is a subtask of information extraction that focuses on identifying and classifying named entities within the text. Named entities are specific words or phrases that represent entities of interest, such as people's names, locations, organizations, dates, and other named entities with real-world significance.

  **Example:** In the sentence "John works at Google in New York and was born on January 1st, 1990," NER would identify "John" as a person, "Google" as an organization, "New York" as a location, and "January 1st, 1990" as a date.

  NER is essential for various applications such as search engines, question-answering systems, information retrieval, and data mining, where extracting named entities is crucial for understanding the context and intent of the text.
- **Classification:** Classification is another critical aspect of information extraction, where documents are assigned labels or categories based on their content. A classifier is a machine learning algorithm that learns from labeled examples to predict the appropriate class label for new, unseen documents.

  **Example:** In a news article classification task, documents may be assigned labels such as "politics," "sports," "technology," etc., based on their content and topic.

  Classifiers can be used to assign class-related metadata to documents, which can include information such as topics, reading levels, sentiment, genre, and more. The specific use of classification depends on the application's requirements and the types of metadata needed for organizing and presenting the documents effectively.

Information extraction, including NER and classification, enhances the structure and semantic understanding of unstructured text, making it more accessible and valuable for various applications. By identifying named entities and assigning labels or categories to documents, information extraction supports tasks like content organization, filtering, recommendation systems, and more. A fundamental part of natural language processing and text analytics empowers a wide range of intelligent systems and applications.

**III. Index Creation Document Statistics**

Index creation is a critical process in search engine architectures, responsible for building data structures that enable efficient and accurate information retrieval. Two important components of index creation are Document Statistics and Weighting.

1. **Document Statistics:** Document statistics involve gathering counts and positions of words and other features in the documents being indexed. These statistics play a significant role in understanding the characteristics of the documents and determining their relevance to search queries. Some of the key document statistics include:
   - **Term Frequency (TF):** Term Frequency is the count of how often a specific term (word or feature) appears in a document. It indicates the importance of a term within the context of a particular document. Documents with higher term frequencies for a given term are considered more relevant to that term.
   - **Positional Information:** Some search engines capture the positions of terms within the documents. This positional information can be utilized in advanced search functionalities, such as phrase search or proximity search, where the order or closeness of terms matters.
   - **Document Length:** The total number of terms in a document is an essential statistic used in some ranking algorithms to normalize the impact of document size on relevancy calculations.
   - **Document ID and Metadata:** Each document is assigned a unique identifier (document ID) during indexing. Document metadata, such as creation date, author, source, etc., may also be stored to provide additional context.
2. **Weighting:** Weighting is a crucial step in index creation, where the relevance of index terms to documents is quantified. It assigns weights to index terms based on their importance in a document or a collection of documents. The weighted index is used in ranking algorithms to determine the order of search results.
   - **TF-IDF Weight:** TF-IDF (Term Frequency-Inverse Document Frequency) is a widely used weighting scheme. It combines both Term Frequency (TF) and Inverse Document Frequency (IDF) to assign weights to terms. The formula for calculating the TF-IDF weight of a term in a document is:

     TF-IDF = TF * IDF
   - **Term Frequency (TF):** As mentioned earlier, this is the count of how often a term appears in a document.
   - **Inverse Document Frequency (IDF):** IDF is a measure of how rare or common a term is across the entire collection of documents. It is calculated as the logarithm of the total number of documents divided by the number of documents containing the term.

The TF-IDF weight reflects the importance of a term both in the document (high TF) and in the collection of documents (low IDF for common terms, high IDF for rare terms). Terms with higher TF-IDF weights are considered more important and are given higher relevance in search results.

**(A) Inversion**

Inversion is a fundamental step in the index creation process for search engines. It is the core process of converting the document-term information into a term-document format, commonly known as an inverted index. This inverted index is the backbone of the search engine's ability to efficiently and quickly retrieve relevant documents for user queries.

***Key Aspects of Inversion***

- **Inverted Index:** An inverted index is a data structure that maps terms (words or features) to the documents in which they appear. It essentially reverses the document-term information, allowing fast retrieval of documents containing specific terms. Each entry in the inverted index is a term, followed by a list of document IDs or pointers to the documents that contain that term.

  **Example:** Term "apple" -> Document IDs {1, 5, 8, 10} Term "banana" -> Document IDs {2, 8, 9, 12}
- **Handling Large Numbers of Documents:** For very large collections of documents, inverting the index can be computationally intensive and resource-consuming. Search engines that index the entire World Wide Web, for example, face immense challenges in handling the vast number of documents available on the web. To address this, search engines use distributed systems and parallel processing to efficiently handle the indexing process.
- **Fast Query Processing:** The format of the inverted index is designed for fast query processing. When a user enters a search query, the search engine can quickly look up the query terms in the inverted index to identify the documents that contain those terms. This allows for fast retrieval of relevant documents, making search results available to users in real-time.



- **Handling Updates:** Search engines constantly update their indexes to include new documents and reflect changes in existing documents. Handling updates efficiently is crucial to maintaining the search engine's relevance and accuracy. Incremental indexing techniques are used to add new documents or update existing ones in the inverted index without rebuilding the entire index from scratch.
- **Compression for Efficiency:** Given the potential size of an inverted index, compression techniques are applied to reduce the index's storage requirements and improve query performance. Various compression algorithms are employed to ensure that the index remains manageable while still offering fast access to information.

Index creation through inversion is a vital process in search engines, enabling fast and efficient retrieval of relevant documents for user queries. Inverted indexes are designed to handle large numbers of documents and updates efficiently. Compression techniques help manage the size of the index while maintaining query performance. The effective handling of the inverted index contributes significantly to the overall performance and responsiveness of the search engine.

**(B) The Problem and Solution**

Index creation for very large document collections, especially on the scale of Google's search index, poses significant challenges. The question of how Google can search through 30 trillion web pages and handle 100 billion search queries per month highlights the complexity of managing such vast amounts of data and delivering fast response times.

**Problem:**

- **Large Document Collections:** With billions of web pages available on the internet, creating and managing an index for such a massive collection is a daunting task. Each web page contains numerous words, and tracking the appearance of every term in every document requires substantial computational resources.
- **Single Index on One Computer:** Creating a single index on a single computer for such an extensive collection of documents is impractical. The computational and storage requirements would be overwhelming, leading to slow response times for search queries.
- **Slow Response:** A single computer processing search queries for such a large index would result in slow response times. Users expect near-instantaneous search results, and any significant delay in retrieving relevant documents would lead to a poor user experience.

**Solution:**

To address these challenges, Google and other large-scale search engines employ parallel and distributed computing techniques. Instead of relying on a single computer and index, they use a distributed system that spans thousands of computers working together.

- **Parallel Computing:** In parallel computing, tasks are divided into smaller sub-tasks that can be executed simultaneously on multiple processors or computers. This approach allows for faster processing by taking advantage of the combined processing power of multiple machines.
- **Distributed Computing:** Distributed computing involves coordinating multiple computers or servers to work as a single system. In the context of search engine indexing, this means distributing the document collection across multiple machines and processing them in parallel.
- **MapReduce:** Google's MapReduce framework is a key technology used to distribute indexing tasks across multiple machines. It enables efficient processing of large datasets by breaking them into smaller "map" tasks, processing them in parallel, and then combining the results ("reduce") to create the index.
- **Sharding:** Another technique used in distributed systems is sharding, where the document collection is divided into smaller subsets called shards, each managed by a separate machine. Each shard is responsible for indexing a specific portion of the document collection.

  By employing parallel and distributed computing techniques, Google can effectively handle the enormous scale of the web and provide fast response times for search queries. Thousands of computers work together to create and maintain the search index, ensuring that users receive search results quickly and efficiently, even for billions of web pages and trillions of documents. This distributed approach is essential for meeting the demands of modern web search and ensuring a seamless user experience.

**(C) Distribution**

Index distribution is a crucial technique in search engine architectures that involves distributing indexes across multiple computers and/or multiple sites. This approach is essential for enabling fast query processing, particularly when dealing with large numbers of documents. Index distribution offers several variations and strategies to efficiently handle the enormous scale of data in modern search engines. Some of the key aspects of index distribution are as follows:

- **Faster Query Processing:** Distributing the index across multiple machines allows for parallel processing of search queries. When a user enters a search query, it can be simultaneously processed by multiple machines, each responsible for a portion of the index. This parallel processing significantly reduces query response times, enabling fast and real-time search results.
- **Document Distribution:** One approach to index distribution is document distribution. In this strategy, the document collection is partitioned and distributed across multiple machines. Each machine is responsible for indexing a specific subset of documents. When a search query is received, it is sent to all relevant machines that hold the documents related to the query's terms.
- **Term Distribution:** Another approach is term distribution, where the index is divided based on individual terms. Each machine is responsible for maintaining the index for specific terms. When a query is received, it is directed to the machines holding the index for the relevant terms in the query.
- **Replication:** To ensure fault tolerance and high availability, index distribution may involve replication. In replication, copies of the index are maintained on multiple machines or sites. If one machine or site becomes unavailable, the search engine can quickly switch to an alternative location, ensuring continuous service and minimizing downtime.
- **Peer-to-Peer (P2P) and Distributed Information Retrieval (IR):** P2P and distributed IR involve search across multiple sites or nodes that collaborate in the search process. P2P systems enable search without the need for centralized servers by leveraging the computing power and resources of participating nodes in the network.
- **Load Balancing:** Index distribution also involves load balancing, where search queries are intelligently distributed among available machines to ensure efficient utilization of resources and prevent overloading of specific machines.
- **Data Consistency:** Ensuring data consistency across distributed indexes is crucial. As the index is updated, changes need to be propagated and synchronized across all machines to maintain the integrity of the search results.

By distributing the index and leveraging parallel processing capabilities, search engines can effectively handle large-scale collections and deliver fast and accurate search results to users. Index distribution is a fundamental part of scaling search engine architectures to handle the vast and dynamic nature of the web.

**IV. User Interaction**

**(A) Query Input**

User Interaction with Query Input is a critical aspect of search engine user interfaces, allowing users to express their information needs using a query language tailored to the search engine's capabilities. The query input process involves providing an interface and parser for users to enter their search queries. The key characteristics of query input in search engines:

- **Interface and Parser:** The search engine provides a user-friendly interface, such as a search box on a web page, to enable users to input their queries. The parser is responsible for processing the user's query in the query language and transforming it into a format that the search engine's indexing and retrieval components can understand.
- **Simple and Complex Queries:** The query input system accommodates a wide range of user queries. Most web queries are relatively simple, consisting of one or more keywords or phrases. However, other applications may use forms or more complex query languages to express intricate search needs.
- **Query Language for Complex Queries:** To support more complex search queries, search engines often use query languages that allow users to express advanced requirements. These query languages enable the combination of multiple criteria and operators to refine search results.
- **Example Query Languages:** Examples of query languages used in Information Retrieval (IR) include:
  - **Boolean Query Language:** Allows users to combine search terms using Boolean operators (AND, OR, NOT) to create compound queries.
  - **Indri Query Language:** Used in the Indri search engine, it supports complex queries, proximity operators, and relevance models to enhance search accuracy.
  - **Galago Query Language:** Used in the Galago search engine, it is designed for efficient querying over large-scale document collections.
- **Similarities to SQL:** IR query languages may resemble SQL (Structured Query Language) used in database applications in terms of their syntax and structure. Both types of query languages support expressing complex search criteria using operators and logical constructs.
- **Content and Structure Specifications:** While IR query languages allow some level of content and structure specifications, they primarily focus on content-based queries. The emphasis is on finding documents that match the user's information needs based on their textual content and relevance to the query terms.



**(B) Query Transformation**

User Interaction: Query Transformation is a vital aspect of the search engine's user interface that aims to enhance the user's initial query and provide more accurate and relevant search results. It involves various text transformation techniques that improve the query both before and after the initial search, making the search process more effective and user-friendly. Some common query transformation techniques include:

- **Spell Checking and Query Suggestion:** When users enter a query, the search engine may perform spell checking to identify and correct spelling mistakes or typos in the query terms. It helps users find relevant results even if they misspell words. Additionally, the search engine can suggest alternative query terms based on commonly searched terms or related terms, helping users refine their query and find more relevant results.
- **Query Expansion:** Query expansion involves adding additional terms or synonyms to the original query to broaden its scope and increase the chances of finding relevant documents. The expansion terms can be based on word similarity, word co-occurrence statistics, or ontologies that capture the semantic relationships between terms.
- **Relevance Feedback:** Relevance feedback is a technique that allows users to provide feedback on the search results they receive. Based on this feedback, the search engine can adjust the query to better match the user's intent. For example, if a user marks certain search results as relevant or irrelevant, the search engine may modify the original query to emphasize or de-emphasize certain terms accordingly.
- **Query Rewriting:** Query rewriting involves transforming the user's original query into an alternative representation that may yield better results. This can be achieved through various text transformation techniques, such as stemming, removing stop words, and converting phrases into their constituent terms.
- **Query Reformulation:** Query reformulation is a user-centric approach to query transformation. It involves interacting with the user by presenting options for modifying the query based on the user's information needs. The user can choose from these options or provide feedback to further refine the query.

**(C) Query Expansion**

Query Expansion is a user interaction technique in search engines that involves adding additional terms to the user's original query to broaden its scope and improve the search results' relevance. There are several methods for query expansion, including finding synonyms of words, discovering various morphological forms of words, and searching for both synonyms and morphological forms. Let's explore these methods in detail:

- **Finding Synonyms of Words:** In this method, the search engine identifies synonyms (words with similar meanings) of the terms present in the user's query. The synonyms are then added to the original query to expand its coverage and increase the likelihood of finding relevant documents.

  **Example:** Original Query: "smartphone" Query Expansion: "smartphone OR mobile OR cellphone OR handset"
- **Finding Various Morphological Forms of Words:** Morphological forms refer to different inflections or word variations that share the same base or root. For example, variations of a word may include plurals, verb tenses, or different grammatical forms. By identifying and including various morphological forms of the terms in the original query, the search engine increases the chances of retrieving relevant documents that may contain these variations.

  **Example:** Original Query: "run" Query Expansion: "run OR runs OR running OR ran"

- **Searching for Synonyms and Morphological Forms:** This method combines both synonym identification and various morphological forms discovery. It involves searching for synonyms of the query terms and also expanding the query to include different morphological forms of those terms.

  **Example:** Original Query: "happy" Query Expansion: "happy OR joyful OR delighted OR happiness OR happily OR happier"

  The goal of query expansion is to capture a broader set of relevant documents that might contain similar or related content, even if they do not exactly match the user's original query terms. By including synonyms and various morphological forms, search engines improve recall (the ability to retrieve relevant documents) and enhance the overall search experience for users.

  It's worth noting that query expansion is not always performed automatically by all search engines. In some cases, users may have the option to enable or disable query expansion, depending on their search preferences. Additionally, query expansion techniques may vary based on the search engine's algorithms and the language or domain being searched.

**(D) Relevance Feedback**

- Relevance Feedback is a crucial aspect of search engine user interfaces that involves gathering feedback from users to improve the relevance of search results. Relevance feedback helps the search engine understand the user's information needs better and refine the search results accordingly. There are different types of relevance feedback based on how the feedback is collected:
- **Explicit Feedback:** Explicit feedback is obtained when a user explicitly indicates the relevance of a document retrieved for their query. This feedback is usually in the form of a user's response, such as a "thumbs up" or "thumbs down" icon next to search results, allowing users to vote on the relevance of individual documents. Users may also have the option to provide more detailed feedback, such as rating the document's relevance on a scale of 1 to 5.
- **Implicit Feedback:** Implicit feedback is inferred from user behavior during the search process. Instead of the user explicitly indicating the relevance of a document, the search engine infers relevance based on the user's actions. Some examples of implicit feedback include:
  - **Selected Documents:** If a user clicks on a search result and spends time on the corresponding web page, it implies that the document is likely relevant to their query.
  - **Duration of Time Spent Viewing a Document:** The amount of time a user spends viewing a document can provide an indication of its relevance. Longer viewing durations may imply higher relevance.
  - **Page Browsing or Scrolling Actions:** Scrolling through a document or navigating to other pages on the website can indicate engagement and relevance.
- **Blind Feedback:** Blind feedback automates the process of relevance feedback without requiring explicit user actions. The search engine analyzes various signals and user behavior data to automatically assess the relevance of search results. Machine learning algorithms and ranking models can use this feedback to fine-tune search results and improve relevance over time.

  The goal of relevance feedback is to enhance the search engine's understanding of the user's intent and preferences. By collecting feedback from users, search engines can learn from user interactions and adapt search results to better match the user's information needs. Relevance feedback is an ongoing process, allowing search engines to continuously improve their ranking algorithms and deliver more accurate and personalized search results to users.

**(E) Output**

Output is a critical component of search engine user interfaces, responsible for presenting the search results in a user-friendly and informative manner. The output component constructs the display of ranked documents for a query, providing users with relevant and engaging search results. The key aspects of search engine output:

- **Display of Ranked Documents:** The output component arranges the search results in a ranked list, showing the most relevant documents at the top. The ranking is based on the search engine's algorithms, which consider factors like relevance, content quality, and user feedback. The ranked list allows users to quickly access the most pertinent information related to their query.
- **Snippets Generation:** For each search result in the ranked list, the output component generates snippets.



Snippets are short excerpts from the document that highlight how the query matches the content. This allows users to preview the relevance of the document without having to click on it, making it easier to determine if the result is useful.

- **Highlighting Important Words and Passages:** The output component may highlight important words and passages in the search results, showing where the query terms appear within the documents. This highlighting helps users quickly identify the relevance of the documents to their information needs.
- **Retrieving Appropriate Advertising:** In many search engine applications, the output component retrieves and displays appropriate advertising alongside the organic search results. The ads are typically related to the user's query or context, allowing advertisers to reach their target audience effectively.
- **Clustering and Visualization Tools:** Some search engines provide additional visualization tools, such as clustering, to help users explore and navigate search results more efficiently. Clustering groups similar search results together, enabling users to identify related documents. Visualization tools can include charts, graphs, and other graphical representations of the search results, making it easier for users to comprehend complex information.
- **Pagination and Navigation:** The output component may provide pagination or infinite scrolling to handle large search result sets. This allows users to navigate through multiple pages of search results and find the information they need without overwhelming the screen with all results at once.
- **Personalization:** In some search engines, the output component may incorporate personalization, tailoring search results to individual users based on their search history, preferences, and past interactions. Personalized search results aim to deliver more relevant and personalized content to each user.

**V. Ranking**

Ranking is a core component of search engines and plays a critical role in determining the order in which search results are presented to users. The ranking process involves calculating scores for documents based on a ranking algorithm, which is the heart of the search engine's retrieval system. Here are the key aspects of ranking in search engines:

- **Scoring:** The ranking process assigns scores to documents based on their relevance to the user's query. Each document receives a score that represents its estimated relevance to the query. The higher the score, the more relevant the document is considered to be.
- **Ranking Algorithm:** The ranking algorithm is a mathematical formula that calculates the scores for documents based on various factors, such as the presence of query terms in the document, the frequency of query terms, and other relevance signals. The algorithm takes into account the importance of each query term and the significance of its occurrence in the document.
- **Query and Document Term Weights:** In the scoring process, each term in the query ($q_i$) and document ($d_i$) is assigned a weight that represents its importance in the calculation of the document's score. These term weights determine how much influence each term has on the overall relevance score of the document.
- **Variations of Ranking Algorithms:** There are numerous variations of ranking algorithms and retrieval models used in search engines. Different algorithms take different factors into account and use various mathematical techniques to calculate document scores. Examples of ranking algorithms include TF-IDF, BM25, language models, and learning-to-rank models.
- **Core Component of Search Engine:** Ranking is at the core of search engine functionality. After the search engine retrieves a set of candidate documents that match the user's query, the ranking process determines the order in which those documents are presented in the search results. This is crucial for providing users with the most relevant and valuable information based on their search queries.
- **Ongoing Research Area in IR:** Ranking is a continuously evolving area of research in Information Retrieval (IR). Researchers are constantly working on developing new and more sophisticated ranking algorithms that improve the accuracy and effectiveness of search engines. The goal is to enhance the retrieval quality and deliver better search experiences for users.

**Ranking Performance Optimization**

- **Designing Efficient Ranking Algorithms:** Efficient ranking algorithms are designed to calculate document scores quickly and accurately. These algorithms consider various factors such as term frequencies,

document lengths, and query-term weights to determine relevance efficiently. Optimized ranking algorithms strike a balance between accuracy and computational complexity.

- **Term-at-a-Time vs. Document-at-a-Time Processing:** In ranking optimization, the choice between term-at-a-time and document-at-a-time processing affects efficiency. Term-at-a-time processing scores documents by considering all query terms at once, whereas document-at-a-time processing scores documents one term at a time. The choice depends on factors like the index structure, query complexity, and hardware resources.
- **Safe vs. Unsafe Optimizations:** In the context of ranking, safe optimizations maintain the integrity of the ranking algorithm and ensure that relevant documents are not excluded due to optimizations. Unsafe optimizations may introduce false positives or false negatives in the results. Search engines prioritize safe optimizations to maintain result quality.
- **Distribution:** Distribution is a key strategy to handle large-scale search queries and document collections efficiently. By distributing the search and ranking tasks across multiple machines or nodes, search engines can parallelize processing and reduce response times for users.
- **Processing Queries in a Distributed Environment:** In a distributed environment, queries are processed across multiple machines or nodes to achieve faster search performance. A query broker is responsible for distributing queries to appropriate nodes and assembling results from the distributed nodes.
- **Caching as Distributed Searching:** Caching is a form of distributed searching where search results are stored temporarily in cache memory. When a query matches a cached result, the search engine can quickly retrieve the pre-computed result from the cache, reducing the need for re-computation and improving response times.
- **Load Balancing:** Load balancing is an essential optimization technique in distributed systems to ensure that resources are evenly utilized across nodes. It prevents overloading of specific nodes, maximizing overall system efficiency.
- **Index Compression:** Optimizing the size of the search index through compression reduces storage requirements and speeds up query processing. Various compression techniques, such as delta encoding or variable byte encoding, are used to achieve efficient index storage.

**Distribution**

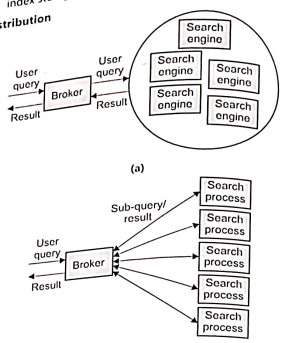



*Fig. 5.6: Distribution*

**VI. Evaluation**

- **Logging:** Logging involves recording user queries and interactions with the search engine. Query logs and clickthrough data are valuable sources of information that provide insights into user behavior, preferences, and information needs. Analyzing these logs helps search engines improve various components, such as query suggestion, spell checking, ranking algorithms, query caching, advertising relevance, and more.
- **Query Log Analysis:** Analyzing query logs allows search engines to understand user search patterns and identify common queries or search trends. This information is invaluable for improving search relevance and user experience. Query log analysis also aids in the development of personalized search features, which tailor search results to individual users based on their historical search behavior.
- **Clickthrough Data Analysis:** Clickthrough data reveals which search results users click on and how they interact with the search results page. This information is used to evaluate the relevance and quality of the search results. By analyzing clickthrough data, search engines can identify which documents users find most relevant and use that feedback to refine the ranking algorithms.



- **Ranking Effectiveness Measurement:** Evaluating the effectiveness of ranking algorithms is critical for delivering high-quality search results. Metrics like precision, recall, F1 score, and Normalized Discounted Cumulative Gain (NDCG) are commonly used to assess ranking effectiveness. Experimenting with different ranking models and tuning parameters based on these metrics helps optimize search relevance.
- **System Efficiency Performance Analysis:** Measuring the efficiency and performance of the search engine system is essential for providing fast response times and a seamless user experience. Performance metrics such as response time, throughput, and resource utilization are monitored to identify bottlenecks and areas for optimization.
- **A/B Testing:** A/B testing (also known as split testing) is a method used to compare different variations of search engine components, such as ranking algorithms or user interface changes. By randomly presenting different versions of the search engine to users and analyzing their interactions, A/B testing helps determine which variations perform better.
- **Relevance Feedback Evaluation:** Relevance feedback evaluation involves assessing the impact of user feedback (explicit and implicit) on search relevance. This feedback is used to refine search results and ensure that the search engine adapts to users' changing needs over time.

### 5.2.2 Cluster Based Architecture

Cluster-based architecture is a popular approach used in web retrieval systems to improve efficiency and scalability. In this architecture, the search engine distributes the workload across multiple machines or nodes, known as clusters, to handle the enormous scale of the web and accommodate a large number of users and queries. Each cluster operates independently, processing a subset of the data and search requests. The key features of a cluster-based web retrieval architecture are as follows:

- **Distributed Indexing and Searching:** The web retrieval system partitions the index and document collection across multiple clusters. Each cluster is responsible for storing and managing a portion of the index and documents. When a user submits a query, it is distributed to relevant clusters based on the query terms.
- **Load Balancing:** Load balancing is essential in cluster-based architectures to ensure that the search requests are evenly distributed across clusters. Load balancers are used to route incoming queries to the appropriate clusters based on their capacity and load.
- **Parallel Processing:** Cluster-based architectures allow for parallel processing of search queries. Each cluster can independently process the queries assigned to it, which significantly improves the overall search speed and response time.
- **Fault Tolerance and Redundancy:** By replicating index data and distributing it across multiple clusters, the architecture provides fault tolerance and redundancy. If one cluster becomes unavailable or experiences issues, the load can be redistributed to other healthy clusters to ensure continuous service.
- **Scalability:** Cluster-based architectures are highly scalable, as additional clusters can be added to the system to accommodate growing data and query volumes. This allows search engines to handle a vast number of web pages and users effectively.
- **Query Routing and Result Merging:** A query router directs incoming queries to the appropriate clusters based on the query terms. After processing the queries, the results from all clusters are merged and ranked to form the final search results presented to the user.
- **Index Distribution:** The index is divided into smaller segments, and each cluster maintains a subset of the index. This index distribution helps reduce the memory and storage requirements for individual clusters.
- **MapReduce Framework:** Some cluster-based web retrieval systems leverage MapReduce or similar distributed computing frameworks to facilitate parallel processing and efficient data analysis.

**Example:** Google's Cluster-Based Web Retrieval Architecture

Google, one of the world's largest search engines, uses a cluster-based architecture to handle the vast number of web pages and user queries it receives every day. Here's how the cluster-based architecture works in Google's web retrieval system:

- **Indexing and Document Distribution:** Google's web crawlers continuously crawl the web, discovering and indexing new web pages. The crawled data is processed and divided into smaller segments known as shards. Each shard contains a subset of the web pages and is stored on a separate server or node.
- **Cluster Formation:** Google's data centers comprise thousands of individual servers, organized into clusters. Each cluster consists of multiple machines working together to handle search requests and store index data.
- **Load Balancing and Query Routing:** When a user submits a search query on the Google search engine, the query is directed to a load balancer. The load balancer's role is to distribute the incoming queries evenly across multiple clusters, ensuring that no single cluster is overwhelmed with requests.
- **Parallel Query Processing:** Once a query is routed to a specific cluster, the cluster independently processes the query in parallel. Each shard in the cluster searches its index for matching documents related to the query terms.
- **Result Merging and Ranking:** After processing the query, each shard generates a set of search results. The results from all the shards within the cluster are merged, and a final ranking of documents is computed based on various relevance factors, including the number of query term occurrences, link analysis, and other relevance signals.
- **Result Presentation:** The ranked search results are presented to the user, usually as a list of web page links on the search results page. Each link is accompanied by a snippet of text from the corresponding web page to provide a preview of the page's content.
- **Fault Tolerance and Redundancy:** Google's cluster-based architecture ensures fault tolerance and redundancy by replicating index data across multiple servers and data centers. If one cluster or server experiences an issue, the load can be redirected to other healthy clusters, ensuring continuous service.

By employing a cluster-based architecture, Google is able to efficiently process billions of web pages and millions of search queries every day. The distributed nature of the architecture allows Google to handle the enormous scale of the web, provide fast response times, and deliver highly relevant search results to its users worldwide.

### 5.2.3 Distributed Architectures

Distributed architectures refer to the design and implementation of systems that distribute processing tasks across multiple nodes or machines to achieve various goals such as scalability, fault tolerance, and improved performance. In distributed architectures, components work collaboratively and communicate with each other over a network to complete complex tasks. Here are some key aspects of distributed architectures:

- **Scalability:** Distributed architectures are highly scalable as they can handle an increasing workload by adding more machines to the system. The workload can be distributed across multiple nodes, allowing the system to grow and accommodate a larger number of users or data volumes.
- **Fault Tolerance and Redundancy:** Distributed architectures provide fault tolerance by replicating data and services across multiple nodes. If one node fails or experiences issues, other nodes can take over the workload, ensuring continuous service and data availability.
- **Load Balancing:** Load balancing is a crucial component of distributed architectures. It involves distributing the workload evenly among the nodes to ensure that no single node is overloaded, preventing performance bottlenecks.
- **Parallel Processing:** In distributed architectures, tasks can be processed in parallel across multiple nodes, leading to improved performance and reduced processing times. This is especially useful for computationally intensive tasks that can be divided into smaller sub-tasks and processed simultaneously.
- **Data Distribution and Replication:** Data is distributed and replicated across multiple nodes in distributed architectures. This data distribution helps optimize data access, as each node can store a subset of the data, reducing the need for centralized storage.
- **Communication and Messaging:** Components in distributed architectures communicate with each other using messaging protocols. Message passing and event-driven communication are common mechanisms used to exchange information and coordinate actions among nodes.
- **Decentralization:** Distributed architectures often exhibit decentralized control, where decision-making and processing are distributed across nodes. This decentralization leads to greater flexibility and resilience.



**Examples of Distributed Architectures:**

- **Distributed File Systems:** Examples include Hadoop Distributed File System (HDFS) and Amazon S3, which distribute data across multiple nodes for storage and retrieval.
- **Distributed Databases:** Examples include Apache Cassandra and MongoDB, which replicate data across nodes to ensure fault tolerance and scalability.
- **Cloud Computing:** Cloud services such as Amazon Web Services (AWS) and Microsoft Azure use distributed architectures to provide scalable and on-demand computing resources.
- **Internet of Things (IoT):** IoT systems often rely on distributed architectures to handle the vast amounts of data generated by sensors and devices.

**Example:** Distributed Web Application Architecture

Imagine a popular e-commerce website that experiences high traffic and has a large database of products and user information. To handle the scale and ensure reliable performance, the e-commerce website is designed using a distributed architecture.

- **Load Balancer:** The distributed architecture includes a load balancer that sits in front of multiple web servers. The load balancer's role is to evenly distribute incoming user requests across the available web servers. This ensures that no single server is overloaded, and the user requests are efficiently distributed.
- **Web Servers:** Multiple web servers form the backend of the application. Each web server can handle incoming user requests independently. These servers are responsible for processing user actions, such as displaying product pages, handling shopping cart interactions, and processing checkout requests.
- **Distributed Database:** The application employs a distributed database system to store product information, user profiles, and other data. The distributed database is partitioned into multiple shards, with each shard hosted on a different database server. This partitioning allows data to be distributed across various servers, reducing the potential for data bottlenecks and ensuring efficient data access.
- **Caching Layer:** A caching layer is introduced to improve application performance and reduce database load. Frequently accessed data, such as product listings or user preferences, is cached in memory on separate cache servers. This helps speed up data retrieval, reducing the need for frequent database access for commonly requested information.
- **Replication for Fault Tolerance:** To ensure fault tolerance and data redundancy, the distributed architecture may employ database replication. Data is replicated across multiple database servers in real-time or with periodic updates. If one database server becomes unavailable, data can be accessed from the replicas, ensuring continuous service.
- **Message Queue and Asynchronous Processing:** The distributed architecture may use a message queue to enable asynchronous processing of tasks. For instance, when a user places an order, instead of immediately processing the order on the main web server, the task is placed in a message queue. A separate worker process (running on another server) picks up the task from the queue and processes it independently. This asynchronous approach helps to handle a large number of tasks efficiently and improves response times.
- **Content Delivery Network (CDN):** To serve static content like images and media files quickly, a Content Delivery Network (CDN) can be incorporated into the architecture. CDNs distribute and cache content across multiple servers located in different geographical regions, ensuring fast delivery of content to users from nearby servers.

### 5.2.4 Search Engine Ranking

Search engine ranking is the process of determining the relevance and order of search results in response to a user's query. When a user enters a search query, the search engine's ranking algorithm evaluates the indexed documents to identify the most relevant ones and presents them to the user in a ranked list. The goal of search engine ranking is to provide users with the most valuable and relevant results for their queries.

**Key Aspects of Search Engine Ranking:**

- **Ranking Algorithm:** The ranking algorithm is the core component of a search engine's ranking process. It uses various factors and signals to calculate the relevance of documents to the user's query. The algorithm considers elements like keyword frequency, document quality, user feedback, link analysis, and other relevance metrics.

- **Relevance Signals:** Search engines use numerous relevance signals to assess the quality and relevance of documents. These signals include term frequency (how often query terms appear in a document), document quality (e.g., the authority and trustworthiness of the website), and user behavior (e.g., click-through rates and dwell time).
- **Query-Document Matching:** The ranking algorithm matches the query terms with the indexed documents to identify potential matches. The more closely a document matches the user's query, the higher its relevance score.
- **Ranking Factors and Weights:** Each relevance signal is assigned a weight in the ranking algorithm to determine its importance in calculating the overall relevance score of a document. The ranking factors and their weights may vary based on the search engine and the specific ranking algorithm being used.
- **Personalization:** Some search engines employ personalization to tailor search results to individual users based on their search history and preferences. Personalization aims to provide more relevant and customized search results for each user.
- **Iterative Ranking:** Search engines often use iterative ranking processes, where initial rankings are refined and improved based on user behavior and feedback. As users interact with search results, the ranking algorithm may adjust the rankings to better match user preferences.
- **Evaluating Ranking Effectiveness:** Search engines continuously evaluate the effectiveness of their ranking algorithms using metrics like precision, recall, click-through rates, and user satisfaction. This evaluation allows search engines to refine and optimize their algorithms over time.
- **Anti-Spam Measures:** Search engines implement anti-spam measures to identify and penalize low-quality or manipulative content. This helps ensure that search results are trustworthy and relevant to users.

### 5.2.5 Link Based Ranking

Link-based ranking, also known as link analysis or link-based relevance ranking, is a method used by search engines to assess the importance and relevance of web pages based on the links that point to them from other pages on the internet. The fundamental idea behind link-based ranking is that if many reputable and relevant web pages link to a particular page, it is likely to be valuable and relevant.

**Key Aspects of Link-Based Ranking:**

- **PageRank Algorithm:** The most famous link-based ranking algorithm is the PageRank algorithm, developed by Larry Page and Sergey Brin, the co-founders of Google. PageRank assigns a numerical value, called PageRank score, to each web page based on the quantity and quality of incoming links it receives from other pages.
- **Quality and Relevance of Links:** In link-based ranking, not all links are treated equally. The algorithm considers the quality and relevance of the linking pages. A link from a highly reputable and relevant website is more valuable than a link from an obscure or unrelated site.
- **Importance of Outgoing Links:** Web pages that link to other reputable and relevant pages can also gain some influence from those links. The outgoing links from a page can impact its own PageRank score, and the quality of those outgoing links is considered in the algorithm.
- **Iterative Calculation:** The PageRank algorithm is an iterative process. Initially, all pages are assigned a default PageRank score. In each iteration, the algorithm recalculates the PageRank scores of pages based on the current scores of their linking pages. This process continues until the PageRank scores converge to stable values.
- **Link Graph:** To implement link-based ranking, search engines construct a link graph that represents the relationships between web pages based on the links. Each page is represented as a node, and the links between pages are represented as edges in the graph.
- **Link-Based Relevance:** Link-based ranking complements traditional content-based ranking methods (such as keyword analysis) by providing a way to assess the authority and importance of pages based on their relationships with other pages.
- **Link Spam and Manipulation:** Link-based ranking is susceptible to manipulation, and some website owners have attempted to artificially inflate their PageRank scores through link schemes and link farms. To counter this, search engines continuously update their algorithms and use anti-spam measures to identify and penalize manipulative link practices.



**Purpose of Link-Based Ranking**

The purpose of link-based ranking is to evaluate the importance, authority, and relevance of web pages based on the links that point to them from other pages on the internet. This approach is used to enhance the ranking of web pages in search engine results and to improve the quality and relevance of search results for users. Link-based ranking is particularly useful for both static (query-independent) and dynamic (query-dependent) ranking scenarios.

- **Static (Query-Independent) Ranking:** In the context of static ranking, link-based ranking considers the structure of the web as a whole, independent of specific user queries. It assigns a score or ranking to each web page based on the quantity and quality of incoming links from other reputable and relevant pages. This enables search engines to determine the overall importance and authority of web pages across the entire web.
- **Dynamic (Query-Dependent) Ranking:** In dynamic ranking, link-based ranking can be combined with other ranking factors and signals, such as content relevance to specific user queries. When a user enters a search query, the search engine uses a combination of link-based ranking and content-based ranking to determine the most relevant and valuable pages to present in the search results.

**Applications of Link-Based Ranking:**

- Search in Social Networks: Link-based ranking is extensively used in social networks to provide search functionality within the network. For example, in social media platforms like Facebook, Twitter, or LinkedIn, link-based ranking helps identify influential users or popular posts based on the number of followers, shares, retweets, and other social interactions.
- Search on the Web: In traditional web search engines like Google and Bing, link-based ranking plays a significant role in evaluating the authority and relevance of web pages. Pages with a high number of incoming links from reputable sources are considered more important and are ranked higher in search results. This helps improve the quality of search results by promoting authoritative and valuable content.

In both social networks and web search, link-based ranking contributes to delivering more accurate and meaningful search results. By analyzing the relationships between pages through links, search engines can identify reputable sources of information and prioritize relevant content in search results. This approach enhances the user experience by presenting high-quality content and reducing the visibility of low-quality or irrelevant pages.

When working with a set of connected objects, you can assign weights to the edges (connections) between the objects to represent different characteristics or strengths of those connections. The weights can be numerical values that indicate the importance, influence, or similarity of the connections. Depending on your specific use case and objectives, you can apply various centrality metrics and classical algorithms to analyze and rank the connected objects. Some common approaches include:

- **Centrality Metrics:** Centrality metrics help identify the most important or influential objects in the network. Some commonly used centrality metrics include:
  - **Degree Centrality:** The number of connections (edges) that each object has. High-degree centrality indicates a highly connected and important object in the network.
  - **Betweenness Centrality:** The extent to which an object lies on the shortest paths between other objects. High betweenness centrality suggests that an object acts as a bridge or mediator in the network.
  - **Eigenvector Centrality:** A measure that considers both the number of connections an object has and the centrality of its neighboring objects. It assigns higher scores to objects connected to other highly central objects.
  - **Closeness Centrality:** A measure of how quickly an object can reach other objects in the network. High closeness centrality indicates that an object is closer to all other objects in the network.
- **Classical Algorithms:** There are classical algorithms specifically designed for ranking objects in a network based on various criteria. Two popular algorithms are:
  - **(i) HITS (Hyperlink-Induced Topic Search) / Hubs and Authorities:** HITS algorithm evaluates web pages based on their roles as hubs (pages that link to many other pages) and authorities (pages that are linked by many hubs). It is used in link analysis and web search.
  - **(ii) PageRank:** PageRank is a link-based ranking algorithm used by search engines like Google. It assigns a numerical value to each page based on the number and quality of links pointing to it. Pages with higher PageRank scores are considered more important and are ranked higher in search results.

**HITS (Hubs and Authorities)**

HITS, also known as Hubs and Authorities, is an algorithm used in link analysis to identify authoritative web pages and hubs in a network of interconnected web pages. It was introduced by Jon Kleinberg in 1998 and has been widely used in web search and information retrieval applications.

The HITS algorithm works by iteratively computing two scores for each web page in a given network:

- **Hub Score:** The hub score represents how well a web page serves as a hub, meaning it links to other authoritative pages. A web page with a high hub score is considered a good hub, as it points to many other relevant and valuable pages.
- **Authority Score:** The authority score reflects how important or authoritative a web page is based on the number of incoming links it receives from other authoritative pages. A web page with a high authority score is considered to be a reliable and significant source of information.

The HITS algorithm starts with an initial set of hub and authority scores for all web pages in the network. Then, it iteratively updates these scores until they converge to stable values. In each iteration, the hub score of each page is recalculated based on the authority scores of the pages it links to, and the authority score of each page is recalculated based on the hub scores of the pages that link to it.

The iterative process continues until the hub and authority scores no longer change significantly. At this point, the algorithm has identified the hubs and authorities in the network.

**Applications of HITS:**

- **Web Search and Ranking:** HITS has been used in web search engines to improve the relevance and ranking of search results. By identifying authoritative pages (authorities) and hubs that link to other relevant pages, HITS helps improve the quality of search results.
- **Information Retrieval:** HITS is utilized in information retrieval tasks to identify the most important documents in a collection based on their connections to other documents and their linking behavior.
- **Social Network Analysis:** HITS can be applied to analyze social networks, identifying influential individuals (authorities) and well-connected members (hubs) within the network.

HITS is a valuable algorithm for understanding the link structure and authority of web pages in a network. However, it's worth noting that HITS is sensitive to link spam and manipulative practices, as it can be influenced by link schemes. To address this, modern search engines use more sophisticated link-based ranking algorithms like PageRank, which builds upon the concepts of HITS and incorporates additional factors for robustness and relevance.

## 5.3 PAGE RANKING ALGORITHM

The PageRank algorithm is a link-based ranking algorithm used by search engines, most notably by Google, to determine the importance and relevance of web pages in their index. It was developed by Larry Page and Sergey Brin, the co-founders of Google, while they were students at Stanford University. PageRank is a critical component of Google's search engine and has played a significant role in revolutionizing web search.

The underlying idea of the PageRank algorithm is that a web page's importance is influenced by the number and quality of incoming links it receives from other pages. In other words, a page's relevance is not only determined by its content but also by its relationships with other pages on the web.

**Key Steps of the PageRank Algorithm:**

- **Graph Representation:** The web pages and their links are represented as a directed graph, where each page is a node, and the links between pages are represented as edges.
- **Initial Score Assignment:** Initially, each page is assigned an equal PageRank score. In the early stages of the algorithm, all pages are considered to have the same level of importance.
- **Iterative Calculation:** The PageRank algorithm then iteratively updates the PageRank scores of each page. In each iteration, the PageRank score of a page is recalculated based on the PageRank scores of the pages that link to it.
- **Weighted Contributions:** The contribution of a linking page's PageRank to a linked page's PageRank is proportional to the linking page's own PageRank and the number of outbound links it has. Pages with higher PageRank scores pass more authority to the pages they link to.



- **Damping Factor:** To prevent the algorithm from getting stuck in loops or dead ends, a damping factor (typically set to around 0.85) is introduced. This factor represents the probability that a user will continue clicking on links instead of navigating to a new page by other means. It ensures the PageRank scores converge to stable values.
- **Convergence:** The iterative process continues until the PageRank scores reach convergence, meaning they no longer change significantly between iterations. At this point, the algorithm has determined the final PageRank scores for all pages in the graph.
- **Ranking:** The pages are ranked based on their final PageRank scores. Pages with higher PageRank scores are considered more important and are ranked higher in search engine results.

**Random Walk**

The web can be represented like a directed graph where nodes represent the web pages and edges form links between them. Typically, if a node (web page) i is linked to a node j, it means that i refers to j.

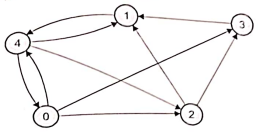



Fig. 5.7

**Example of a Directed Graph**

We have to define what is the importance of a web page. As a first approach, we could say that it is the total number of web pages that refer to it. If we stop to this criteria, the importance of these web pages that refer to it is not taken into account. In other words, an important web page and a less important one has the same weight. Another approach is to assume that a web page spread its importance equally to all web pages it links to. By doing that, we can then define the score of a node j as follows:

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

where $r_i$ is the score of the node i and $d_i$ its out-degree.

From the example above, we can write this linear system:

$$\begin{cases} r_0 = \frac{r_2}{3} \\ r_1 = \frac{r_3}{2} + \frac{r_4}{3} + r_3 \\ r_2 = \frac{r_0}{3} + \frac{r_4}{3} \\ r_3 = \frac{r_0}{2} + \frac{r_2}{3} \\ r_4 = \frac{r_1}{3} + r_1 \end{cases}$$

By passing the right-hand side of this linear system into the left-hand side, we get a new linear system that we can solve by using Gaussian elimination. But this solution is limited for small graphs. Indeed, as this kind of graphs are sparse and Gauss elimination modifies the matrix when performing its operations, we lose the sparsity of the matrix and it would take more memory space. In the worst case, the matrix can no longer be stored.

**Markov Chain and PageRank**

Since a Markov Chain is defined by an initial distribution and a transition matrix, the above graph can be seen as a Markov chain with the following transition matrix:

$$P = \begin{pmatrix} 0 & 0 & 0 & 0 & \frac{1}{3} \\ 0 & 0 & \frac{1}{2} & 1 & \frac{1}{3} \\ \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} \\ \frac{1}{3} & 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{3} & 1 & 0 & 0 & 0 \end{pmatrix}$$

**Transition Matrix Corresponding to our Example**

We notice that P transpose is row stochastic which is a condition to apply Markov chain theorems.

For the initial distribution, let's consider that it is equal to :

$$\pi = \left(\frac{1}{n}, \frac{1}{n}, \ldots, \frac{1}{n}, \frac{1}{n}\right)$$

where n is the total number of nodes. This means that the random walker will choose randomly the initial node from where it can reach all other nodes.

At every step, the random walker will jump to another node according to the transition matrix. the probability distribution is then computed for every step. This distribution tells us where the random walker is likely to be

after a certain number of steps. The probability distribution is computed using the following equation:

$$\pi^{(t+1)} = P\pi^{(t)}$$

A stationary distribution of a Markov chain is a probability distribution $\pi$ with $\pi = P\pi$. This means that the distribution will not change after one step. It is important to note that not all Markov chains admit a stationary distribution.

If a Markov chain is strongly connected, which means that any node can be reached from any other node, then it admits a stationary distribution. It is the case in our problem. So, after an infinitely long walk, we know that the probability distribution will converge to a stationary distribution $\pi$.

All we have to do is solving this equation:

$$\pi = P\pi$$

We notice that $\pi$ is an eigenvector of the matrix P with the eigenvalue 1. Instead of computing all eigenvectors of P and select the one which corresponds to the eigenvalue 1, we use the Frobenius-Perron theorem.

According to Frobenius-Perron theorem, if a matrix A is a square and positive matrix (all its entries are positive), then it has a positive eigenvalue r, such as $|\lambda_i| < r$, where $\lambda$ is an eigenvalue of A. The eigenvector v of A with eigenvalue r is positive and is the unique positive eigenvector.

In our case, the matrix P is positive and square. The stationary distribution $\pi$ is necessarily positive because it is a probability distribution. We conclude that $\pi$ is the dominant eigenvector of P with the dominant eigenvalue 1.

To compute $\pi$, we use the power method iteration which is an iterative method to compute the dominant eigenvector of a given matrix A. From an initial approximation of the dominant eigenvector b that can be initialized randomly, the algorithm will update it until convergence using the following algorithm:

**Algorithm 1:** Power method

```
while (not converge) do
    b = A * b
    norm = compute_norm(b)
    b = b/norm
end while=()
```

**Power Method Algorithm**

As mentioned before, the probability distribution at time t defines the probability that the walker will be in a node after t steps. It means that the higher the probability, the more important is the node. We can then rank our web pages according to the stationary distribution we get using the power method.

**Teleportation and Damping Factor**

In the web graph, for example, we can find a web page i which refers only to web page j and j refers only to i. This is what we call spider trap problem. We can also find a web page which has no outlink. It is commonly named Dead end.

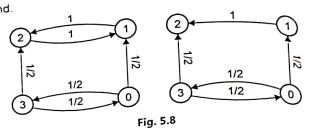



Fig. 5.8

**Dead Ends and Spider Traps Illustration**

In the case of a spider trap, when the random walker reaches the node 1 in the above example, he can only jump to node 2 and from node 2, he can only reach node 1, and so on. The importance of all other nodes will be taken by nodes 1 and 2. In the above example, the probability distribution will converge to $\pi = (0, 0.5, 0.5, 0)$. This is not the desired result.

In the case of Dead ends, when the walker arrives at node 2, it can't reach any other node because it has no outlink. The algorithm cannot converge.

To get over these two problems, we introduce the notion of teleportation.

Teleportation consists of connecting each node of the graph to all other nodes. The graph will be then complete. The idea is with a certain probability $\beta$, the random walker will jump to another node according to the transition matrix P and with a probability $(1-\beta)/n$, it will jump randomly to any node in the graph. We get then the new transition matrix R:

$$R = \beta P + (1-\beta)ve^T$$

where v is a vector of ones, and e a vector of 1/n

$$e^T = \left(\frac{1}{n}, \ldots, \frac{1}{n}\right)$$

$$v = (1, \ldots, 1)^T$$

$\beta$ is commonly defined as the damping factor. In practice, it is advised to set $\beta$ to 0.85.



By applying teleportation in our example, we get the following new transition matrix:

$$R = \begin{pmatrix} \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1}{3}\beta + \frac{1-\beta}{5} \\ \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1}{2}\beta + \frac{1-\beta}{5} & \beta + \frac{1-\beta}{5} & \frac{1}{3}\beta + \frac{1-\beta}{5} \\ \frac{1}{3}\beta + \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1}{3}\beta + \frac{1-\beta}{5} \\ \frac{1}{3}\beta + \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1}{2}\beta + \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} \\ \frac{1}{3}\beta + \frac{1-\beta}{5} & \beta + \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} & \frac{1-\beta}{5} \end{pmatrix}$$

**Our New Transition Matrix**

The matrix R has the same properties than P which means that it admits a stationary distribution, so we can use all the theorems we saw previously.

### 5.3.1 Simple Ranking Functions and Evaluations

Simple ranking functions and evaluations are essential components of information retrieval systems and search engines. They are used to determine the relevance and order of search results based on user queries. Let's explore some common simple ranking functions and evaluation metrics:

**Simple Ranking Functions:**

a. **Term Frequency-Inverse Document Frequency (TF-IDF):**

TF-IDF is a widely used ranking function that calculates the importance of a term in a document relative to a collection of documents. It is based on two factors: Term Frequency (TF) and Inverse Document Frequency (IDF). TF measures the frequency of a term in a document, while IDF measures how rare or common the term is across the entire document collection. The TF-IDF score of a term in a document is the product of its TF and IDF scores.

b. **BM25 (Best Matching 25):**

BM25 is an extension of the TF-IDF ranking function that introduces additional parameters to control the term weighting and document length normalization. It is considered an improvement over the traditional TF-IDF model for better search result ranking.

**Evaluation Metrics:**

a. **Precision:**

Precision measures the proportion of relevant documents among the retrieved documents. It indicates how well the search engine is performing in returning only the relevant results to the user's query. It is computed as the number of relevant documents retrieved divided by the total number of retrieved documents.

b. **Recall:**

Recall measures the proportion of relevant documents retrieved out of all the relevant documents in the collection. It indicates how effectively the search engine retrieves all relevant documents for a given query. It is computed as the number of relevant documents retrieved divided by the total number of relevant documents.

c. **F1 Score:**

The F1 score is the harmonic mean of precision and recall. It provides a single metric that balances both precision and recall. The F1 score is used when there is a need to consider both the completeness (recall) and correctness (precision) of the retrieval.

d. **Mean Average Precision (MAP):**

MAP is commonly used in evaluating the performance of information retrieval systems over multiple queries. It computes the average precision for each query and then takes the mean of these average precision scores. MAP is considered a robust evaluation metric when dealing with large sets of queries and varying relevance judgments.

e. **Normalized Discounted Cumulative Gain (NDCG):**

NDCG is a metric used to evaluate the quality of search result rankings, especially in scenarios where the relevance judgments are graded (e.g., relevance levels 0 to 4). It measures the usefulness of the ranked documents by discounting the relevance of documents that appear lower in the ranking.

## 5.4 WEB CRAWLER

A web crawler, also known as a spider, web robot, or web bot, is an automated program or script that systematically browses the internet and collects information from wet pages. Web crawlers are a fundamental component o search engines, helping to index and gather data from vast number of websites, making it accessible throug search queries. Here's how web crawlers work:

- **Starting Point:** A web crawler begins its journey fro a list of seed URLs, which are typically a small set known and trusted web pages. These seed URLs se as the starting points for the crawler to navigate explore the web.
- **Fetching Web Pages:** The crawler sends HTTP requ to the seed URLs and retrieves the corresponding pages' HTML content. The crawler identifies the

of links present on the pages, extracting them to create a list of new URLs to visit.

- **URL Frontier:** The list of URLs to visit, known as the URL frontier or crawl frontier, acts as a queue or stack of URLs awaiting processing. The crawler prioritizes the URLs in the frontier, considering factors such as the URL's importance, freshness, or relevance.
- **Following Links:** The crawler visits the URLs from the frontier one by one, fetching the corresponding web pages and extracting more links. This process continues recursively, with the crawler following links and adding new URLs to the URL frontier.
- **Politeness and Crawling Etiquette:** Web crawlers need to be respectful of web servers' resources and avoid overwhelming them with too many requests in a short period. Politeness rules, such as obeying robots.txt files (which instruct crawlers on which parts of the website to crawl and which to avoid) and adhering to crawl rate limits, help crawlers maintain good crawling etiquette.
- **Duplicate URL Management:** Web crawlers often encounter duplicate URLs leading to the same content. To avoid redundant crawling and index storage, they employ duplicate URL management techniques to identify and handle duplicate URLs efficiently.
- **Indexing and Storing Data:** As the web crawler fetches web pages, it extracts and stores relevant data from the pages, such as text content, metadata, links, and other information. This data is then indexed and organized to enable efficient search and retrieval by the search engine.
- **Continuous Crawling:** Web crawlers are continuously active, as the web is continually changing with new content being published and old content being updated or removed. Therefore, crawlers revisit previously crawled web pages to detect changes and update their indexes accordingly.

### 5.4.1 Web Crawler Structure

The structure of a web crawler typically consists of several components that work together to efficiently and systematically browse the internet and collect data from web pages. Here's an overview of the main components in the structure of a web crawler:

- **Seed URLs:** The web crawler starts its journey from a list of seed URLs. These seed URLs act as the initial entry points to the web, from which the crawler begins its crawling process.
- **URL Frontier:** The URL frontier is a queue or stack that holds the URLs waiting to be visited and crawled. It is a critical component that manages the list of URLs to be processed by the crawler.
- **Fetcher:** The fetcher is responsible for sending HTTP requests to the URLs in the URL frontier and retrieving the corresponding web pages' HTML content. It acts as the web crawler's interface with web servers to download web pages.
- **URL Filter:** The URL filter checks whether a URL should be added to the URL frontier for crawling. It ensures that only relevant and allowable URLs are included and that the crawler adheres to the crawling etiquette, such as obeying robots.txt directives.
- **Parser:** The parser extracts relevant information from the HTML content of the web pages, such as text, links, metadata, and other data. It processes the retrieved web pages to extract the necessary information for indexing and storage.
- **URL Deduplication:** URL deduplication ensures that the same web page is not crawled multiple times if it appears with different URLs. This component prevents redundancy and optimizes the crawling process.
- **Indexer:** The indexer is responsible for organizing and storing the collected data from web pages in a structured manner. It prepares the data for efficient search and retrieval by building an index that facilitates quick access to relevant information.
- **Scheduler:** The scheduler controls the order in which URLs are fetched and processed by the crawler. It prioritizes URLs based on factors like relevance, freshness, importance, or any specific crawling policies.
- **Politeness Policy:** The politeness policy enforces crawl rate limits to ensure that the web crawler does not overload web servers with too many requests in a short period. It follows the guidelines set by the website owners to maintain good crawling etiquette.
- **Crawl Control and Throttling:** Crawl control and throttling mechanisms manage the rate at which the web crawler accesses and downloads web pages. These mechanisms prevent excessive crawling and ensure the crawler operates within the website's resource limits.

The interaction and coordination of these components in the web crawler structure enable it to efficiently traverse the web, collect data from web pages, and build a comprehensive and up-to-date index of the internet. The well-designed structure ensures that the crawler respects web server resources, avoids duplication, and provides valuable data for search engines and other information retrieval applications.



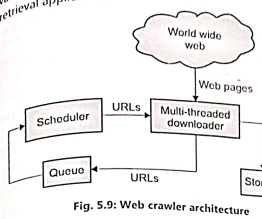



**Fig. 5.9: Web crawler architecture**

- Web Crawler Architecture refers to the design and structure of a program that automatically browses the web for information. The architecture of a web crawler is responsible for defining how the crawler functions, what it does, and how it interacts with the websites it visits.
- At a high level, a web crawler has three main components: the front end, the back end, and the storage system. The front end is the user interface where the user inputs the initial URL and specifies what information they want to extract.
- The back end is responsible for performing the actual web crawling process and consists of multiple modules such as a URL scheduler, a downloader, and a parser.
- The URL scheduler is responsible for keeping track of which URLs have been visited and deciding which URLs should be visited next.
- The downloader retrieves the HTML content of the web pages and passes it to the parser. The parser then extracts the relevant information and stores it in the storage system.
- The storage system is responsible for keeping track of the information that has been extracted by the parser and organizing it in a way that is easily accessible and searchable. This can be a database or a file system.
- A well-designed web crawler architecture is crucial to the success of a web crawling project. It determines how the web crawler operates, what information it can extract, and how that information can be stored and retrieved.

### 5.4.2 Web Crawler Libraries

There are several web crawler libraries available that provide developers with pre-built tools and functionalities to create web crawlers and scrape data from websites. These libraries handle the low-level details of HTTP requests, URL management, and data extraction, allowing developers to focus on building specific crawling logic. Some popular web crawler libraries include:

- **Scrapy:** Scrapy is a powerful and widely-used web scraping and crawling framework written in Python. It allows developers to define custom crawling rules and data extraction methods using Python scripts. Scrapy provides robust support for handling asynchronous requests, handling cookies, and managing the crawling process efficiently. It is highly extensible and suitable for both simple and complex crawling tasks.
- **BeautifulSoup:** BeautifulSoup is a Python library used for parsing HTML and XML documents. While not a full-fledged web crawler library, it is often used in conjunction with other libraries like Requests to extract data from web pages. BeautifulSoup makes it easy to navigate and search the HTML content of web pages, making data extraction straightforward.
- **Apache Nutch:** Apache Nutch is an open-source web crawler and search engine written in Java. It is designed for scalable crawling and indexing of the web. Nutch supports distributed crawling, enabling developers to deploy it on large-scale clusters for handling massive crawling tasks. It is widely used in enterprise-level search and information retrieval systems.
- **Heritrix:** Heritrix is another open-source web crawler written in Java. It is developed by the Internet Archive and designed specifically for archiving web content. Heritrix supports a wide range of customization options and provides sophisticated crawling control, enabling developers to tailor it for various crawling needs.
- **Goutte:** Goutte is a web scraping and crawling library for PHP. It is built on top of the Guzzle HTTP library and provides a straightforward interface for crawling web pages and extracting data. Goutte is useful for developers working with PHP-based web applications.
- **Puppeteer:** Puppeteer is a Node.js library developed by Google. It is not specifically designed for web crawling but is commonly used for automating browser tasks, such as navigating web pages and scraping data

rendered by JavaScript. Puppeteer can be a powerful tool for handling dynamic websites that heavily rely on client-side JavaScript rendering.

These web crawler libraries offer various features and capabilities suited to different crawling tasks and programming languages. Developers can choose the most appropriate library based on their project requirements, programming language preference, and the complexity of the crawling tasks they need to accomplish.

**Python Scrapy**

Scrapy is a fast, open-source web crawling framework written in Python, used to extract the data from the web page with the help of selectors based on XPath.

Scrapy was first released on June 26, 2008 licensed under BSD, with a milestone 1.0 releasing in June 2015.

**Why Use Scrapy?**

- It is easier to build and scale large crawling projects.
- It has a built-in mechanism called Selectors, for extracting the data from websites.
- It handles the requests asynchronously and it is fast.
- It automatically adjusts crawling speed using Auto-throttling mechanism.
- Ensures developer accessibility.

**Features of Scrapy**

- Scrapy is an open source and free to use web crawling framework.
- Scrapy generates feed exports in formats such as JSON, CSV, and XML.
- Scrapy has built-in support for selecting and extracting data from sources either by XPath or CSS expressions.
- Scrapy based on crawler, allows extracting data from the web pages automatically.

**Advantages**

- Scrapy is easily extensible, fast, and powerful.
- It is a cross-platform application framework (Windows, Linux, Mac OS and BSD).
- Scrapy requests are scheduled and processed asynchronously.
- Scrapy comes with built-in service called **Scrapyd** which allows to upload projects and control spiders using JSON web service.
- It is possible to scrap any website, though that website does not have API for raw data access.

**Disadvantages**

- Scrapy is only for Python 2.7. +
- Installation is different for different operating systems.

crapy can be installed by using pip. To install, run the following command –

```
pip install Scrapy
```

**Windows**

**Note:** Python 3 is not supported on Windows OS.

**Step 1:** Install Python 2.7 from Python

Set environmental variables by adding the following paths to the PATH:

```
C:\Python27\;C:\Python27\Scripts\;
```

You can check the Python version using the following command:

```
python --version
```

**Step 2:** Install OpenSSL.

Add C:\OpenSSL-Win32\bin in your environmental variables.

**Note:** OpenSSL comes preinstalled in all operating systems except Windows.

**Step 3:** Install Visual C++ 2008 redistributables.

**Step 4:** Install pywin32.

**Step 5:** Install pip for Python versions older than 2.7.9.

You can check the pip version using the following command:

```
pip --version
```

**Step 6:** To install scrapy, run the following command –

```
pip install Scrapy
```

**Anaconda**

If you have anaconda or miniconda installed on your machine, run the below command to install Scrapy using conda:

```
conda install -c scrapinghub scrapy
```

Scrapinghub company supports official conda packages for Linux, Windows, and OS X.

**Note:** It is recommended to install Scrapy using the above command if you have issues installing via pip.



**Ubuntu 9.10 or Above**

The latest version of Python is pre-installed on Ubuntu OS. Use the Ubuntu packages aptgettable provided by Scrapinghub. To use the packages –

**Step 1:** You need to import the GPG key used to sign Scrapy packages into APT keyring:

```
sudo apt-key adv --keyserver
hkp://keyserver.ubuntu.com:80 --recv 627220E7
```

**Step 2:** Next, use the following command to create /etc/apt/sources.list.d/scrapy.list file:

```
echo 'deb http://archive.scrapy.org/ubuntu scrapy
main' | sudo tee
/etc/apt/sources.list.d/scrapy.list
```

**Step 3:** Update package list and install scrapy:

```
sudo apt-get update && sudo apt-get install scrapy
```

**Archlinux**

You can install Scrapy from AUR Scrapy package using the following command:

```
yaourt -S scrapy
```

**Mac OS X**

Use the following command to install Xcode command line tools:

```
xcode-select --install
```

Instead of using system Python, install a new updated version that doesn't conflict with the rest of your system.

**Step 1:** Install homebrew.

**Step 2:** Set environmental PATH variable to specify that homebrew packages should be used before system packages:

```
echo "export PATH =
/usr/local/bin:/usr/local/sbin:$PATH" >> ~/.bashrc
```

**Step 3:** To make sure the changes are done, reload **.bashrc** using the following command –

```
source ~/.bashrc
```

**Step 4:** Next, install Python using the following command –

```
brew install python
```

**Step 5:** Install Scrapy using the following command –

```
pip install Scrapy
```

The Scrapy command line tool is used for controlling Scrapy, which is often referred to as '**Scrapy tool**'. It includes the commands for various objects with a group of arguments and options.

**Configuration Settings**

Scrapy will find configuration settings in the **scrapy.cfg** file. Following are a few locations:

- C:\scrapy(project folder)\scrapy.cfg in the system
- ~/.config/scrapy.cfg ($XDG_CONFIG_HOME) and ~/.scrapy.cfg ($HOME) for global settings
- You can find the scrapy.cfg inside the root of the project.

Scrapy can also be configured using the following environment variables:

- SCRAPY_SETTINGS_MODULE
- SCRAPY_PROJECT
- SCRAPY_PYTHON_SHELL

**Default Structure Scrapy Project**

The following structure shows the default file structure of the Scrapy project:

```
scrapy.cfg          - Deploy the configuration file
project_name/       - Name of the project
  _init_.py
  items.py          - It is project's items file
  pipelines.py      - It is project's pipelines file
  settings.py       - It is project's settings file
  spiders           - It is the spiders directory
    _init_.py
    spider_name.py
    . . .
```

The scrapy.cfg file is a project root directory, which includes the project name with the project settings. For instance:

```
[settings]
default = [name of the project].settings

[deploy]
#url = http://localhost:6800/
project = [name of the project]
```

**Using Scrapy Tool**

Scrapy tool provides some usage and available commands as follows:

```
Scrapy X.Y  - no active project
Usage:
```

```
scrapy [options] [arguments]
```

Available commands:

```
crawl    It puts spider (handle the URL) to work for crawling data
fetch    It fetches the response from the given URL
```

**Creating a Project**

You can use the following command to create the project in Scrapy:

```
scrapy startproject project_name
```

This will create the project called project_name directory. Next, go to the newly created project, using the following command:

```
cd project_name
```

**Controlling Projects**

You can control the project and manage them using the Scrapy tool and also create the new spider, using the following command:

```
scrapy genspider mydomain mydomain.com
```

The commands such as crawl, etc. must be used inside the Scrapy project. You will come to know which commands must run inside the Scrapy project in the coming section.

Scrapy contains some built-in commands, which can be used for your project. To see the list of available commands, use the following command:

```
scrapy -h
```

When you run the following command, Scrapy will display the list of available commands as listed:

- **fetch:** It fetches the URL using Scrapy downloader.
- **runspider:** It is used to run self-contained spider without creating a project.
- **settings:** It specifies the project setting value.
- **shell:** It is an interactive scraping module for the given URL.
- **startproject:** It creates a new Scrapy project.
- **version:** It displays the Scrapy version.
- **view:** It fetches the URL using Scrapy downloader and show the contents in a browser.

You can have some project related commands as listed:

- **crawl:** It is used to crawl data using the spider.
- **check:** It checks the items returned by the crawled command.
- **list:** It displays the list of available spiders present in the project.
- **edit:** You can edit the spiders by using the editor.
- **parse:** It parses the given URL with the spider.
- **bench:** It is used to run quick benchmark test (Benchmark tells how many number of pages can be crawled per minute by Scrapy).

**Custom Project Commands**

You can build a custom project command with **COMMANDS_MODULE** setting in Scrapy project. It includes a default empty string in the setting. You can add the following custom command:

```
COMMANDS_MODULE = 'mycmd.commands'
```

Scrapy commands can be added using the scrapy.commands section in the setup.py file shown as follows:

```
from setuptools import setup, find_packages


setup(name = 'scrapy-module_demo',
  entry_points = {
    'scrapy.commands': [
      'cmd_demo = my_module.commands:CmdDemo',
    ],
  },
)
```

**Description**

Spider is a class responsible for defining how to follow the links through a website and extract the information from the pages.

The default spiders of Scrapy are as follows:

**scrapy.Spider**

It is a spider from which every other spiders must inherit. It has the following class:

```
class scrapy.spiders.Spider
```

Table 5.1 shows the fields of scrapy.Spider class:

**Table 5.1**

| Sr. No. | Field and Description |
|---|---|
| 1. | **name**<br>It is the name of your spider. |
| 2. | **allowed_domains**<br>It is a list of domains on which the spider crawls. |



| Sr. No. | Field and Description |
|---|---|
| 3. | **start_urls**<br>It is a list of URLs, which will be the roots for later crawls, where the spider will begin to crawl from. |
| 4. | **custom_settings**<br>These are the settings, when running the spider, will be overridden from project wide configuration. |
| 5. | **crawler**<br>It is an attribute that links to Crawler object to which the spider instance is bound. |
| 6. | **settings**<br>These are the settings for running a spider. |
| 7. | **logger**<br>It is a Python logger used to send log messages. |
| 8. | **from_crawler(crawler,*args,**kwargs)**<br>It is a class method, which creates your spider. The parameters are:<br>• **crawler** – A crawler to which the spider instance will be bound.<br>• **args(list)** – These arguments are passed to the method _init_().<br>• **kwargs(dict)** – These keyword arguments are passed to the method _init_(). |
| 9. | **start_requests()**<br>When no particular URLs are specified and the spider is opened for scrapping, Scrapy calls start_requests() method. |
| 10. | **make_requests_from_url(url)**<br>It is a method used to convert urls to requests. |

| Sr. No. | Field and Description |
|---|---|
| 11. | **parse(response)**<br>This method processes the response and returns scrapped data following more URLs. |
| 12. | **log(message[,level,component])**<br>It is a method that sends a log message through spiders logger. |
| 13. | **closed(reason)**<br>This method is called when the spider closes. |

### 5.4.3 Beautiful Soup

**Installing BeautifulSoup**

As BeautifulSoup is not a standard library, we need to install it. We are going to use the BeautifulSoup 4 package (known as bs4).

**1. Linux Machine**

To install bs4 on Debian or Ubuntu linux using system package manager, run the below command –

```
$sudo apt-get install python-bs4 (for python 2.x)
$sudo apt-get install python3-bs4 (for python 3.x)
```

You can install bs4 using easy_install or pip (in case you find problem in installing using system packages):

```
$easy_install beautifulsoup4
$pip install beautifulsoup4
```

(You may need to use easy_install3 or pip3 respectively if you're using python3)

**2. Windows Machine**

To install beautifulsoup4 in windows is very simple, especially if you have pip already installed.

```
>pip install beautifulsoup4
```

```
(myEnv) C:\Users\yadur>pip install beautifulsoup4
Collecting beautifulsoup4
  Downloading https://files.pythonhosted.org/packages/3b/c8/a55eb6ea11cd7e5ac4bacd7f6c5ac4d03b99bcb7a7e5b2bf1831ef7f/beautifulsoup4-4.8.1-py3-none-any.whl (143kB)
    100% |################################| 143kB 159kB/s
Collecting soupsieve>=1.2
  Downloading https://files.pythonhosted.org/packages/81/94/03c0f04471fc245d08da99f7da6ac29cd9f0ac44c71a9286f2288139/soupsieve-1.9.5-py2.py3-none-any.whl
Installing collected packages: soupsieve, beautifulsoup4
Successfully installed beautifulsoup4-4.8.1 soupsieve-1.9.5

(myEnv) C:\Users\yadur>pip list
Package        Version
-------------- -------
beautifulsoup4 4.8.1
bs4            19.3.1
setuptools     41.6.0
soupsieve      1.9.5
wheel          0.33.6
```

So now beautifulsoup4 is installed in our machine. Let us talk about some problems encountered after installation.

**Problems after Installation**

On windows machine you might encounter, wrong version being installed error mainly through;

- **error:** ImportError "No module named HTMLParser", then you must be running python 2 version of the code under Python 3.
- **error:** ImportError "No module named html.parser" error. then you must be running Python 3 version of the code under Python 2.

Best way to get out of above two situations is to re-install the BeautifulSoup again, completely removing existing installation.

If you get the SyntaxError "Invalid syntax" on the line ROOT_TAG_NAME = u'[document]', then you need to convert the python 2 code to python 3, just by either installing the package:

```
$ python3 setup.py install
```

or by manually running python's 2 to 3 conversion script on the bs4 directory:

```
$ 2to3-3.2 -w bs4
```

**Installing a Parser**

By default, Beautiful Soup supports the HTML parser included in Python's standard library, however it also supports many external third party python parsers like lxml parser or html5lib parser.

To install lxml or html5lib parser, use the command –

**Linux Machine**

```
$apt-get install python-lxml
$apt-get insall python-html5lib
```

**Windows Machine**

```
$pip install lxml
$pip install html5lib
```

```
My(Env) C:\Users\yadur>pip install lxml
Collecting lxml
  Downloading https://files.pythonhosted.org/packages/bc/87/c3ec4e5e6d5f81dcd7172401733413d9cfc3620a8906197a9914e8583ed6c/lxml-4.4.1-cp37-cp37m-win_amd64.whl (3.7MB)
     |████████████████████████████████| 3.7MB 50kB/s
Installing collected packages: lxml
Successfully installed lxml-4.4.1

My(Env) C:\Users\yadur>pip install html5lib
Collecting html5lib
  Downloading https://files.pythonhosted.org/packages/a5/62/bbd2be0e7943ec8504b517e62bcf693ca2033264c1763e0bfe142ca0b912b/html5lib-1.0.1-py2.py3-none-any.whl (117kB)
     |████████████████████████████████| 122kB 75kB/s
Collecting webencodings
  Downloading https://files.pythonhosted.org/packages/f4/24/2a3e3df732393fed8b3ebf2ec078f05546de641fe1b667ee316ec1dcf3b7/webencodings-0.5.1-py2.py3-none-any.whl
Collecting six>1.9
  Downloading https://files.pythonhosted.org/packages/65/26/32b8464df2a97e6dd1b656ed26b2c194606c16fe163c695a992b36c114ef/six-1.13.0-py2.py3-none-any.whl
Installing collected packages: webencodings, six, html5lib
Successfully installed html5lib-1.0.1 six-1.13.0 webencodings-0.5.1
```

Generally, users use lxml for speed and it is recommended to use lxml or html5lib parser if you are using older version of python 2 (before 2.7.3 version) or python 3 (before 3.2.2) as python's built-in HTML parser is not very good in handling older version.

**Running Beautiful Soup**

It is time to test our Beautiful Soup package in one of the html pages (taking web page: https://www.tutorialspoint.com/index.htm, you can choose any-other web page you want) and extract some information from it.

In the below code, we are trying to extract the title from the webpage:

```
from bs4 import BeautifulSoup
import requests
url = "https://www.tutorialspoint.com/index.htm"
req = requests.get(url)
soup = BeautifulSoup(req.text, "html.parser")
print(soup.title)
```

**Output**

<title>H2O, Colab, Theano, Flutter, KNime, Mean.js, Weka, Solidity, Org.Json, AWS QuickSight, JSON.Simple, Jackson Annotations, Passay, Boon, MuleSoft, Nagios, Matplotlib, Java NIO, PyTorch, SLF4J, Parallax Scrolling, Java Cryptography</title>

One common task is to extract all the URLs within a webpage. For that we just need to add the below line of code:

```
for link in soup.find_all('a'):
print(link.get('href'))
```

**Output**

https://www.tutorialspoint.com/index.htm
https://www.tutorialspoint.com/about/about_careers.htm
https://www.tutorialspoint.com/questions/index.php
https://www.tutorialspoint.com/online_dev_tools.htm
https://www.tutorialspoint.com/codingground.htm
https://www.tutorialspoint.com/current_affairs.htm
https://www.tutorialspoint.com/upsc_ias_exams.htm
https://www.tutorialspoint.com/tutor_connect/index.php
https://www.tutorialspoint.com/whiteboard.htm
https://www.tutorialspoint.com/netmeeting.php
https://www.tutorialspoint.com/index.htm
https://www.tutorialspoint.com/tutorialslibrary.htm
https://www.tutorialspoint.com/videotutorials/index.php
https://store.tutorialspoint.com
https://www.tutorialspoint.com/gate_exams_tutorials.htm
https://www.tutorialspoint.com/html_online_training/index.asp
https://www.tutorialspoint.com/css_online_training/index.asp
https://www.tutorialspoint.com/3d_animation_online_training/index.asp
https://www.tutorialspoint.com/swift_4_online_training/index.asp
https://www.tutorialspoint.com/blockchain_online_training/index.asp
https://www.tutorialspoint.com/reactjs_online_training/index.asp
https://www.tutorix.com
https://www.tutorialspoint.com/videotutorials/top-courses.php
https://www.tutorialspoint.com/the_full_stack_web_development/index.asp
....
....
https://www.tutorialspoint.com/online_dev_tools.htm
https://www.tutorialspoint.com/free_web_graphics.htm
https://www.tutorialspoint.com/online_file_conversion.htm
https://www.tutorialspoint.com/netmeeting.php
https://www.tutorialspoint.com/free_online_whiteboard.htm
https://www.tutorialspoint.com
https://www.facebook.com/tutorialspointindia
https://plus.google.com/u/0/+tutorialspoint
http://www.twitter.com/tutorialspoint
http://www.linkedin.com/company/tutorialspoint
https://www.youtube.com/channel/UCVLbzhxVTiTLiVKeGV7WEBg
https://www.tutorialspoint.com/index.htm
/about/about_privacy.htm#cookies
/about/faq.htm
/about/about_helping.htm
/about/contact_us.htm

Similarly, we can extract useful information using beautifulsoup4

Now let us understand more about "soup" in above example.

**Beautiful Soup - Souping the Page**

In the previous code example, we parse the document through beautiful constructor using a string method. Another way is to pass the document through open filehandle.

```
from bs4 import BeautifulSoup
with open("example.html") as fp:
   soup = BeautifulSoup(fp)
soup = BeautifulSoup("<html>data</html>")
```

First the document is converted to Unicode, and HTML entities are converted to Unicode characters.</p>

```
import bs4
html = '''<b>tutorialspoint</b>, <i>&web scraping &data science;</i>'''
soup = bs4.BeautifulSoup(html, 'lxml')
print(soup)
```

**Output**

<html> <body> tutorialspoint, &web scraping &data science; </body> </html>

BeautifulSoup then parses the data using HTML parser or you explicitly tell it to parse using an XML parser.

**HTML Tree Structure**

Before we look into different components of a HTML page, let us first understand the HTML tree structure.

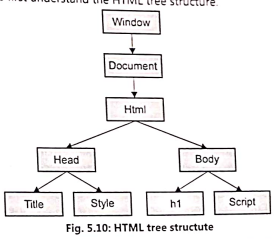



**Fig. 5.10: HTML tree structute**

The root element in the document tree is the html, which can have parents, children and siblings and this determines by its position in the tree structure. To move among HTML elements, attributes and text, you have to move among nodes in your tree structure.

Let us suppose the webpage is as shown below:

TutorialsPoint

**Tutorialspoint Online Library**

**It's all Free**

Which translates to an html document as follows:

```
<html><head><title>TutorialsPoint</title></head><h1>Tutorialspoint Online Library</h1><p><b>It's all Free</b></p></body></html>
```

Which simply means, for above html document, we have a html tree structure as follows:

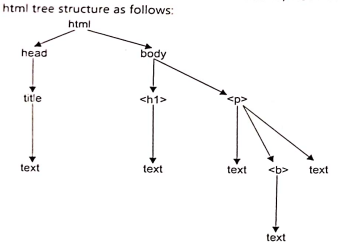



**Fig. 5.11**

### 5.4.4 Applications of Web Crawlers

Web crawlers have a wide range of applications across various domains and industries. They play a crucial role in collecting, indexing, and organizing data from the internet, making it accessible for different purposes. Here are some common applications of web crawlers:

- **Search Engines:** Web crawlers are the backbone of search engines like Google, Bing, and Yahoo. They continuously crawl the web, collecting information from web pages, and build comprehensive indexes of web content. These indexes enable search engines to provide relevant search results to users when they enter queries.
- **Content Aggregation:** Web crawlers are used by content aggregation platforms to gather and consolidate information from multiple sources. These platforms curate and present content from various websites, blogs, news sources, and social media in one place, allowing users to access a diverse range of content without navigating to different websites.
- **Price Comparison and Product Information:** E-commerce websites and price comparison platforms employ web crawlers to extract product information, prices, and availability from various online retailers. This data is then used to provide consumers with comparisons, enabling them to find the best deals and make informed purchase decisions.
- **Web Monitoring and Competitive Intelligence:** Businesses use web crawlers to monitor their online presence, track competitors' activities, and analyze market trends. Web monitoring tools powered by web crawlers can track mentions, customer reviews, social media posts, and other online content related to a company or brand.
- **Academic Research:** Web crawlers are employed in academic research to collect data from scholarly articles, research papers, and other academic resources. Researchers use these data sets to analyze trends, conduct sentiment analysis, and gather insights in various fields of study.
- **Social Media Analysis:** Web crawlers are used to gather data from social media platforms such as Twitter, Facebook, and Instagram. This data is used for sentiment analysis, social network analysis, brand monitoring, and understanding public opinion on various topics.
- **News Aggregation:** News aggregators use web crawlers to gather news articles and updates from different news websites and sources. This allows users to access the latest news and updates from multiple publishers in one place.
- **Data Mining and Machine Learning:** Web crawlers are used to collect data sets for data mining and machine learning tasks. These data sets can be used for training machine learning models, sentiment analysis, and natural language processing applications.
- **Business Listings and Directories:** Web crawlers are utilized to collect business information such as contact details, addresses, and business descriptions from various directories and websites. This information is then used to build business directories and listings for consumers.



- **Academic Search Engines:** Web crawlers power academic search engines like Google Scholar, which indexes academic publications and research papers, making them easily discoverable to scholars and researchers.

## CASE STUDY: STUDY OF GOOGLE MAP/ GEOGUSSER INFORMATION RETRIEVAL

***Google Maps and Geoguessr***

- **Background:** Google Maps is a widely used web mapping service that provides users with interactive maps, satellite imagery, street view, and route planning. Geoguessr, on the other hand, is an online geography game that utilizes Google Maps Street View. Players are placed in a random location around the world and must guess their location based on visual cues from the Street View imagery.
- **Information Retrieval in Google Maps:** Google Maps employs sophisticated information retrieval techniques to provide users with accurate and relevant geographical information. Here's how it works:
  - **Search Queries:** Users enter search queries like addresses, landmarks, or types of businesses (e.g., "Eiffel Tower" or "Italian restaurants near me").
  - **Textual Information Retrieval:** Google Maps matches the user's query with its database of locations and landmarks. It considers factors like the accuracy of the match, popularity of the location, and relevance to the user's location.
  - **Geographical Data:** The system retrieves geographical data including coordinates, addresses, contact information, reviews, photos, and more.
  - **Visual Information (Street View):** For selected locations, Google Maps provides Street View imagery, allowing users to virtually explore the area. This visual information enhances the user's understanding of the location.
- **Information Retrieval in Geoguessr:** Geoguessr leverages the Street View imagery from Google Maps to create an engaging and challenging game. Here's how information retrieval is involved:
  - **Random Location Generation:** Geoguessr selects a random location from around the world using Google Maps Street View.
  - **Visual Clues:** Players rely on visual cues from the Street View imagery to guess their location. They observe features such as architecture, vegetation, signage, language, and more.
  - **Player Input:** Players make a guess by placing a marker on a world map, indicating where they believe the location is.
  - **Scoring and Feedback:** Geoguessr evaluates the accuracy of the guess by measuring the distance between the actual location and the player's guess. Players receive scores based on their accuracy.
- **Comparative Study:** A comparative study in this context could involve evaluating the performance of different algorithms or techniques for randomly selecting locations, generating clues, or assessing player guesses. It could also involve analyzing user engagement, accuracy, and overall enjoyment of the game under various setups.
- **Insights and Applications:** By studying the mechanics of Google Maps and Geoguessr, researchers can gain insights into effective strategies for information retrieval and engagement in location-based applications. Developers can use these insights to improve search functionalities, create more immersive location-based experiences, and enhance user interactions with geographical data.

This case study highlights how information retrieval techniques are applied in a fun and educational context, demonstrating their relevance in both practical and entertainment settings.

## EXERCISE

1. What is the purpose of a web crawler in the context of search engines?
2. Name two important requirements that determine the architecture of a search engine.
3. Explain the components involved in the text acquisition process of a search engine.
4. How does a web crawler identify and acquire documents for indexing?
5. What are the roles of the parser and tokenizer in the text transformation process?
6. Define "stop words" and explain their impact on search engine efficiency and effectiveness.
7. How does link analysis contribute to the ranking process in web search engines?

8. Describe the purpose of the PageRank algorithm and its application in search engine ranking.
9. What are some popular web crawler libraries used for web scraping and crawling tasks?
10. How does Scrapy differ from BeautifulSoup in terms of their functionalities?
11. Explain the relevance of URL frontier and URL deduplication in the web crawler's structure.
12. Discuss the importance of politeness policies in web crawling and how they are enforced.
13. What are the key evaluation metrics used to measure the performance of information retrieval systems?
14. How does the F1 score differ from precision and recall in search engine evaluation?
15. Describe the main components of a web crawler's structure and their respective functions.
16. In which industries or domains are web crawlers commonly used, and what applications do they serve?
17. How does a web crawler contribute to the functioning of content aggregation platforms?
18. What are the benefits of using web crawlers for web monitoring and competitive intelligence?
19. How can web crawlers be utilized in social media analysis and sentiment analysis?
20. Explain the role of web crawlers in academic research, particularly in data collection and analysis.

✠ ✠ ✠

UNIT - VI

# IR APPLICATIONS

## 6.1 MULTIMEDIA INFORMATION RETRIEVAL

- Multimedia Information Retrieval (MIR) is a field that deals with the retrieval, organization, and analysis of multimedia data. It encompasses a wide range of techniques and technologies that enable users to search, access, and analyze multimedia content efficiently.
- Multimedia data refers to any information that involves multiple media types, such as text, images, audio, video, and other forms of data. With the explosive growth of digital content on the internet and various multimedia platforms, effective retrieval of multimedia data has become essential for various applications, including search engines, content recommendation systems, digital libraries, and more.

**Key Components of Multimedia Information Retrieval:**

- **Data Representation:** Representing multimedia data in a form suitable for retrieval is a crucial step. For instance, images and videos can be represented as feature vectors or histograms, while audio can be represented as spectrograms or Mel-Frequency Cepstral Coefficients (MFCCs).
- **Indexing and Retrieval:** To enable efficient searching, multimedia databases need to be indexed. Indexing involves creating structures that facilitate the retrieval of relevant data quickly. Various indexing techniques like inverted files, tree-based indexing, and hashing are used to speed up the search process.
- **Content-Based Retrieval:** In content-based retrieval, the system finds multimedia content similar to a query based on its visual, auditory, or textual features rather than relying on metadata or annotations. This is commonly used for searching images and videos based on their visual similarity.
- **Text-Based Retrieval:** While content-based retrieval is useful for visual and auditory media, text-based retrieval involves using keywords and text-based descriptions to retrieve multimedia content that matches the query.
- **Multimedia Analysis:** Techniques like object recognition, speech recognition, emotion analysis, and scene detection are used to extract meaningful information from multimedia data, making it easier to index and retrieve.
- **Relevance Feedback:** In some cases, users' feedback is collected to refine search results. For example, if a user searches for images of "beaches" and the results include some irrelevant images, the user can provide feedback to improve subsequent searches.
- **Evaluation Metrics:** Evaluating the performance of multimedia information retrieval systems is essential to assess their effectiveness. Metrics like precision, recall, and F-measure are commonly used for evaluation.

**Challenges in Multimedia Information Retrieval:**

- **Large-Scale Data:** With the vast amount of multimedia data available online, managing and efficiently searching through large-scale databases can be challenging.
- **Heterogeneous Data:** Multimedia data comes in various formats, making it complex to handle different media types in a unified retrieval system.
- **Feature Extraction:** Extracting meaningful features from multimedia data that accurately represent its content is a complex task.
- **Semantic Gap:** There is often a disconnect between low-level features extracted from multimedia data and high-level semantic understanding, making relevance determination challenging.
- **User Intent Understanding:** Understanding the user's intent while formulating a query and delivering relevant results remains a challenging problem.

Multimedia Information Retrieval is an active and evolving field, with ongoing research and advancements aiming to address these challenges and improve the efficiency and effectiveness of multimedia search and analysis systems.

### 6.1.1 Spoken Language Audio Retrieval

Spoken Language Audio Retrieval, also known as Spoken Term Detection (STD) or Spoken Term Recognition (STR), is a specific task in the field of multimedia information retrieval that focuses on finding occurrences of specific spoken words or phrases within audio recordings. It involves searching through a large collection of spoken language audio data to identify and retrieve segments that contain the target spoken term or keyword.

**The Task of Spoken Language Audio Retrieval is Challenging Due to Several Factors:**

- **Speech Recognition:** First, the system needs to convert the audio data into text through Automatic Speech Recognition (ASR) techniques. ASR aims to transcribe spoken language into its corresponding textual representation. The quality and accuracy of ASR play a crucial role in the overall effectiveness of Spoken Language Audio Retrieval.
- **Keyword Spotting:** Once the audio is transcribed, the system performs keyword spotting to identify the occurrences of the target spoken word or phrase. Keyword spotting focuses on locating the specific terms in the audio and extracting the relevant segments containing those keywords.
- **Large-scale Data:** Spoken language audio databases can be extensive, and efficient search algorithms are required to handle the large volume of data effectively.
- **Noisy and Real-world Data:** Spoken language audio recordings often come with various challenges like background noise, accents, speech variations, and environmental conditions, which can impact the accuracy of both ASR and keyword spotting.
- **Vocabulary and Language Modeling:** Proper vocabulary and language modeling are essential to handle different word pronunciations, variations, and language-specific characteristics.

**Applications of Spoken Language Audio Retrieval Include:**

- Voice search in audio or video content
- Multimedia content retrieval based on spoken keywords
- Content-based ad targeting in audio and video streams
- Building voice assistants and voice-controlled systems

State-of-the-art Spoken Language Audio Retrieval systems typically utilize various techniques, such as:

- Advanced ASR models, including deep learning-based approaches like Recurrent Neural Networks (RNNs) and transformer models.
- Effective keyword spotting algorithms, often incorporating neural network-based methods.
- Query expansion and spoken term modeling techniques to improve retrieval performance.
- Fusion of audio and text-based features for better accuracy.

As with many machine learning and information retrieval tasks, the continuous development of deep learning models and the availability of large, labeled datasets have significantly improved the performance of Spoken Language Audio Retrieval systems in recent years.

Spoken Language Audio Retrieval involves searching and retrieving audio content based on the spoken language or text present in the audio. In this example, we will build a simple spoken language audio retrieval system using Python and the popular speech recognition library called SpeechRecognition.

Before running the code, make sure you have installed the SpeechRecognition library. You can install it using pip:

```
pip install SpeechRecognition
```

Next, we need an audio file containing spoken language that we want to search for. For this example, let's assume you have an audio file named "sample_audio.wav" with some spoken language content.

Now, let's write the Python code to perform spoken language audio retrieval:

```
import speech_recognition as sr

def audio_to_text(audio_file):
    recognizer = sr.Recognizer()

    # Load the audio file
    with sr.AudioFile(audio_file) as source:
        audio_data = recognizer.record(source)

    # Recognize the speech in the audio
    try:
        text = recognizer.recognize_google(audio_data)
        return text
    except sr.UnknownValueError:
        print("Could not understand audio")
        return ""
    except sr.RequestError as e:
        print(f"Error occurred; {e}")
        return ""

def main():
    audio_file = "sample_audio.wav"

    # Convert audio to text
    spoken_text = audio_to_text(audio_file)

    if spoken_text:
        print("Spoken text in the audio:")
        print(spoken_text)
    else:
        print("No spoken text detected in the audio.")

if __name__ == "__main__":
    main()
```



In this code, we define a function audio_to_text(audio_file) that takes the path of the audio file as input and uses the SpeechRecognition library to convert the speech in the audio to text. We use Google's Web Speech API for speech recognition, but you can also use other APIs supported by SpeechRecognition.

The audio_to_text() function returns the recognized text, or an empty string if no speech is detected or an error occurs during speech recognition.

In the main() function, we call the audio_to_text() function with the path of the "sample_audio.wav" file. The recognized text (spoken language content) is then printed to the console.

Please note that the accuracy of speech recognition can vary based on factors like audio quality, background noise, and the language spoken in the audio. For better accuracy, you may need to explore more advanced speech recognition models or techniques.

### 6.1.2 Non-Speech Audio Retrieval

Non-speech audio retrieval is the process of searching and retrieving audio content that does not contain human speech. Unlike spoken language audio retrieval, which focuses on finding specific spoken words or phrases, non-speech audio retrieval deals with identifying and retrieving audio segments that contain sounds other than human speech. This includes sounds such as music, environmental sounds, animal sounds, music instruments, mechanical sounds, and more.

**(a) Non-Speech Audio Retrieval Finds Applications in Various Domains, Including:**

- **Music Retrieval:** Identifying and retrieving music tracks based on audio content. This can be used in music recommendation systems, music search engines, and content-based music retrieval systems.
- **Sound Effects Retrieval:** Searching for specific sound effects, like explosions, footsteps, or car engines, for use in multimedia production, video games, and film industry.
- **Environmental Sound Retrieval:** Identifying and retrieving sounds from the environment, such as birds chirping, rain, traffic noises, etc. This can be used in soundscape analysis, wildlife monitoring, and urban noise studies.
- **Musical Instrument Retrieval:** Searching for audio segments containing specific musical instruments. This can be useful for music education, instrument recognition, and music production.

**(b) The Process of Non-speech Audio Retrieval involves Several Steps, Including:**

- **Audio Feature Extraction:** Converting the audio signal into a suitable representation for analysis and retrieval. Common audio features include Mel-Frequency Cepstral Coefficients (MFCCs), chroma features, spectral features, and others.
- **Indexing and Retrieval:** Creating an index of the audio content to facilitate efficient searching and retrieval. Various indexing techniques, such as inverted files, tree-based indexing, and hashing, can be used.
- **Querying:** Formulating a query representing the target non-speech sound to find similar audio segments in the database.
- **Similarity Matching:** Calculating the similarity between the query and the audio segments in the database to rank the retrieved results.

- **Relevance Feedback:** Incorporating user feedback to refine search results and improve the accuracy of the retrieval system.

Machine learning techniques, particularly deep learning models, have shown promising results in non-speech audio retrieval tasks, especially when dealing with large and complex audio datasets. Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) are commonly used to learn discriminative audio representations for effective retrieval.

Simple example of non-speech audio retrieval using Python. In this example, we will use the Librosa library to extract audio features and the NearestNeighbors algorithm from scikit-learn to perform similarity matching for retrieval. We will focus on retrieving audio segments with specific musical instruments.

Before running the code, make sure you have installed the required libraries:

```
pip install librosa
pip install scikit-learn
```

For this example, we assume you have a directory containing audio files with various musical instruments. Let's name this directory "audio_dataset" and assume it contains audio files in WAV format.

Here's the Python code:

```
import os
import librosa
import numpy as np
from sklearn.neighbors import NearestNeighbors

def extract_features(audio_file):
    # Load the audio file
    y, sr = librosa.load(audio_file, sr=None)

    # Extract Mel-frequency cepstral coefficients (MFCCs) as features
    mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)

    # Calculate the mean of each MFCC coefficient along the time axis to get a single feature vector
    mfcc_mean = np.mean(mfccs, axis=1)
    return mfcc_mean

def retrieve_similar_instruments(query_file, dataset_dir):
    # Extract features from the query audio
    query_features = extract_features(query_file)

    # Initialize a list to hold the features of the dataset
    dataset_features = []
    audio_files = os.listdir(dataset_dir)

    # Extract features from all audio files in the dataset
    for audio_file in audio_files:
        audio_path = os.path.join(dataset_dir, audio_file)
        feature_vector = extract_features(audio_path)
        dataset_features.append(feature_vector)

    # Convert the list of feature vectors to a numpy array
    dataset_features = np.array(dataset_features)

    # Initialize the NearestNeighbors algorithm
    nn = NearestNeighbors(n_neighbors=5, algorithm='auto')
    nn.fit(dataset_features)

    # Find the nearest neighbors (most similar instruments) for the query features
    distances, indices = nn.kneighbors([query_features])

    # Print the most similar instruments
    print("Most similar instruments to the query audio:")
    for idx in indices[0]:
        print(audio_files[idx])

def main():
    # Path to the query audio file
    query_file = "query_audio.wav"

    # Directory containing the audio dataset
    dataset_dir = "audio_dataset"

    retrieve_similar_instruments(query_file, dataset_dir)

if __name__ == "__main__":
    main()
```

In this code, we define the extract_features() function to extract Mel-Frequency Cepstral Coefficients (MFCCs) from an audio file. MFCCs are commonly used as audio features for music-related tasks.

The retrieve_similar_instruments() function takes the path to the query audio file and the directory containing the audio dataset as input. It extracts MFCC features from both the query audio and the audio files in the dataset. Then, it uses the NearestNeighbors algorithm to find the most similar instruments based on the MFCC features. The function prints the names of the five most similar instruments found in the dataset.

Please make sure to replace "query_audio.wav" with the path to your query audio file and "audio_dataset" with the path to the directory containing your audio dataset. Also, ensure that your audio files in the dataset are in WAV format and have different musical instruments to test the retrieval.

### 6.1.3 Graph Retrieval

Graph retrieval, also known as graph-based information retrieval, is a specialized area of information retrieval that focuses on searching and retrieving information from graph-structured data. In this context, a graph consists of nodes representing entities or objects and edges representing relationships or connections between these entities. Graph retrieval is commonly used in various domains where data is represented in a graph format, such as social networks, knowledge graphs, biological networks, and citation networks.

**(a) Graph Retrieval Involves Two Key Components:**

1. **Query Specification:** The user specifies a query in the form of a graph pattern or a subgraph, representing the information they are interested in retrieving. The query can be a simple subgraph with specific nodes and edges or a more complex graph pattern representing a specific structure.
2. **Graph Matching and Ranking:** The retrieval system searches through the graph database to find graphs or subgraphs that match the specified query. The matching process involves identifying subgraphs in the database that are isomorphic or structurally similar to the query. Once the candidate graphs are identified, they are ranked based on their relevance to the query. The ranking is typically done using graph similarity measures or relevance scoring methods.

Graph retrieval is challenging due to the complexity of graph data and the need to efficiently search large graph databases. Various graph indexing and search techniques have been developed to enhance the efficiency of graph retrieval. Some common techniques include:

- **Graph Indexing:** Preprocessing the graph database to create an index that facilitates faster graph matching and retrieval. Indexing techniques like graph signatures, inverted indexes, and graph embeddings are used to speed up the search process.
- **Graph Matching Algorithms:** Algorithms that efficiently find subgraphs in the database that are isomorphic or structurally similar to the query. Subgraph isomorphism algorithms, graph edit distance algorithms, and graph kernels are commonly used for graph matching.
- **Graph Embeddings:** Transforming graph data into low-dimensional vector representations (embeddings) to enable similarity comparison using vector space techniques.
- **Semantic-based Retrieval:** Incorporating semantic information and knowledge about the graph data to improve retrieval accuracy.

**(b) Graph Retrieval has Numerous Applications, Including:**

- Keyword-based search in knowledge graphs.
- Social network analysis and community detection.
- Identifying relevant scientific papers in citation networks.
- Recommender systems based on user-item interaction graphs.

The field of graph retrieval is an active area of research, and ongoing advancements in graph data representation, indexing techniques, and graph matching algorithms continue to improve the efficiency and effectiveness of graph retrieval systems.

To create a Python example of graph retrieval, we will use NetworkX, a popular Python library for working with graph data. In this example, we will demonstrate how to perform a simple graph retrieval task by searching for subgraphs that match a specific query pattern in a small graph database.

Before running the code, make sure you have installed NetworkX:

```
pip install network
```

Let's assume we have a small graph database represented as follows:

```
import networkx as nx

# Define the graph database
graph_database = nx.Graph()

# Add nodes to the graph
graph_database.add_nodes_from([1, 2, 3, 4, 5])

# Add edges to the graph
graph_database.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5)])
```

Now, let's define a query pattern and implement the graph retrieval function:

```
def graph_retrieval(graph_database, query_pattern):
    # Initialize a list to store the matching subgraphs
    matching_subgraphs = []

    # Iterate through all nodes in the graph database to find matching subgraphs
    for node in graph_database.nodes():
        # Get the subgraph centered at the current node
        subgraph = nx.ego_graph(graph_database, node, radius=1, center=True)

        # Check if the subgraph matches the query pattern
        if nx.is_isomorphic(subgraph, query_pattern):
            matching_subgraphs.append(subgraph)

    return matching_subgraphs

def main():
    # Define the query pattern (subgraph) we want to search for
    query_pattern = nx.Graph()
    query_pattern.add_nodes_from([1, 2])
    query_pattern.add_edges_from([(1, 2)])

    # Perform graph retrieval
    matching_subgraphs = graph_retrieval(graph_database, query_pattern)

    # Print the matching subgraphs
    print("Matching Subgraphs:")
    for subgraph in matching_subgraphs:
        print(subgraph.nodes())
        print(subgraph.edges())

if __name__ == "__main__":
    main()
```

In this example, we define a simple graph database with nodes {1, 2, 3, 4, 5} and edges {(1, 2), (1, 3), (2, 4), (2, 5)}. Our query pattern is a subgraph consisting of nodes {1, 2} and edge (1, 2).

The graph_retrieval() function iterates through all nodes in the graph database and extracts subgraphs centered at each node with a given radius. It then checks if the subgraph matches the query pattern using NetworkX's is_isomorphic() function, which checks if two graphs are isomorphic (i.e., structurally identical).

In the main() function, we call the graph_retrieval() function with the graph database and the query pattern. The function returns a list of matching subgraphs, and we print them to the console.

Please note that this is a simple illustrative example. In real-world scenarios, larger graph databases, more complex query patterns, and more sophisticated graph matching techniques would be used for practical graph retrieval tasks.



### 6.1.4 Imagery Retrieval

Imagery retrieval, also known as Content-Based Image Retrieval (CBIR), is the process of searching and retrieving images from a large image database based on their visual content rather than using textual metadata or tags. The goal of imagery retrieval is to find images that are visually similar to a given query image, making it a valuable tool for various applications, such as image search engines, image recommendation systems, and image organization tools.

The process of imagery retrieval typically involves the following steps:

1. **Feature Extraction:** Extracting meaningful and discriminative features from images to represent their visual content. Commonly used features include color histograms, texture descriptors, shape descriptors, and deep learning-based features extracted from pre-trained Convolutional Neural Networks (CNNs).
2. **Indexing and Storage:** Indexing the feature vectors of all images in the database to facilitate efficient and fast searching. Various indexing techniques, such as inverted files, KD-trees, and product quantization, can be used to speed up the search process.
3. **Query Formulation:** Converting the query image into a feature vector using the same feature extraction techniques applied to the database images.
4. **Similarity Measurement:** Calculating the similarity between the query image's feature vector and the feature vectors of images in the database. Different similarity metrics like Euclidean distance, cosine similarity, or chi-squared distance are commonly used.
5. **Ranking and Retrieval:** Ranking the images in the database based on their similarity to the query image and retrieving the top-k most similar images.
6. **Visualization and Presentation:** Displaying the retrieved images to the user in a visually appealing and organized manner.

**Python Example:**

Here's a simple Python example of imagery retrieval using the popular library OpenCV for feature extraction and scikit-learn for similarity measurement:

```
import cv2
import os
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def extract_feature(image_path):
    # Load the image using OpenCV
    image = cv2.imread(image_path)

    # Resize the image to a fixed size (optional)
    image = cv2.resize(image, (256, 256))

    # Convert the image to a feature vector (e.g., using color histograms)
    hist = cv2.calcHist([image], [0, 1, 2], None, [8, 8, 8], [0, 256, 0, 256, 0, 256])
    hist = cv2.normalize(hist, hist).flatten()

    return hist

def image_retrieval(query_image_path, database_dir):
    # Extract features from the query image
    query_feature = extract_feature(query_image_path)

    # Initialize a dictionary to store the similarity scores
    similarity_scores = {}

    # Iterate through all images in the database directory
    for filename in os.listdir(database_dir):
        image_path = os.path.join(database_dir, filename)
        # Extract features from each image in the database
        image_feature = extract_feature(image_path)
        # Calculate the cosine similarity between the query image and the database image
        similarity = cosine_similarity([query_feature], [image_feature])[0][0]
        # Store the similarity score for each image
        similarity_scores[filename] = similarity
```

```
    # Sort the similarity scores in descending order
    sorted_scores = sorted(similarity_scores.items(), key=lambda x: x[1], reverse=True)

    # Retrieve and return the top-k similar images
    top_k = 5
    top_k_images = [os.path.join(database_dir, item[0]) for item in sorted_scores[:top_k]]

    return top_k_images

def main():
    # Path to the query image
    query_image_path = "query_image.jpg"

    # Directory containing the database images
    database_dir = "image_database"

    # Perform image retrieval
    top_k_images = image_retrieval(query_image_path, database_dir)

    # Print the paths of the top-k similar images
    print("Top-k similar images:")
    for image_path in top_k_images:
        print(image_path)

if __name__ == "__main__":
    main()
```

In this example, we define a simple color histogram feature extraction method using OpenCV. The extract_feature() function takes an image path as input, loads the image, resizes it, and computes its color histogram. The histogram is then normalized and flattened to obtain the feature vector.

The image_retrieval() function performs imagery retrieval by calculating the cosine similarity between the query image and each image in the database directory. The top-k similar images are then retrieved and printed to the console.

Please note that this example uses a basic color histogram feature, which may not provide optimal retrieval accuracy for complex images. For more sophisticated image retrieval tasks, advanced deep learning-based features and techniques like CNNs are commonly used.

### 6.1.5 Video Retrieval Recommender System

- **Introduction:** A Video Retrieval Recommender System is an information retrieval system that aims to recommend relevant videos to users based on their preferences and the content of videos. It combines techniques from content-based video retrieval and recommender systems to provide personalized video recommendations.
- **Content-Based Video Retrieval:** Content-based video retrieval focuses on extracting visual features from videos and using these features to search and retrieve similar videos. The visual features can include color histograms, texture descriptors, motion features, and deep learning-based features extracted from video frames or segments. In a video retrieval recommender system, content-based techniques are used to find videos that are visually similar to a given query video.
- **Recommender Systems:** Recommender systems aim to predict users' interests and preferences to provide personalized recommendations. There are different types of recommender systems, including collaborative filtering, content-based filtering, and hybrid approaches. In a video retrieval recommender system, the focus is on content-based filtering, where videos are recommended based on their content similarity to the user's interests.
- **Workflow of a Video Retrieval Recommender System:** The workflow of a video retrieval recommender system typically involves the following steps:
  1. **User Profiling:** Collecting information about the user's preferences, interests, and past video interactions to build a user profile.
  2. **Video Representation:** Extracting visual features from videos to represent their content.
  3. **Query Formulation:** Formulating user queries based on their preferences or the content of a selected video.
  4. **Content-Based Video Retrieval:** Using content-based video retrieval techniques to find videos that are visually similar to the query video or the user's preferences.



5. **Relevance Ranking:** Ranking the retrieved videos based on their relevance to the query or user preferences.
6. **Recommendation Generation:** Selecting the top-k relevant videos as recommendations to the user.
7. **Feedback Incorporation:** Collecting user feedback on the recommended videos to improve the system's performance over time.

- **Challenges:** Building an effective video retrieval recommender system comes with several challenges:
  1. **Feature Extraction:** Selecting and extracting meaningful and discriminative visual features from videos is crucial for accurate retrieval and recommendation.
  2. **Scalability:** Handling large-scale video databases efficiently and providing real-time recommendations can be challenging.
  3. **User Modeling:** Capturing accurate user preferences and building robust user profiles is essential for personalized recommendations.
  4. **Cold Start Problem:** Addressing the cold start problem, where the system lacks sufficient user data to make accurate recommendations for new users.

**Applications:** Video retrieval recommender systems have diverse applications, including:

- **Video Streaming Platforms:** Recommending relevant videos to users based on their viewing history and preferences.
- **Video Sharing Platforms:** Assisting users in discovering videos that match their interests.
- **Personalized Video Search Engines:** Enabling users to search for videos based on visual content rather than just text-based metadata.
- **Video Content Recommendation in E-Learning:** Recommending educational videos to students based on their learning preferences.

**Advancements:** Advancements in deep learning, particularly in video understanding and feature extraction using Convolutional Neural Networks (CNNs), have significantly improved the performance of video retrieval recommender systems. Pre-trained CNN models, such as I3D (Inflated 3D ConvNet), can be fine-tuned for video feature extraction, leading to more effective content-based retrieval and recommendation.

Overall, video retrieval recommender systems play a crucial role in enhancing user experiences, increasing engagement, and enabling effective video content discovery in various domains. As technology and algorithms continue to evolve, video retrieval recommender systems are expected to become more accurate and efficient in providing personalized video recommendations.

Creating a complete video retrieval recommender system requires a lot of components and data, which can be complex. basic Python code example that demonstrates the core functionality of content-based video retrieval using simple features. We'll use OpenCV for video processing and scikit-learn for similarity calculations.

For this example, we'll assume that we have a small video database with a few video files. We'll use color histograms as the video features for simplicity.

**Requirements:**

- **OpenCV:** A popular computer vision library to work with video frames.
- **Scikit-Learn:** A machine learning library to perform similarity calculations.

You can install the required libraries using pip:

```
pip install opencv-python
pip install scikit-learn
```

Here's the basic Python code example:

```
import cv2
import os
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def extract_video_features(video_path):
    cap = cv2.VideoCapture(video_path)
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Initialize an empty array to store features from all frames
    all_features = np.empty((frame_count, 256 * 3))
    idx = 0

    while True:
        ret, frame = cap.read()
```

```
    if not ret:
        break

    # Convert the frame to a feature vector (e.g., using color histograms)
    hist = cv2.calcHist([frame], [0, 1, 2], None, [8, 8, 8], [0, 256, 0, 256, 0, 256])
    hist = cv2.normalize(hist, hist).flatten()

    all_features[idx] = hist
    idx += 1

  cap.release()

  return all_features

def video_retrieval(query_video_path, video_database_dir):
  # Extract features from the query video
  query_features = extract_video_features(query_video_path)

  # Initialize a dictionary to store the similarity scores
  similarity_scores = {}

  # Iterate through all videos in the database directory
  for filename in os.listdir(video_database_dir):
    video_path = os.path.join(video_database_dir, filename)
    # Extract features from each video in the database
    video_features = extract_video_features(video_path)
    # Calculate the average cosine similarity between the query video and the database video
    similarity = cosine_similarity([query_features.mean(axis=0).reshape(1, -1)], [video_features.mean(axis=0).reshape(1, -1)])[0][0]
    # Store the similarity score for each video
    similarity_scores[filename] = similarity

  # Sort the similarity scores in descending order
  sorted_scores = sorted(similarity_scores.items(), key=lambda x: x[1], reverse=True)

  # Retrieve and return the top-k similar videos
  top_k = 5
  top_k_videos = [os.path.join(video_database_dir, item[0]) for item in sorted_scores[:top_k]]

  return top_k_videos

def main():
  # Path to the query video
  query_video_path = "query_video.mp4"

  # Directory containing the video database
  video_database_dir = "video_database"

  # Perform video retrieval and recommendation
  top_k_videos = video_retrieval(query_video_path, video_database_dir)

  # Print the paths of the top-k similar videos
  print("Top-k similar videos:")
  for video_path in top_k_videos:
    print(video_path)

if __name__ == "__main__":
  main()
```

In this example, we define a simple color histogram feature extraction method using OpenCV. The extract_video_features() function takes a video path as input, reads all frames from the video, and computes the color histogram for each frame. The histograms are then concatenated into a feature vector representing the entire video.



The video_retrieval() function performs content-based video retrieval by calculating the cosine similarity between the query video and each video in the database directory. The top-k similar videos are then retrieved and printed to the console.

Please note that this is a basic example, and more sophisticated video feature extraction methods, such as deep learning-based features from pre-trained models like CNNs, would be required for better video retrieval performance in real-world applications. Additionally, a more comprehensive video database and user preference modeling would be needed for a full-fledged video retrieval recommender system.

***Collaborative Filtering can be Divided into Two Main Approaches:***

1. **User-Based Collaborative Filtering:** In this approach, the system recommends items to a target user based on the preferences of other users who are similar to that target user. The similarity between users is measured based on their past interactions with items. The steps involved in user-based collaborative filtering are as follows:
2. **Build a User-Item Matrix:** Create a matrix where rows represent users, columns represent items, and the matrix elements represent user-item interactions (e.g., ratings).

- **Measure user Similarity:** Calculate the similarity between the target user and all other users based on their interactions with items. Common similarity metrics include cosine similarity, Pearson correlation, and Jaccard similarity.
- **Select Top Similar Users:** Identify the top-k most similar users to the target user.
- **Predict Ratings:** Predict the target user's ratings for items they have not interacted with by taking weighted averages of ratings given by the top-k similar users.
- **Item-Based Collaborative Filtering:** In this approach, the system recommends items to a target user based on the preferences of other users who have interacted with similar items. The similarity between items is measured based on user-item interactions. The steps involved in item-based collaborative filtering are as follows:
  - ➢ **Build a User-Item Matrix:** Create a matrix where rows represent users, columns represent items, and the matrix elements represent user-item interactions (e.g., ratings).
  - ➢ **Measure Item Similarity:** Calculate the similarity between items based on user-item interactions. Common similarity metrics include cosine similarity, Pearson correlation, and Jaccard similarity.
  - ➢ **Select Top Similar Items:** Identify the top-k most similar items to the items the target user has already interacted with.
  - ➢ **Predict Ratings:** Predict the target user's ratings for items they have not interacted with by taking weighted averages of ratings given by users who have interacted with similar items.

## 6.2 RECOMMENDATION SYSTEMS

Recommendation systems predict the user preferences or ratings that users would give to items. The recommendation system is very highly used in movies, news, advisement, music, etc.

The best examples of recommendation systems are Youtube, IMDb, Amazon, Flipkart, etc.

**Introduction**

As a part of writing a blog on the ML topic, I selected a problem statement is Collaborative Filtering. This is a part of the recommendation systems, we have two techniques, In this bog we major focus on Collaborative-based filtering, this blog is about a very simple introduction of collaborative filtering.

In this filtering, we use user and movie rating matrix and then using these ratings we find common movies that have the same interest-as other movies.

- Collaborative-based Filtering.
- Content-based Filtering.

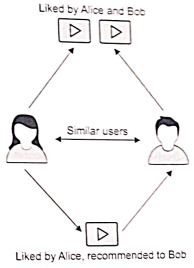



**Fig. 6.1**

### 6.2.1 Collaborative Filtering

Collaborative filtering is used by most recommendation systems to find similar patterns or information of the users, this technique can filter out items that users like on the basis of the ratings or reactions by similar users.

An example of collaborative filtering can be to predict the rating of a particular user based on user ratings for other movies and others' ratings for all movies. This concept is widely used in recommending movies, news, applications, and so many other items.

**Collaborative Filtering :**

Collaborative filtering is a popular technique used in recommender systems to provide personalized recommendations to users. It is based on the idea that users who have agreed in the past will likely agree in the future, and users with similar preferences can be used to predict a user's interests or preferences for certain items.

Let's take one example and understand more about what is Collaborative Filtering, let's assume I have user U1, who likes movies m1, m2, m4. user U2 who likes movies m1,m3,m4, and user U3 who likes movie m1.

So our job is to recommend which are the new movie to watch for the user U3 next.

So here we can see users U1, U2, U3 watch/likes movies m1, so three have the same taste. now in user U1, U2 has like/watch movies m4, so user U3 could like movie m3 so I recommend movie m4, this is the flow of logic.

The key idea of CF is Users who agreed in the past tend to also agree in the future.

| Users | Movie 1 | Movie 2 | Movie 3 | Movie 4 |
|---|---|---|---|---|
| User 1 | 5 | 5 | | 5 |
| User 2 | 4 | | 3 | |
| User 3 | | 1 | | 2 |
| User 4 | 1 | 2 | | |

**Types of Filtering**

There are two types of Collaborative Filtering available:

(i) User-User-based similarity/Collaborative Filtering

(ii) Item-Item-based similarity/Collaborative Filtering

The most popular Collaborative Filtering is item-item-based Collaborative Filtering.

**(i) User-User-Based Collaborative Filtering**

User-user collaborative filtering is one kind of recommendation method which looks for similar users based on the items users have already liked or positively interacted with. Let's take a one eg to understand user-user collaborative filtering.

Let's assume given matrix A which contains user id and item id and rating or movies.

| | Item 1 | Item 2 | Item 3 | Item 4 | Item 5 |
|---|---|---|---|---|---|
| User 1 | 0 | 3 | 0 | 3 | 4 |
| User 2 | 4 | 0 | 0 | 2 | 0 |
| User 3 | 0 | 0 | 3 | 0 | 0 |
| User 4 | 3 | 0 | 4 | 0 | 3 |
| User 5 | 4 | 3 | 0 | 4 | 4 |

Compute a User User similarity follow these steps, so find a similarity between two users we can use cosine similarity.

So cosine similarity means the similarity between two vectors of inner product space, It is measured by the cosine of the angle between two vectors.

Similarity

$$(A, B) = \frac{A \cdot B}{||A|| \times ||B||} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \times \sqrt{\sum_{i=1}^{n} B_i^2}}$$

**How to Compute the Cosine Similarity?**

To compute a cosine similarity we take a one eg here.

doc_of_food = 'this food is good but not a highly recommended by the foodies'

doc_of_election = "prime minister ND modi says Putin had no political interference is the election outcome.

doc_of_putin = "Post elections Vladimir Putin became President of Russia President Vladimir Putin had served as the Prime Minister earlier in his political career"

doc = [doc_of_food, doc__of_election, doc__ofputin]

I am assuming you all know how to encode a text to vector, so here we use a TfidfVectorizer() or CountVectorizer() as encoding the sentences.



Here we use a CountVectorizer to encode our document/text.

```
# Scikit Learn
from sklearn.feature_extraction.text import
CountVectorizer
import pandas as pd

# Create the Document Term Matrix
count_vectorizer =
CountVectorizer(stop_words='english')
count_vectorizer = CountVectorizer()
sparse_matrix = count_vectorizer.fit_transform(doc)

# not necessary: Convert Sparse Matrix to Pandas
Dataframe if you want to see the word frequencies.
doc_term_matrix = sparse_matrix.todense()
df = pd.DataFrame(doc_term_matrix,

columns=count_vectorizer.get_feature_names(),
                  index= [doc_of_food, doc__of_election,
doc__ofputin] )
df
# Compute Cosine Similarity
from sklearn.metrics.pairwise import cosine_similarity
print(cosine_similarity(df, df))
```

**Output:**

```
[[1.         0.59160798 0.34785054]
 [0.59160798 1.         0.37416574]
 [0.34785054 0.37416574 1.        ]]
```

so given a matrix I can compute a cosine similarity matrix as the similarity between two users.

$Sim_{ij}$ = similarity($user_i$ , $user_j$)

**User-User Similarity Matrix**

| | $U_1$ | $U_2$ | | $U_j$ | | $U_{n-1}$ | $U_n$ |
|---|---|---|---|---|---|---|---|
| $U_1$ | 1 | $Sim_{12}$ | … | $Sim_{1j}$ | … | | |
| $U_2$ | | 1 | … | | … | | |
| | . . | . . | . . | . . | . . | . . | . . |
| $U_{n-1}$ | | | … | | … | 1 | |
| $U_n$ | | | … | | … | | 1 |

**Source Ineuron**

- So we find a similarity matrix here and our task is to recommend a new movie /item to a user.
- Suppose you have to recommend a new top 5 similar movie or item for a user 10, so we have a similarity matrix a and we go in a similarity matrix in user 10 and find a top 5 similar values corresponding to the user 10.
- Let's suppose the top 5 similar to user10 is user 9,5,8,1,2. now you go into our user-item matrix and take all items of the user 9,5,8,1,2 where they give a rating value and not watch by the user 10 and combine them. Then we pick all those items and we recommended them for a user 10.
- But there can be a small problem with a user-user similarity base system, user interests change over time, and then similarity values also change, and this is impacted on the recommended system.
- There is another approach which is an item item-based similarity recommendation system.

**Item-Item Based Collaborative Filtering**

- This is also very simple and very similar in idea with USER-USER Similarity Let's dive deep into it.
- This item-item similarity is solve a problem that occurs in a user user-based similarity.
- Here we find a similarity matrix of items/movies, here we find a similarity between the two movies. to find a similarity we use a cosine distance between the two movies.

**Item-Item Similarity Matrix**

| | $I_1$ | $I_2$ | | $I_j$ | | $I_{n-1}$ | $I_n$ |
|---|---|---|---|---|---|---|---|
| $I_1$ | 1 | $Sim_{12}$ | … | $Sim_{1j}$ | … | | |
| $I_2$ | | 1 | … | | … | | |
| | . . | . . | . . | . . | . . | . . | . . |
| $I_i$ | | $Sim_{i2}$ | … | $Sim_{ij}$ | … | | |
| | . . | . . | . . | . . | . . | . . | . . |
| $I_{n-1}$ | | | … | | … | 1 | |
| $I_n$ | | | … | | … | | 1 |

$Sim_{ij}$ = similarity($item_n$, $item_j$)

**So How to Recommend an Item to the User?**

Let's suppose we have to recommend new items to user10, and we know a user10 already likes/watch item7,8,1. Now we go to the item-item similarity matrix, we take the most similar item to items7,8,1 based on the similarity values.

let's suppose the most similar item for item7 is {item9, item4, item10}, the Most similar item to item8 is {item19, item 4, item10} and the Most similar item to item 1 is {item9, item14, item10}

Now we take a very common item from every set of items and the common items are {item9, item4, item10, item 19, item 14} and we recommend these all items to user10.

The most popular filtering is item item-based filtering because over time item is not changed like the user user-based similarity.

**Collaborative Filtering has Several Advantages**

It does not require item features or content information, making it applicable to a wide range of recommendation scenarios.

It can provide personalized recommendations based on user behavior and preferences.

It can handle new items and users without needing explicit information about them.

**However, Collaborative Filtering also has Some Disadvantages:**

- **Cold Start Problem:** Collaborative filtering struggles to make accurate recommendations for new users or items with limited interaction data.
- **Sparsity:** In large datasets, user-item interactions may be sparse, leading to challenges in finding meaningful user or item similarities.
- **Scalability:** As the number of users and items grows, calculating user or item similarities can become computationally expensive.

To address these limitations, hybrid approaches that combine collaborative filtering with other recommendation techniques, such as content-based filtering or matrix factorization, are often used to build more robust and accurate recommender systems.

## 6.2.2 Content Based Recommendation

Content-based recommendation is a popular approach used in recommender systems to provide personalized recommendations to users based on the content or features of items. It focuses on analyzing the characteristics of items and matching them with user preferences to make relevant recommendations. Content-based recommendation is especially useful when there is no or limited user interaction data available or when new items need to be recommended.

**The Key Steps Involved in Content-Based Recommendation are as Follows:**

- **Item Representation:** For each item, relevant content features are extracted to represent its characteristics. The type of features depends on the item type and the domain of the recommendation. For example, in a movie recommendation system, features might include movie genres, actors, directors, release year, and plot summary.
- **User Profile Creation:** The system builds a user profile that represents the user's preferences based on their interactions with items or explicit feedback. The user profile can also be constructed using attributes or features from the items the user has shown interest in.
- **Similarity Calculation:** The similarity between items and the user's profile is calculated using similarity measures, such as cosine similarity, Jaccard similarity, or Pearson correlation. These measures assess the degree of similarity between the features of items and the user's preferences.
- **Ranking and Recommendation:** The system ranks items based on their similarity to the user's profile and recommends the top-k most similar items to the user. The user is presented with these recommended items as personalized suggestions.

**Advantages of Content-Based Recommendation**

- **Independence from User Behavior:** Content-based recommendation does not rely on user interaction data, making it suitable for new users or users with limited interaction history.
- **Explanation of Recommendations:** The recommendations can be easily explained to users based on the features of the items that match their preferences.
- **Serendipitous Discovery:** Content-based recommendation can lead to serendipitous discoveries of items with novel or diverse characteristics that may not have been popular among other users.



**Disadvantages of Content-Based Recommendation**

- **Limited Diversity:** Content-based recommendation might not capture unexpected or diverse item preferences beyond the features used for representation.
- **Over-Specialization:** The system might provide recommendations based on a narrow set of features, leading to over-specialization and not capturing the user's broader interests.
- **Cold Start Problem:** Content-based recommendation can still face a cold start problem when there is limited information about new items or users.
- **Feature Engineering:** The effectiveness of content-based recommendation heavily depends on the selection and quality of relevant item features, which can be challenging to engineer in some cases.

**What are Content Recommender Systems?**

Recommender systems or recommendation systems (sometimes replacing "system" with a synonym such as platform or engine) are a subclass of information filtering system that seek to predict the 'rating' or 'preference' that user would give to an item."

Recommender systems are active information filtering systems which personalize the information coming to a user based on his interests, relevance of the information etc. Recommender systems are used widely for recommending movies, articles, restaurants, places to visit, items to buy etc.

**How do Content Based Recommender Systems Work?**

A content based recommender works with data that the user provides, either explicitly (rating) or implicitly (clicking on a link). Based on that data, a user profile is generated, which is then used to make suggestions to the user. As the user provides more inputs or takes actions on the recommendations, the engine becomes more and more accurate.

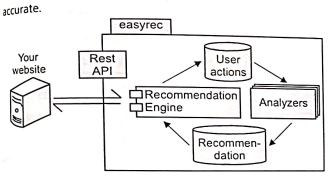



Fig. 6.2

**What are the Concepts Used in Content Based Recommenders?**

The concepts of Term Frequency (TF) and Inverse Document Frequency (IDF) are used in information retrieval systems and also content based filtering mechanisms (such as a content based recommender). They are used to determine the relative importance of a document / article / news item / movie etc.

**Term Frequency (TF) and Inverse Document Frequency (IDF)**

- TF is simply the frequency of a word in a document. IDF is the inverse of the document frequency among the whole corpus of documents. TF-IDF is used mainly because of two reasons: Suppose we search for "the rise of analytics" on Google. It is certain that "the" will occur more frequently than "analytics" but the relative importance of analytics is higher than the search query point of view. In such cases, TF-IDF weighting negates the effect of high frequency words in determining the importance of an item (document).
- But while calculating TF-IDF, log is used to dampen the effect of high frequency words. For example, TF = 3 vs TF = 4 is vastly different from TF = 10 vs TF = 1000. In other words the relevance of a word in a document cannot be measured as a simple raw count and hence the equation below:

**Equation:**

$$w_{t,d} = \begin{cases} 1 + \log_{10} tf_{t,d}, & \text{if } tf_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

| Term Frequency | Weighted Term Frequency |
|---|---|
| 0 | 0 |
| 10 | 2 |
| 1000 | 4 |

It can be seen that the effect of high frequency words is dampened and these values are more comparable to each other as opposed to the original raw term frequency.

After calculating TF-IDF scores, how do we determine which items are closer to each other, rather closer to the user profile? This is accomplished using the Vector Space Model which computes the proximity based on the angle between the vectors.

**How does Vector Space Model Works?**

In this model, each item is stored as a vector of its attributes (which are also vectors) in an n-dimensional space and the angles between the vectors are calculated to determine the similarity between the vectors. Next, the user profile vectors are also created based on his actions on previous attributes of items and the similarity between an item and a user is also determined in a similar way.

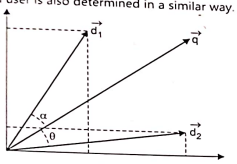



**Fig. 6.3**

Lets try to understand this with an example.

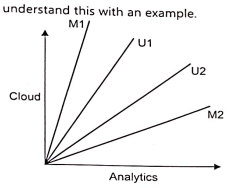



**Fig. 6.4**

Shown above is a 2-D representation of a two attributes, Cloud & Analytics. M1 & M2 are documents. U1 & U2 are users. The document M2 is more about Analytics than cloud whereas M1 is more about cloud than Analytics. I am sure you want to know how the relative importance of documents are measures. Hold on, we are coming to that.

User U1, likes articles on the topic 'cloud' more than the ones on 'analytics' and vice-versa for user U2. The method of calculating the user's likes / dislikes / measures is calculated by taking the cosine of the angle between the user profile vector (Ui) and the document vector.

The ultimate reason behind using cosine is that the value of cosine will increase with decreasing value of the angle between which signifies more similarity. The vectors are length normalized after which they become vectors of length 1 and then the cosine calculation is simply the sum-product of vectors. We will be using the function sumproduct in excel which does the same to calculate similarities between vectors in the next section.

**Case Study 1 : How to Calculate TF – IDF?**

Let's understand this with an example. Suppose, we search for "IoT and analytics" on Google and the top 5 links that appear have some frequency count of certain words as shown below:

**Table 6.1**

| Articles | Analytics | Data | Cloud | Smart | Insight |
|---|---|---|---|---|---|
| Article 1 | 21 | 24 | 0 | 2 | 2 |
| Article 2 | 24 | 59 | 2 | 1 | 0 |
| Article 3 | 40 | 115 | 8 | 10 | 19 |
| Article 4 | 4 | 28 | 5 | 0 | 1 |
| Article 5 | 8 | 48 | 4 | 3 | 4 |
| Article 6 | 17 | 49 | 8 | 0 | 5 |
| DF | 5,000 | 50,000 | 10,000 | 5,00,000 | 7,000 |

Among the corpus of documents / blogs which are used to search for the articles, 5000 contains the word analytics, 50,000 contain data and similar count goes for other words. Let us assume that the total corpus of docs is 1 million ($10^6$).

**Table 6.2: Term Frequency (TF)**

| Articles | Analytics | Data | Cloud | Smart | Insight | Length of Vector |
|---|---|---|---|---|---|---|
| Article 1 | 2.322219295 | 2.380211242 | 0 | 1.301029996 | 1.30102996 | 3.800456039 |
| Article 2 | 2.380211242 | 2.770852012 | 1.301029996 | 1 | 0 | 4.004460697 |
| Article 3 | 2.602059991 | 3.06069784 | 1.903089987 | 2 | 2.278753601 | 5.380804488 |
| Article 4 | 1.60205991 | 2.447158031 | 1.69897004 | 0 | 1 | 3.527276247 |
| Article 5 | 1.903089987 | 2.681241237 | 1.602059991 | 1.477121255 | 1.602059991 | 4.257450611 |
| Article 6 | 2.230448921 | 2.69019608 | 1.903089987 | 0 | 1.69897004 | 4.326697114 |

Table 6.2, for article 1, the term Analytics has a TF of 1+ log1021 = 2.322. In this way, TF is calculated for other attributes of each of the articles. These values make up the attribute vector for each of the articles.



***Inverse Document Frequency (IDF)***

IDF is calculated by taking the logarithmic inverse of the document frequency among the whole corpus of documents. So, if there are a total of 1 million documents returned by our search query and amongst those documents, 'smart' appears in 0.5 million documents. Thus, it's IDF score will be: Log10 ($10^6$/500000) = 0.30.

| DF | 5000 | 50000 | 10000 | 500000 | 7000 |
|---|---|---|---|---|---|
| IDF | 2.301029996 | 1.301029996 | 2 | 0.301029996 | 2.154901959 |
| N = | 1Cr | | | | |

**Fig. 6.5**

We can also see (image above), the most commonly occurring term 'smart' has been assigned the lowest weight by IDF. The length of these vectors are calculated as the square root of sum of the squared values of each attribute in the vector.

| Articles | Analytics | Data | Cloud | Smart | Insight | Length of Vector |
|---|---|---|---|---|---|---|
| Article 1 | 2.322219295 | 2.380211242 | 0 | 1.301029996 | =SQRT(U2^2+V2^2+L2^2+M2^2+N2^2) | |

After we have found out the TF -IDF weights and also vector lengths, let's normalize the vectors.

**Table 6.3**

| Articles | Analytics | Data | Cloud | Smart | Insight | Sum of Normalized Lengths |
|---|---|---|---|---|---|---|
| Articles 1 | 0.61103701 | 0.626296217 | 0 | 0.342335231 | 0.342335231 | 1 |
| Articles 2 | 0.594389962 | 0.691941368 | 0.324895184 | 0.249721517 | 0 | 1 |
| Articles 3 | 0.483581962 | 0.568817887 | 0.353681311 | 0.371691632 | 0.423496822 | 1 |
| Articles 4 | 0.454191812 | 0.693781224 | 0.481666273 | 0 | 0.283504872 | 1 |
| Articles 5 | 0.447002246 | 0.62977624 | 0.376295614 | 0.34694971 | 0.376295614 | 1 |
| Articles 6 | 0.51550845 | 0.621766675 | 0.43984211 | 0 | 0.39267152 | 1 |

Each term vector is divided by the document vector length to get the normalized vector. So, for the term 'Analytics' in Article 1, the normalized vector is: 2.322/3.800. Similarly all the terms in the document are normalized and as you can see Table 6.3 each document vector now has a length of 1.

Now, that we have obtained the normalized vectors, let's calculate the cosine values to find out the similarity between articles. For this purpose, we will take only three articles and three attributes.

**Table 6.4 (a)**

| Articles | Analytics | Data | Cloud |
|---|---|---|---|
| Article 1 | 0.61103701 | 0.626296217 | 0 |
| Article 2 | 0.594389962 | 0.691941368 | 0.324895184 |
| Article 3 | 0.483581962 | 0.568817887 | 0.353681311 |

**Table 6.4 (b)**

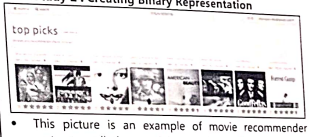
| Article Vector | Cosine Values |
|---|---|
| cos(A1, A2) | 0796554526 |
| cos(A2, A3) | 0.795934246 |
| cos(A1, A3) | 0.651734967 |

Cos(A1,A2) is simply the sum of dot product of normalized term vectors from both the articles. The calculation is as follows:

Cos(A1,A2) = 0.611*0.59 + 0.63 * 0.69 + 0* 0.32 = = 7965

As you can see the articles 1 and 2 are most similar and hence appear at the top two positions of search results. Also, if you remember we had said that Article 1 (M1) is more about analytics than cloud-Analytics has a weight of 0.611 whereas cloud has 0 after length normalization.

**Case Study 2 : Creating Binary Representation**




- This picture is an example of movie recommender system called movielens. This is a movie recommendation site which recommends movies you should watch based on which movies you have rated and how much you have rated them.
- At a rudimentary level, my inputs are taken in by the system and a profile is created against the attributes which contains my likes and dislikes (may be based on some keywords / movie tags I have liked, or not done anything about).

- I have liked 52 movies across various genres. These items (movies) are attributes since it can help us to determine a user's taste. Below is the list of top 6 movies recommended by movie-lens. The user columns specify whether a user has liked / disliked a recommended movie. The 1/0 simply represent whether the movie has adventure / action etc. or not. This type of representation is called a binary representation of attributes.

| Movie | Adventure | Action | Science-Fiction | Drama | Crime | Thriller |
|---|---|---|---|---|---|---|
| Start Wars IV | 1 | 1 | 1 | 0 | 0 | 0 |
| Saving private Ryan | 0 | 0 | 0 | 1 | 0 | 0 |
| American Beauty | 0 | 0 | 0 | 1 | 1 | 0 |
| City of Gold | 0 | 0 | 0 | 1 | 1 | 0 |
| Interstellar | 0 | 0 | 1 | 1 | 0 | 0 |
| The matrix | 1 | 1 | 1 | 0 | 0 | 1 |

| User 1 | User 1 |
|---|---|
| 1 | –1 |
| | |
| | |
| –1 | 1 |
| 1 | |
| | 1 |

In the above example, user 1 has liked Star Wars whereas user 2 has not. Based on these types of inputs a user profile is generated.

Other metrics like count data, simple 1 / 0 representation are also used. The measure depends upon the context of use. For example: for normal movie recommender system, a simple binary representation is more useful whereas for a search query in search engine like google, a count representation might be more appropriate.

Putting it all together – Building a Content based recommender for Analytics Vidhya (AV)

Now that we have learned all these concepts, let's actually try to build a simple content based recommender based on user taste for AV. Many articles have been published on AV. Every article published caters to a segment of audience such that some people might like articles on machine learning algorithms, while other might like R over machine learning. Hence, having a recommender system would help.

To build the recommender system:

First Step: In each week, for each user, we will track user engagement (like / share / comments) with various articles. Let's see how that can be done:

| Articles | Big Data | R | Python | Machine Learning | Learning Paths |
|---|---|---|---|---|---|
| Article 1 | 1 | 0 | 1 | 0 | 1 |
| Article 2 | 0 | 1 | 1 | 1 | 0 |
| Article 3 | 0 | 0 | 0 | 1 | 1 |
| Article 4 | 0 | 0 | 1 | 1 | 0 |
| Article 5 | 0 | 1 | 0 | 0 | 0 |
| Article 6 | 1 | 0 | 0 | 1 | 0 |
| | | | | | |
| DF | 2 | 2 | 3 | 4 | 2 |

| Total Attributes |
|---|
| 3 |
| 3 |
| 2 |
| 2 |
| 1 |
| 2 |

| User 1 | User 2 |
|---|---|
| 1 | –1 |
| –1 | 1 |
| | |
| | 1 |
| 1 | |
| | |

We have used binary representation here. The image shown above represents, Article 1 is about big data, python and learning path. Similarly, article 2 is about R, python and machine learning. User 1 has liked article 1 (shared it on social media) and has not liked article 2. He/She has not engaged with article 3,4,5 except reading them.

**Second Step: Normalize**

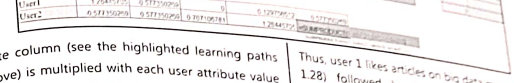
For binary representation, we can perform normalization by dividing the term occurrence(1/0) by the sqrt of number of attributes in the article. Hence, for Article 1: normalized attribute = $1\sqrt{(3)} = 0.577$.

A question that you must ask here is: what happened to TF? and why did we directly do normalization before calculating the TF scores? If you calculate TF without



normalization, the TF scores will be 1+log 10 1 = 1 for attributes that occur and simply 0 for attributes that don't. And, thus the TF scores will also becomes 1/0.

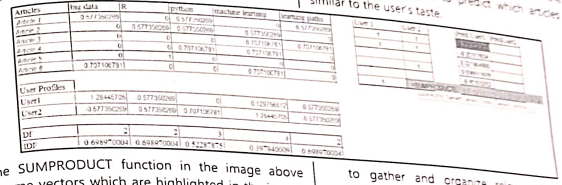
Let's see how the user profiles are generated by using the sumproduct function in excel.



Each attribute column (see the highlighted learning paths columns above) is multiplied with each user attribute value (highlighted part of User 2). This is simply the dot product of vectors that we saw earlier. This gives a user profile (as shown in the User Profiles part above) an understanding of user's taste.

Thus, user 1 likes articles on big data most (highest score of 1.28) followed by learning paths and then machine learning. Similarly, user 2 like articles on machine learning the most.

Now we have the user profile vectors and the article vectors, let's use these to predict which articles will be similar to the user's taste.



**Note:** The SUMPRODUCT function in the image above contains some vectors which are highlighted in the image. B36:F36 is the article 6 vector.

The dot product of article vectors and IDF vectors gives us the weighted scores of each article.

These weighted scores are again used for a dot product with the user profile vector (user 1 here). This gives a probability that the user will like a particular article. For article 1, the probability is 75%.

This concept can be applied to 'n' articles and we can find out which article a user will like the most. Therefore, along with new articles in a week, a separate recommendation can be made to a particular user based on the articles which he hasn't read already.

## 6.3 KNOWLEDGE BASED RECOMMENDATION INFORMATION EXTRACTION AND INTEGRATION

- Knowledge-based recommendation systems are a type of recommender system that relies on domain-specific knowledge and rules to make recommendations. They use information extraction and integration techniques to gather and organize relevant knowledge from various sources and create a knowledge base. This knowledge base is then used to generate personalized recommendations for users.

Here's an overview of the key steps involved in knowledge based recommendation, along with information extraction and integration:

**Information Gathering and Extraction:**

- **Data Sources:** Knowledge-based recommendat systems can gather information from various sour such as structured databases, unstructured text, profiles, product descriptions, reviews, and s media.
- **Information Extraction Techniques:** To e relevant information from these sources, techr like natural language processing (NLP), text m named entity recognition (NER), and inform retrieval may be used. These techniques h understanding and extracting meaningful infor from text and other unstructured data.
- **Knowledge Representation:** Once the information is extracted, it needs to be represe

structured and organized form. This is done by creating a knowledge base, which can be in the form of a knowledge graph, ontology, or database.

Knowledge graphs are commonly used to represent relationships between entities and attributes. Entities can be products, users, or any relevant objects, and attributes can represent properties such as ratings, descriptions, or user preferences.

- **Knowledge Integration:** Data from various sources might be in different formats and structures. Knowledge integration involves harmonizing and merging data from different sources into a unified knowledge base.

  Techniques like data normalization, data cleaning, and data transformation are applied to ensure consistency and accuracy in the knowledge base.

**Rule-Based Recommendation:**

Knowledge-based recommendation systems use rules and logic to generate recommendations for users. These rules are based on domain knowledge and expert rules, which define how items or content should be recommended to users based on their preferences and requirements.

For example, if a user has indicated an interest in action movies, the rule-based recommendation system can use domain knowledge to recommend action movies from the knowledge base.

**Personalization and Reasoning:**

Knowledge-based recommendation systems can personalize recommendations based on user preferences, demographics, and historical interactions.

Reasoning techniques, such as rule-based reasoning, can be applied to infer user preferences or to generate recommendations based on user queries and contextual information.

**Advantages of Knowledge-Based Recommendation**

- **Domain-Specific Recommendations:** Knowledge-based systems can provide recommendations tailored to specific domains and user preferences using domain-specific rules and knowledge.
- **Interpretability:** Recommendations generated by knowledge-based systems are often more interpretable and explainable, as they are based on explicit domain knowledge and rules.
- **Cold Start Handling:** Knowledge-based systems can handle the cold start problem by using explicit knowledge and rules to generate recommendations for new users or items.

**Disadvantages of Knowledge-Based Recommendation:**

- **Scalability:** Knowledge-based systems may face scalability challenges as the knowledge base grows in size and complexity.
- **Dependence on Domain Knowledge:** The effectiveness of knowledge-based recommendation heavily relies on the availability and accuracy of domain-specific knowledge and rules.
- To overcome limitations, hybrid approaches that combine knowledge-based recommendation with collaborative filtering or content-based techniques are often used to build more robust and accurate recommender systems. These hybrid systems can leverage the strengths of multiple approaches and provide more diverse and accurate recommendations to users.

### 6.3.1 Extracting Data from Text

Extracting data from text is a crucial task in Natural Language Processing (NLP) and information retrieval. It involves automatically identifying and extracting structured information or entities from unstructured text data, which enables the conversion of textual information into a structured and usable format. This process is essential for various NLP applications, including text analysis, sentiment analysis, information retrieval, question answering, and more.

- **Challenges in Data Extraction from Text:** Data extraction from text can be challenging due to the following factors:
  - **Unstructured Nature:** Text data is unstructured, meaning it lacks a predefined format or structure that can be easily parsed by machines.
  - **Ambiguity:** Natural language is inherently ambiguous, with words and phrases often having multiple meanings or interpretations.
  - **Variability:** Text data can be diverse in terms of writing styles, spelling variations, abbreviations, and typographical errors.
  - **Contextual Dependencies:** The meaning of words and phrases often depends on the context in which they are used.
  - **Named Entity Recognition (NER):** Identifying and classifying named entities, such as names of persons, organizations, locations, dates, etc., is a specific challenge in data extraction.



- **Techniques for Data Extraction from Text:** Several techniques are used for data extraction from text:
  - **Tokenization:** Breaking down the text into individual units called tokens (words, phrases, or characters).
  - **Part-of-Speech (POS) Tagging:** Assigning grammatical categories (e.g., noun, verb, adjective) to tokens to understand their roles in a sentence.
  - **Named Entity Recognition (NER):** Identifying and classifying named entities in the text.
  - **Regular Expressions:** Using pattern-matching rules to extract specific patterns or entities from text.
  - **Information Retrieval:** Searching for specific keywords or phrases in the text to extract relevant information.
  - **Keyword Extraction:** Identifying important and relevant keywords or phrases in the text.
  - **Machine Learning Techniques:** Using supervised and unsupervised machine learning algorithms to learn patterns and extract information from text.
- **Applications of Data Extraction from Text:** Data extraction from text has diverse applications, including:
  - **Information Retrieval:** Extracting information to answer user queries from large text collections or databases.
  - **Sentiment Analysis:** Extracting sentiments and opinions from text data.
  - **Text Summarization:** Extracting key sentences or phrases to create a concise summary of a document.
  - **Entity Linking:** Linking extracted entities to relevant knowledge bases or databases.
  - **Question Answering Systems:** Extracting answers to user questions from text data.
  - **Content Categorization:** Categorizing text into predefined topics or classes.

Information extraction from text involves automatically extracting structured information or entities from unstructured text data. This process is essential in knowledge-based recommendation systems to gather relevant data from various sources, such as product descriptions, reviews, and user profiles, and then integrate this information into the knowledge base for generating personalized recommendations.



Let's demonstrate information extraction from text using a simple example of extracting movie names and their corresponding genres from a list of movie descriptions.

**Example Text Data:**

1. "The Shawshank Redemption is a drama film directed by Frank Darabont."
2. "Inception is a science fiction action film directed by Christopher Nolan."
3. "The Dark Knight is a superhero film directed by Christopher Nolan."
4. "Pulp Fiction is a crime film directed by Quentin Tarantino."

We will use Python and the Natural Language Toolkit (NLTK) library for this example.

First, make sure you have NLTK installed:

```
pip install nltk
```

Here's the Python code for information extraction from the example text data:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag

def extract_movie_info(text):
    # Tokenize the text into words
    words = word_tokenize(text)

    # Perform part-of-speech (POS) tagging to identify word types
    tagged_words = pos_tag(words)

    # Initialize variables to store movie name and genre
    movie_name = ""
    genre = ""

    for i in range(len(tagged_words)):
        # Look for movie names followed by "is" (indicating description) and genre
        if tagged_words[i][1] == "NNP" and tagged_words[i+1][0] == "is":
            movie_name = tagged_words[i][0]
            genre = tagged_words[i+2][0]
```

```
            break
    return movie_name, genre

def main():
    text_data = [
        "The Shawshank Redemption is a drama film directed by Frank Darabont.",
        "Inception is a science fiction action film directed by Christopher Nolan.",
        "The Dark Knight is a superhero film directed by Christopher Nolan.",
        "Pulp Fiction is a crime film directed by Quentin Tarantino."
    ]

    for text in text_data:
        movie_name, genre = extract_movie_info(text)
        print(f"Movie: {movie_name}, Genre: {genre}")

if __name__ == "__main__":
    main()
```

**Output:**

Movie: The Shawshank Redemption, Genre: drama

Movie: Inception, Genre: science fiction

Movie: The Dark Knight, Genre: superhero

Movie: Pulp Fiction, Genre: crime

In this example, the extract_movie_info() function processes the text data using the NLTK library. It tokenizes the text into words and performs part-of-speech (POS) tagging to identify word types. It then extracts movie names and their corresponding genres from the text.

Please note that this is a simple example for illustration purposes. In real-world applications, more sophisticated natural language processing techniques and Named Entity Recognition (NER) methods can be used to handle complex text data and extract a broader range of information for knowledge-based recommendation systems.

## 6.4 SEMANTIC WEB

- The Semantic Web is an extension of the World Wide Web that aims to make web content more easily understandable and accessible to both humans and machines. It is based on the idea of adding semantic meaning to web data, allowing information to be linked, queried, and interpreted in a more intelligent and meaningful way.
- The vision of the Semantic Web was proposed by Sir Tim Berners-Lee, the inventor of the World Wide Web, in a scientific paper published in 2001. It is an ongoing initiative and is part of the broader field of artificial intelligence and knowledge representation.

**Key Concepts and Technologies of the Semantic Web:**

- **Resource Description Framework (RDF):** RDF is a fundamental data model used in the Semantic Web to represent information as triples (subject-predicate-object). Each triple expresses a statement about a resource, where the subject is the resource, the predicate is the property or relationship, and the object is the value.
- **Ontologies:** Ontologies are formal representations of knowledge that define concepts, relationships, and properties in a specific domain. They provide a shared vocabulary for describing and organizing information on the Semantic Web.
- **Web Ontology Language (OWL):** OWL is a standard language for creating ontologies on the Semantic Web. It allows for more complex modeling and reasoning about relationships between resources and concepts.
- **SPARQL:** SPARQL (SPARQL Protocol and RDF Query Language) is a query language used to query and retrieve data from RDF-based datasets on the Semantic Web.
- **Linked Data:** Linked Data principles promote the practice of interlinking and connecting data across different sources on the web. By using RDF and URIs (Uniform Resource Identifiers) to identify resources, datasets become part of a global data web.
- **Inference and Reasoning:** Inference engines and reasoning systems are used to draw conclusions and make deductions based on the knowledge represented in RDF and ontologies.

**Advantages and Applications of the Semantic Web:**

- **Data Integration:** The Semantic Web enables the integration of data from different sources, making it easier to link and combine information.
- **Machine-Readable Content:** By adding semantic meaning to web data, machines can better understand and process the information, leading to more intelligent applications.



- **Improved Search and Query:** Semantic Web technologies facilitate more precise and context-aware search and query capabilities.
- **Knowledge Sharing and Interoperability:** Ontologies and Linked Data promote sharing and interoperability between different systems and domains.
- **E-commerce and Recommendations:** The Semantic Web can enhance e-commerce by enabling more sophisticated product recommendations and personalized services.
- **Life Sciences and Healthcare:** Semantic Web technologies are used in life sciences and healthcare for data integration, interoperability, and knowledge representation.

**Challenges and Future Directions:**

Despite significant progress, the widespread adoption of the Semantic Web still faces challenges related to data quality, scalability, and the need for broader community involvement.

Future directions of the Semantic Web include exploring new applications, improving inference and reasoning capabilities, and developing more efficient and user-friendly tools for ontology creation and management.

Overall, the Semantic Web holds the promise of a more intelligent and interconnected web, where data is not only accessible but also meaningful and actionable for both humans and machines.

The Semantic Web has significant implications for recommender systems, enhancing their capabilities to provide more accurate, personalized, and context-aware recommendations. Semantic Web technologies enable recommender systems to better understand and represent the semantics of user preferences, item characteristics, and contextual information. This leads to improved recommendation quality, better handling of data heterogeneity, and enhanced interoperability between different recommendation sources. Here's how the Semantic Web impacts recommender systems:

- **Rich Data Representation:** Semantic Web technologies, such as RDF and OWL, provide a standardized and flexible way to represent data, including user profiles, item descriptions, and contextual information. By adopting RDF-based representations, recommender systems can model user preferences and item characteristics in a more detailed and structured manner.
- **Linked Data and Data Integration:** Recommender systems can benefit from Linked Data principles, where data from various sources is interlinked, forming a global data network. By integrating and leveraging data from different recommendation sources, such as e-commerce websites, social networks, and content providers, recommender systems can access a wider range of information for making recommendations.
- **Ontologies for Domain Knowledge:** Ontologies can capture domain-specific knowledge and provide a common vocabulary for understanding user preferences and item properties. Recommender systems can leverage ontologies to create more accurate and meaningful recommendations based on the specific context of the domain.
- **Inference and Reasoning:** Semantic Web reasoning engines can be employed to infer implicit relationships and derive new knowledge from the existing data. For example, recommender systems can use reasoning to identify user interests based on inferred relationships between items and users' past interactions.
- **Context-Aware Recommendations:** The Semantic Web allows the integration of contextual information, such as time, location, and user context, into the recommendation process. By considering contextual factors, recommender systems can provide more relevant and timely recommendations.
- **Personalization and Privacy:** Semantic Web technologies can help in enhancing user privacy in recommender systems. Users can control the access and sharing of their data through standardized access control mechanisms provided by RDF-based representations.
- **Hybrid Recommendation Approaches:** The Semantic Web can facilitate the integration of different recommendation techniques, such as collaborative filtering, content-based filtering, and knowledge-based recommendation. Hybrid approaches can combine the strengths of these techniques to provide more accurate and diverse recommendations.
- **Scalability and Interoperability:** Semantic Web technologies offer scalability and interoperability advantages, making it easier to integrate recommender systems into larger ecosystems and data networks.

Incorporating Semantic Web technologies into recommender systems is an active area of research and development. By embracing the principles of the Semantic

Web, recommender systems can take advantage of richer data representations, context-awareness, and interoperability to deliver more effective and personalized recommendations to users.

Let's consider an example of a movie recommender system that utilizes Semantic Web technologies to provide more accurate and personalized movie recommendations.

Assume that the recommender system collects movie data from various sources, such as movie databases, user ratings, and social media, and represents the data using Semantic Web technologies like RDF and ontologies.

**Example Scenario:** Suppose we have the following RDF triples representing some movie data:

**Triple 1:** Subject: Movie1 Predicate: hasTitle Object: "Inception"

**Triple 2:** Subject: Movie1 Predicate: hasGenre Object: "Science Fiction"

**Triple 3:** Subject: Movie2 Predicate: hasTitle Object: "The Dark Knight"

**Triple 4:** Subject: Movie2 Predicate: hasGenre Object: "Action"

**Triple 5:** Subject: User1 Predicate: likesMovie Object: Movie1

**Triple 6:** Subject: User1 Predicate: likesMovie Object: Movie2

With this data, the Semantic Web recommender system can perform the following:

- **Data Integration and Linking:** The system can interlink data from different sources. For example, it can link Movie1 and Movie2 to external movie databases, enriching the movie descriptions with more detailed information like actors, directors, and release dates.
- **Ontologies for Genre-Based Recommendations:** The recommender system can use an ontology defining movie genres and their relationships. By inferring that User1 likes movies of the "Science Fiction" and "Action" genres, the system can recommend other movies with similar genres.
- **Context-Aware Recommendations:** The recommender system can take into account contextual information, such as the time of day or the user's location. For example, if it's a weekend, the system may recommend movies suitable for weekend entertainment.
- **Personalized Recommendations:** By analyzing the user's likes and preferences from RDF triples, the system can create a user profile. It can then recommend movies based on the user's past interactions and inferred interests.
- **Hybrid Recommendations:** The system can use hybrid recommendation approaches. For instance, it can combine collaborative filtering (based on user ratings) with content-based filtering (based on movie genres) to provide more diverse and accurate recommendations.
- **Reasoning for Inferred Interests:** The system can use reasoning to infer user interests. Since User1 likes both "Inception" (Science Fiction) and "The Dark Knight" (Action), it can infer that User1 may also be interested in other Science Fiction or Action movies.

Based on the above information, if User1 likes "Inception" and "The Dark Knight," the recommender system may suggest other movies from the "Science Fiction" and "Action" genres, such as "Interstellar," "Blade Runner," or "Avengers."

This example illustrates how the Semantic Web technologies enable a recommender system to leverage richer data representations, context-awareness, and inference capabilities to provide more relevant and personalized movie recommendations to users.

## 6.5 COLLECTING AND INTEGRATING SPECIALIZED INFORMATION ON THE WEB

Collecting and integrating specialized information from the web involves gathering data from various web sources that focus on a specific domain or topic. The process requires web scraping, data extraction, and data integration techniques to aggregate and combine data from multiple sources into a unified dataset. Here's a general outline of the steps involved:

- **Identify Specialized Sources:** Determine the web sources that provide specialized information relevant to your domain or topic of interest. These sources may include websites, online databases, forums, social media platforms, and other online resources.
- **Web Scraping:** Use web scraping techniques to extract data from the identified web sources. Web scraping involves parsing the HTML content of web pages to extract relevant data, such as text, images, links, and structured data.



- **Data Extraction:** After web scraping, process the extracted data to filter out noise and irrelevant information. Convert the unstructured data into a structured format that can be easily processed and integrated.
- **Data Transformation:** If the data from different sources is in various formats, transform it into a common format, such as CSV, JSON, or RDF, to facilitate integration.
- **Data Integration:** Merge and combine the extracted data from different sources into a unified dataset. The integration process might involve resolving conflicts, handling duplicates, and aligning data with a common schema or ontology.
- **De-duplication:** Ensure that the integrated dataset is free of duplicates, especially when information from multiple sources might overlap.
- **Normalization:** Normalize the data to ensure consistency and standardization across different attributes and entities in the dataset.
- **Entity Linking:** If the data includes references to entities (e.g., names of people, locations, organizations), perform entity linking to connect those references to corresponding entities in a knowledge base or external database.
- **Data Quality Assurance:** Conduct data quality checks to verify the accuracy, completeness, and reliability of the integrated dataset.
- **Storage and Querying:** Store the integrated dataset in a suitable data repository, such as a database, data warehouse, or graph database. Implement query mechanisms to access and retrieve the information efficiently.
- **Updates and Maintenance:** Periodically update the integrated dataset to keep it up-to-date with the latest information from the web sources.

Let's consider an example of collecting and integrating specialized information from different travel websites to create a unified dataset of hotel reviews for a specific destination.

- **Example Scenario:** Suppose we want to create a dataset of hotel reviews for the city of "New York." We identify three specialized travel websites that provide hotel reviews: "TripAdvisor," "Booking.com," and "Expedia."
- **Web Scraping:** We use web scraping techniques to extract hotel review data from each of the identified websites. For each hotel in New York, we collect information such as hotel name, review rating, review text, reviewer name, and review date.
- **Data Extraction:** After scraping, we process the extracted data to filter out irrelevant information and clean the text data. We ensure that each review is associated with the correct hotel and remove any noise from the scraped content.
- **Data Transformation:** If the data from the three websites is in different formats, we transform it into a common format, such as JSON, to facilitate integration. For example:

**Review 1:**

```
{
  "website": "TripAdvisor",
  "hotel_name": "Hotel A",
  "rating": 4.5,
  "review_text": "Great hotel, excellent service!",
  "reviewer_name": "John Smith",
  "review_date": "2023-07-25"
}
```

**Review 2:**

```
{
  "website": "Booking.com",
  "hotel_name": "Hotel B",
  "rating": 3.8,
  "review_text": "Average stay, could be better.",
  "reviewer_name": "Emily Johnson",
  "review_date": "2023-07-22"
}
...
```

- **Data Integration:** We merge the extracted data from TripAdvisor, Booking.com, and Expedia into a unified dataset. The integrated dataset includes all the relevant hotel reviews for hotels in New York from the three sources.
- **De-Duplication:** We ensure that the dataset is free of duplicate reviews. If the same review appears on multiple websites, we keep only one instance of it in the dataset.

- **Normalization:** We ensure that the data is normalized, such as converting review ratings to a common scale (e.g., 1 to 5) and ensuring consistent date formats.
- **Entity Linking:** We perform entity linking to connect hotel names and reviewer names to corresponding entities in a knowledge base or external database, improving data consistency.
- **Data Quality Assurance:** We conduct data quality checks to verify the accuracy and completeness of the integrated dataset. This may involve identifying and handling missing or erroneous data.
- **Storage and Querying:** We store the integrated dataset in a database, allowing for efficient querying and retrieval of hotel reviews for various analysis and recommendation tasks.
- **Updates and Maintenance:** To keep the dataset up-to-date, we periodically re-scrape the websites and update the integrated dataset with the latest hotel reviews.

The resulting integrated dataset provides a comprehensive collection of hotel reviews for New York, aggregated from multiple specialized sources, and can be used for various applications, such as sentiment analysis, recommendation systems, and tourism analytics.

## CASE STUDY: DEMONSTRATE COLLABORATIVE FILTERING USING ANY DATASETS TO RECOMMEND ITEMS TO USERS

For this demonstration, let's assume we have a dataset where users have rated movies. We'll use this dataset to recommend movies to a specific user based on the preferences of similar users.

**Dataset:**

Here's a simplified example of the movie ratings dataset:

| User | Movie A | Movie B | Movie C | Movie D | Movie E |
|---|---|---|---|---|---|
| User 1 | 5 | 4 | - | 2 | - |
| User 2 | - | 3 | 2 | - | 4 |
| User 3 | 4 | - | 5 | 1 | - |
| User 4 | - | 2 | 1 | 3 | 5 |

In this example, users have rated movies on a scale from 1 to 5, with "1" being the lowest rating and "5" being the highest rating. "-" indicates that a user hasn't rated a particular movie.

**Steps:**

1. **User Similarity Calculation:**
   - Calculate the similarity between users based on their movie ratings. Common similarity measures include cosine similarity or Pearson correlation.
   - For instance, you can calculate the cosine similarity between User 1 and User 2 based on their ratings for Movie B and Movie D.
2. **Find Similar Users:**
   - Identify users who are similar to the target user (for whom you want to make recommendations).
   - Let's assume User 3 is our target user, and we find that User 1 and User 4 are similar to User 3.
3. **Item Recommendation:**
   - Recommend items (movies) to the target user based on what the similar users have liked.
   - For example, if User 1 and User 4 have rated Movie A highly and User 2 has rated Movie C highly, we can recommend Movie A and Movie C to User 3.

**Outcome:**

In this example, collaborative filtering recommends Movie A and Movie C to User 3 based on the preferences of similar users (User 1 and User 4 for Movie A, and User 2 for Movie C).

In practice, collaborative filtering algorithms scale to much larger datasets and use more sophisticated methods to calculate user similarities and generate recommendations. Common techniques include user-based collaborative filtering and item-based collaborative filtering. Additionally, matrix factorization methods and machine learning models can be applied to enhance recommendation accuracy.

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Create a sample movie ratings dataset
data = {
    'User': ['User1', 'User2', 'User3', 'User4'],
    'MovieA': [5, None, 4, None],
    'MovieB': [4, 3, None, 2],
    'MovieC': [None, 2, 5, 1],
    'MovieD': [2, None, 1, 3],
    'MovieE': [None, 4, None, 5]
}

df = pd.DataFrame(data)

# Fill missing values with 0
df.fillna(0, inplace=True)

# Calculate cosine similarity between users
user_similarity = cosine_similarity(df.iloc[:, 1:])

# Target user (index 2) and its similarity to other users
target_user_similarities = user_similarity[2, :]

# Find similar users (excluding target user itself)
similar_users =
df['User'][target_user_similarities.argsort()[::-1][1:]]

# Recommend movies based on what similar users liked
recommended_movies = df[similar_users].mean()
recommended_movies =
recommended_movies.sort_values(ascending=False)

print("Recommended movies:")
print(recommended_movies)
```



## EXERCISE

1. Define information extraction and explain its importance in knowledge-based recommendation systems.
2. What are the key steps involved in data extraction from text, and how can natural language processing techniques be used for this purpose?
3. Explain the concept of the Semantic Web and how it aims to enhance the understanding and accessibility of web content.
4. Describe the role of RDF in the Semantic Web and provide an example of representing data using RDF triples.
5. How can ontology-based knowledge representation benefit recommender systems? Provide an example of using ontologies for genre-based movie recommendations.
6. Discuss the advantages of using the Semantic Web in recommender systems to provide personalized and context-aware recommendations.
7. How does the Linked Data principle contribute to data integration and interoperability in the Semantic Web context?
8. Describe the process of web scraping and data extraction for collecting specialized information from the web.
9. What are the challenges faced in collecting and integrating specialized information from various web sources?
10. Explain the importance of entity linking in data integration and how it helps to maintain data consistency.
11. Describe the concept of collaborative filtering and how it can be combined with content-based recommendation in hybrid recommender systems.
12. How can the Semantic Web technologies facilitate the integration of different recommendation approaches, such as collaborative filtering and content-based filtering?
13. Discuss the benefits of using hybrid recommendation approaches in recommender systems.
14. Provide an example of context-aware recommendations and explain how the Semantic Web can be utilized for such recommendations.
15. How can the Semantic Web improve the explanation and interpretability of recommendations to users?
16. Discuss the challenges faced in ensuring data quality in knowledge-based recommendation systems.

17. Explain how the Semantic Web can contribute to handling the cold start problem in recommendation systems.
18. Create an RDF triple representing a user's preference for a movie and use it to demonstrate personalized movie recommendations based on a hybrid recommendation approach.
19. Describe the role of inference and reasoning in the Semantic Web context and how it can enhance the quality of recommendations.
20. In the context of the Semantic Web, how can the use of on tologies benefit the process of integrating specialized information for recommendation systems?

✠ ✠ ✠

# MODEL QUESTION PAPERS

## In-Semester Examination

**Time: 1 Hour** Max. Marks: 30

**Instructions to the candidates:**
(1) Answer Q.1 or Q.2, Q.3 or Q.4.
(2) Figures to the right side indicate full marks.
(3) Neat diagrams must be drawn wherever necessary.
(4) Assume suitable data, if necessary.

1. **(a)** Explain the key processes involved in Information Retrieval. (5)
   **(b)** What are the advantages of Latent Semantic Indexing in capturing document semantics? (5)
   **(c)** Explain the concept of term weighting in the Vector Space Model. (5)

**OR**

2. **(a)** Describe the key components of the Document subsystem in the Acquisition process of Information Retrieval. (5)
   **(b)** What is the purpose of user feedback in the Information Retrieval process, and how can it improve system performance? (5)
   **(c)** Compare and contrast the Vector Space Model and the Probabilistic Model in Information Retrieval. (5)
3. **(a)** What are the different types of dictionaries? Provide examples of each type. (5)
   **(b)** Describe the interleaving technique in dictionaries and how it improves query performance. (5)
   **(c)** Explain how posting lists are used in dictionaries and query processing. (5)

**OR**

4. **(a)** Explain the concept of an inverted index and its significance in Information Retrieval systems. (5)
   **(b)** Compare and contrast static inverted index and dynamic inverted index. (5)
   **(c)** How does precomputing score contributions optimize query processing in IR systems? (5)

## End-Semester Examination

**Time : 3 Hours** Max. Marks : 70

**Instruction to candidates :**
(1) Answer Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.
(2) Figures to the right side indicate full marks.
(3) Neat diagrams should be drawn wherever necessary.
(4) Assume suitable data, if necessary.

1. **(a)** How does LSI address the curse of dimensionality in high-dimensional term-document matrices? (6)
   **(b)** How do advances in deep learning, word embeddings, and semantic search impact Information Retrieval? (6)
   **(c)** Discuss the working principles of hash-based dictionaries and their applications in IR systems. (6)

**OR**

2. **(a)** Discuss the advantages and disadvantages of dynamic indexing in IR systems. (6)
   **(b)** How does precomputing score contributions optimize query processing in IR systems? (6)
   **(c)** Describe the Latent Semantic Indexing (LSI) model in Information Retrieval and its purpose. (6)